Chad Galloway
CST-250 Programming in C# II
Grand Canyon University
Nov. 09, 2025
Activity 3

Files
https://github.com/CGalloway3/CST-250-Projects/tree/master/Activity3

Video
https://www.loom.com/share/6e68231da7924e208b7aa5dbfced8d52

PART 1

Flow Chart

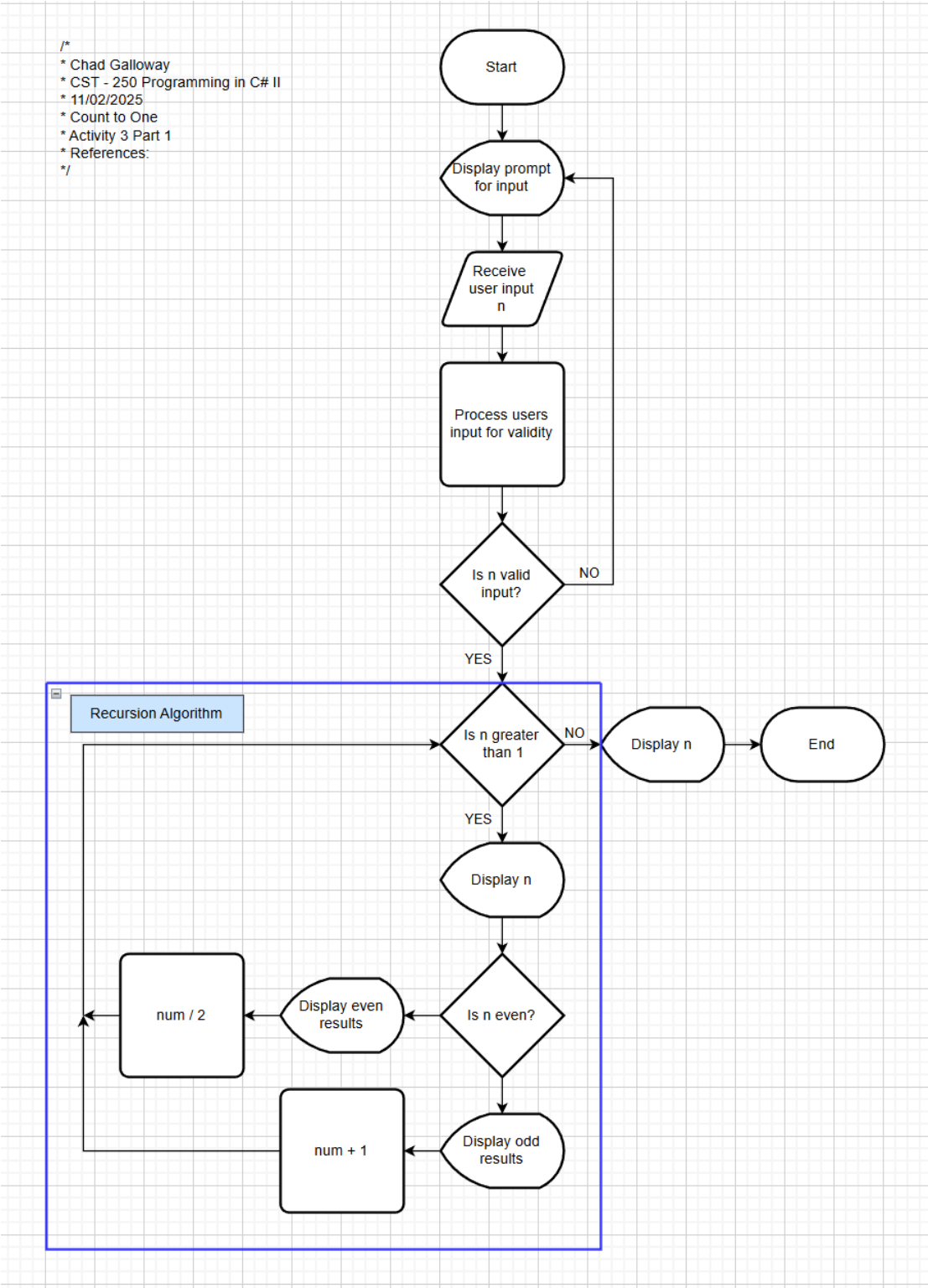


Figure 1-1: Flow chart of Activity 3 part one, count to one recursion console application.

UML Class Diagram

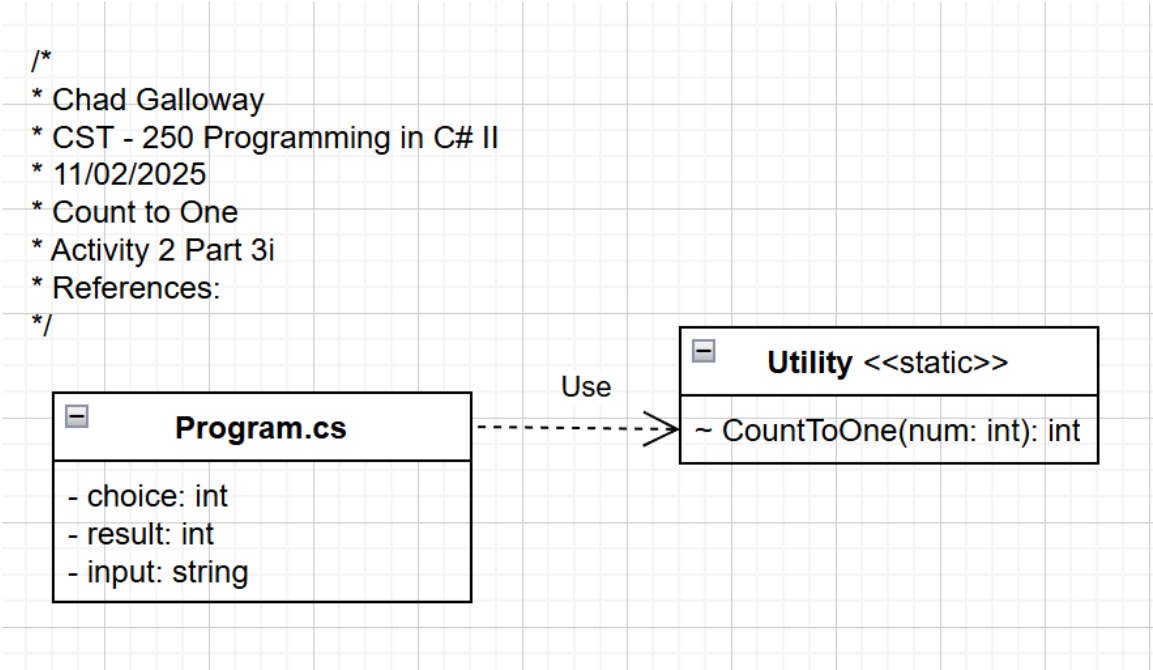


Figure 1-2: UML Class Diagram for Activity 3 part one, count to one recursion console application.

Screen Shots

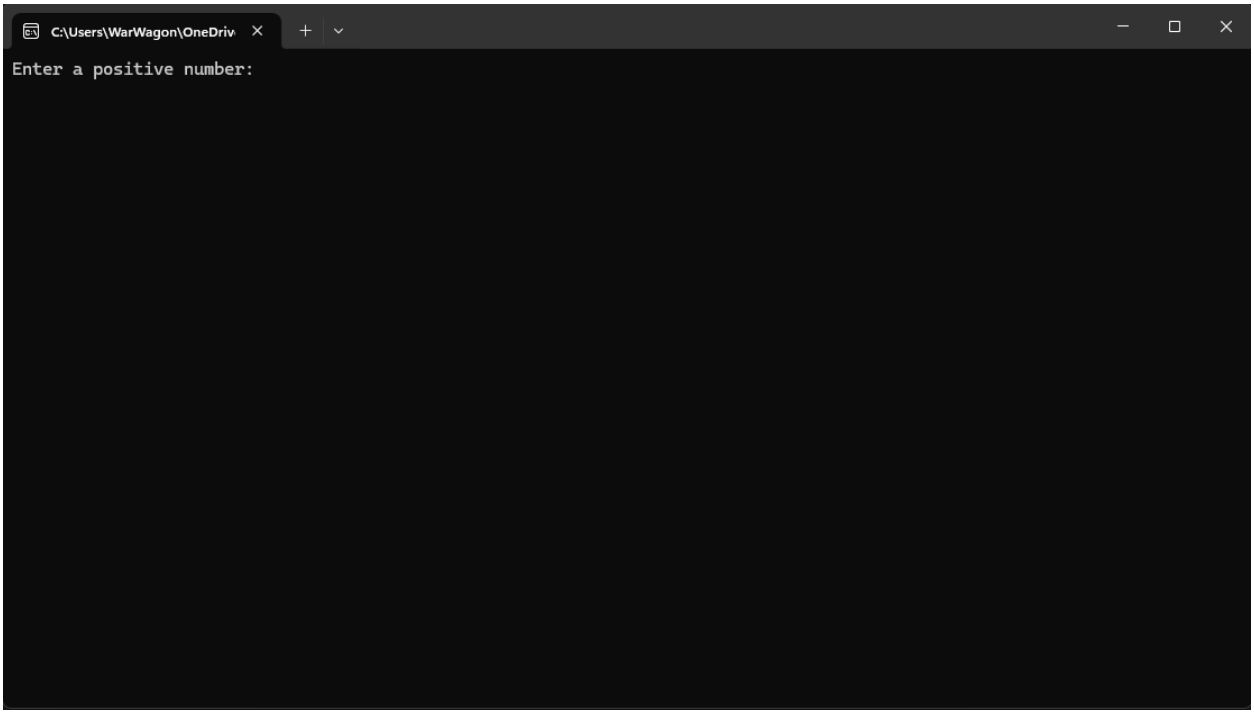


Figure 1-3: Part 1 App Running in Initial load state.

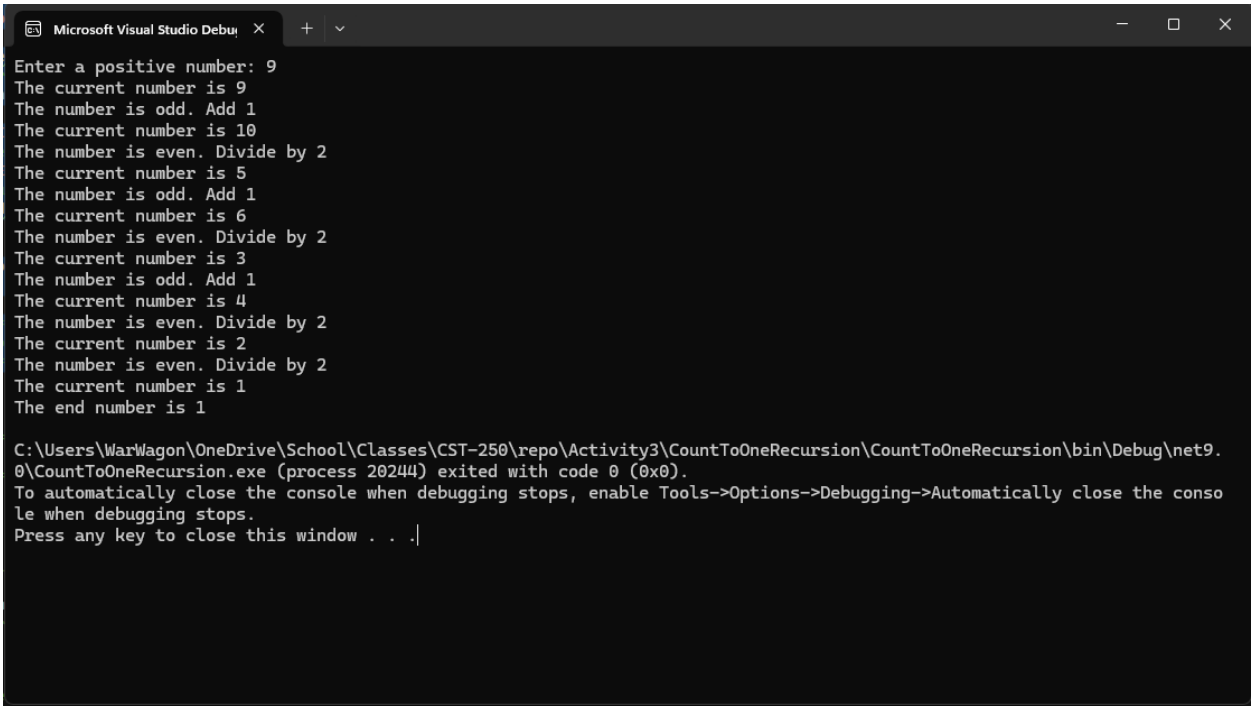


Figure 1-4: Screenshot of successful output

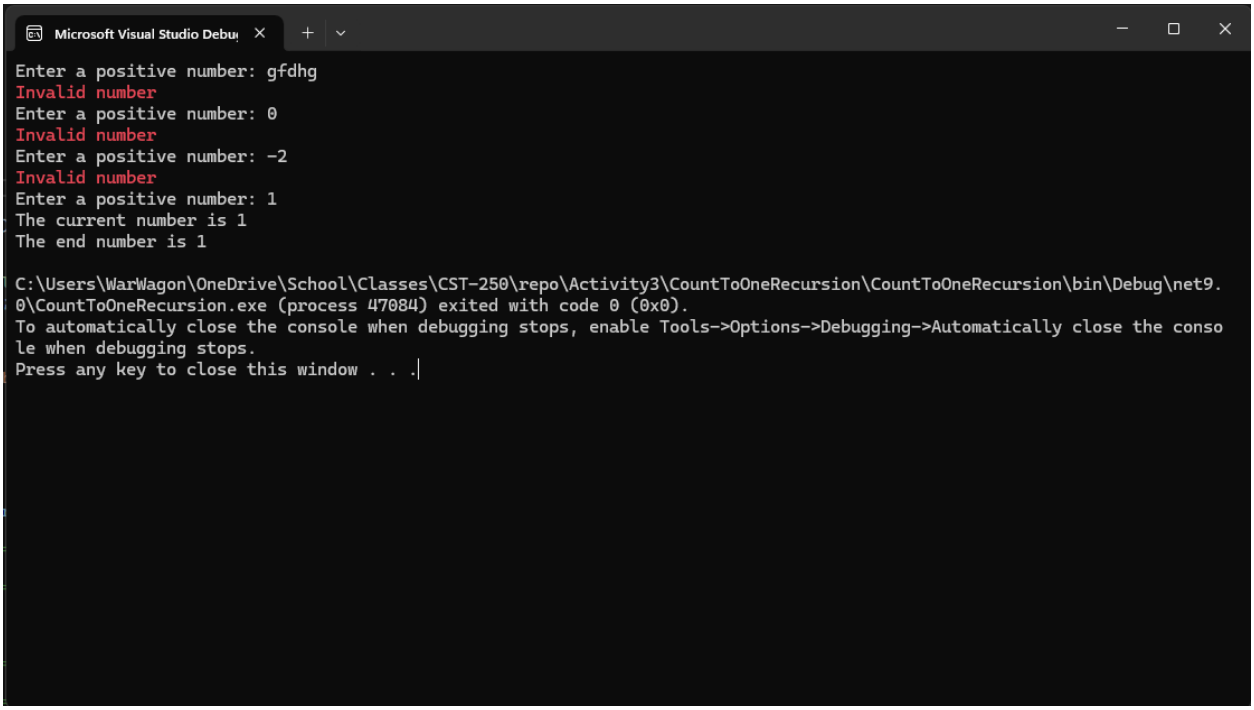
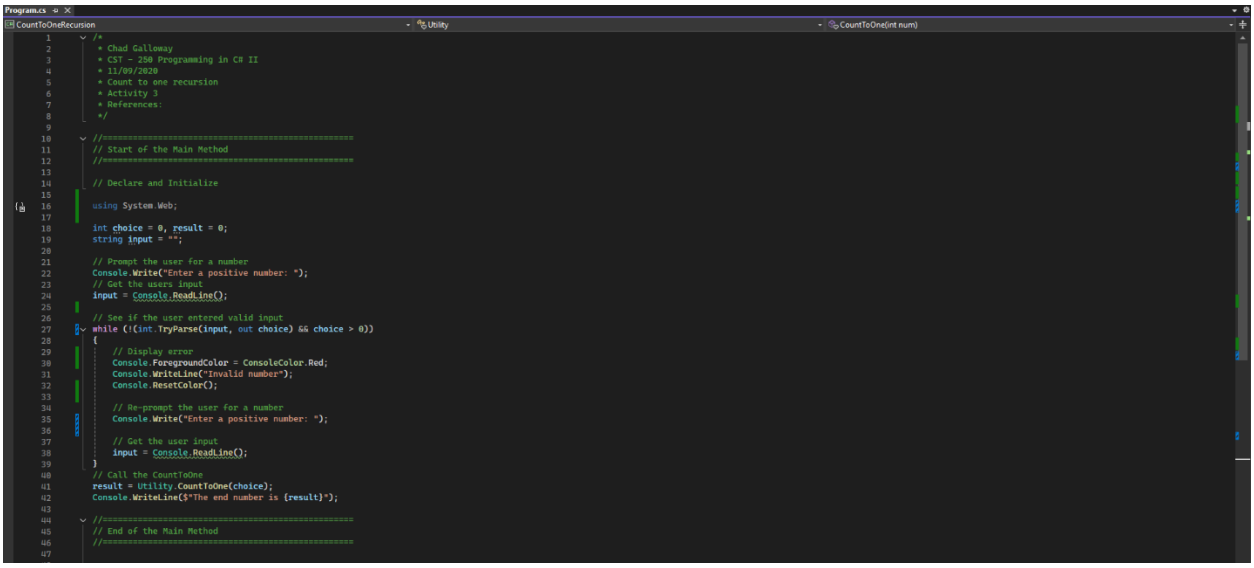


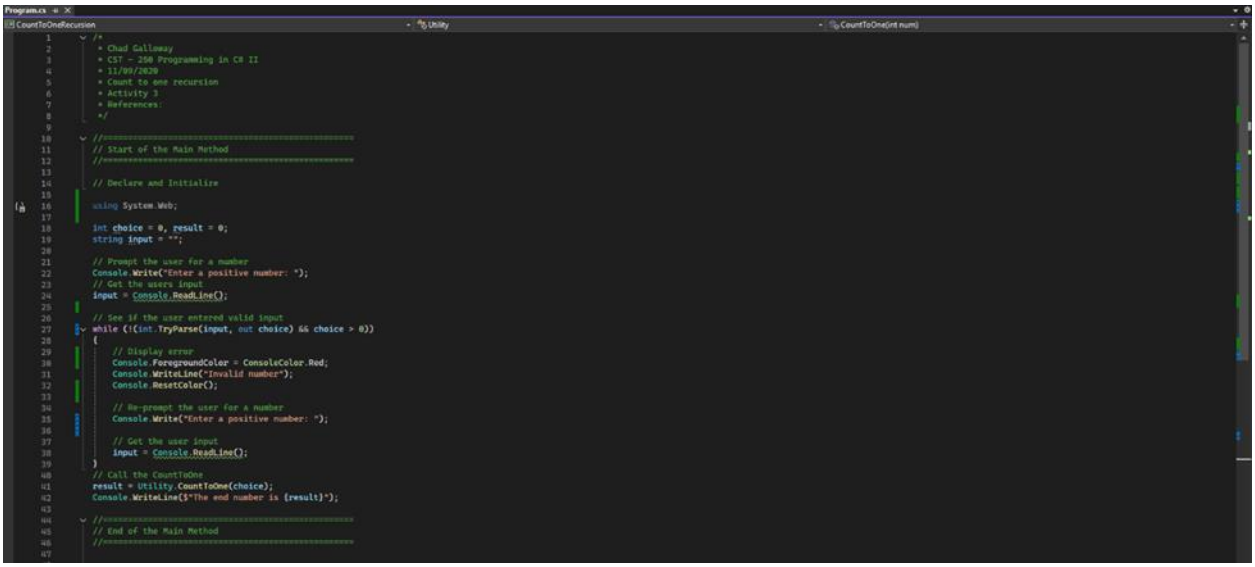
Figure 1-5: Screenshot of Error Handling

Code



```
/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/09/2020
 * Count to one recursion
 * Activity 3
 * References:
 */

//================================================
// Start of the Main Method
//================================================

// Declare and Initialize

using System.Web;

int choice = 0, result = 0;
string input = "";

// Prompt the user for a number
Console.Write("Enter a positive number: ");
// Get the users input
input = Console.ReadLine();

// See if the user entered valid input
while (!(int.TryParse(input, out choice) && choice > 0))
{
    // Display error
    Console.ForegroundColor = ConsoleColor.Red;
    Console.WriteLine("Invalid number");
    Console.ResetColor();

    // Re-prompt the user for a number
    Console.Write("Enter a positive number: ");

    // Get the user input
    input = Console.ReadLine();
}
// Call the CountToOne
result = Utility.CountToOne(choice);
Console.WriteLine($"The end number is {result}");

//================================================
// End of the Main Method
//================================================
```

Figure 1-6: Code Citations and the main method.



```
/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/09/2020
 * Count to one recursion
 * Activity 3
 * References:
 */

//================================================
// Start of the Main Method
//================================================

// Declare and Initialize

using System.Web;

int choice = 0, result = 0;
string input = "";

// Prompt the user for a number
Console.Write("Enter a positive number: ");
// Get the users input
input = Console.ReadLine();

// See if the user entered valid input
while (!(int.TryParse(input, out choice) && choice > 0))
{
    // Display error
    Console.ForegroundColor = ConsoleColor.Red;
    Console.WriteLine("Invalid number");
    Console.ResetColor();

    // Re-prompt the user for a number
    Console.Write("Enter a positive number: ");

    // Get the user input
    input = Console.ReadLine();
}
// Call the CountToOne
result = Utility.CountToOne(choice);
Console.WriteLine($"The end number is {result}");

//================================================
// End of the Main Method
//================================================
```

Figure 1-7: Utility class and the count to one method.

PART 1 CHALLENGES

Screen Shots

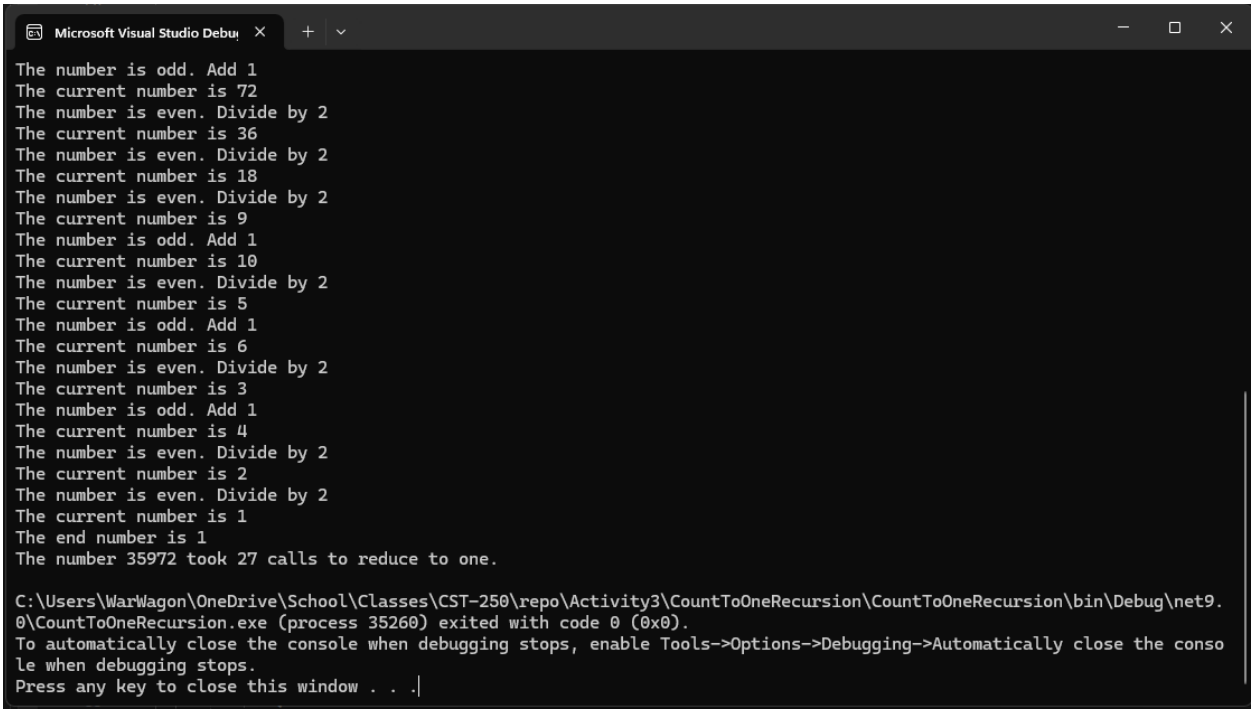


Figure 1-8: Task 1 challenge successful program completion.

Code

```
39      }
40      // Call the CountToOne
41      Utility.ResetCallCount();
42      result = Utility.CountToOne(choice);
43      Console.WriteLine($"The end number is {result}");
44      Console.WriteLine($"The number {choice} took {Utility.GetCount()} calls to reduce to one.");
45
46      //================================================
47      // End of the Main Method
```

Figure 1-9: Changes to the main method for task 1 challenge.

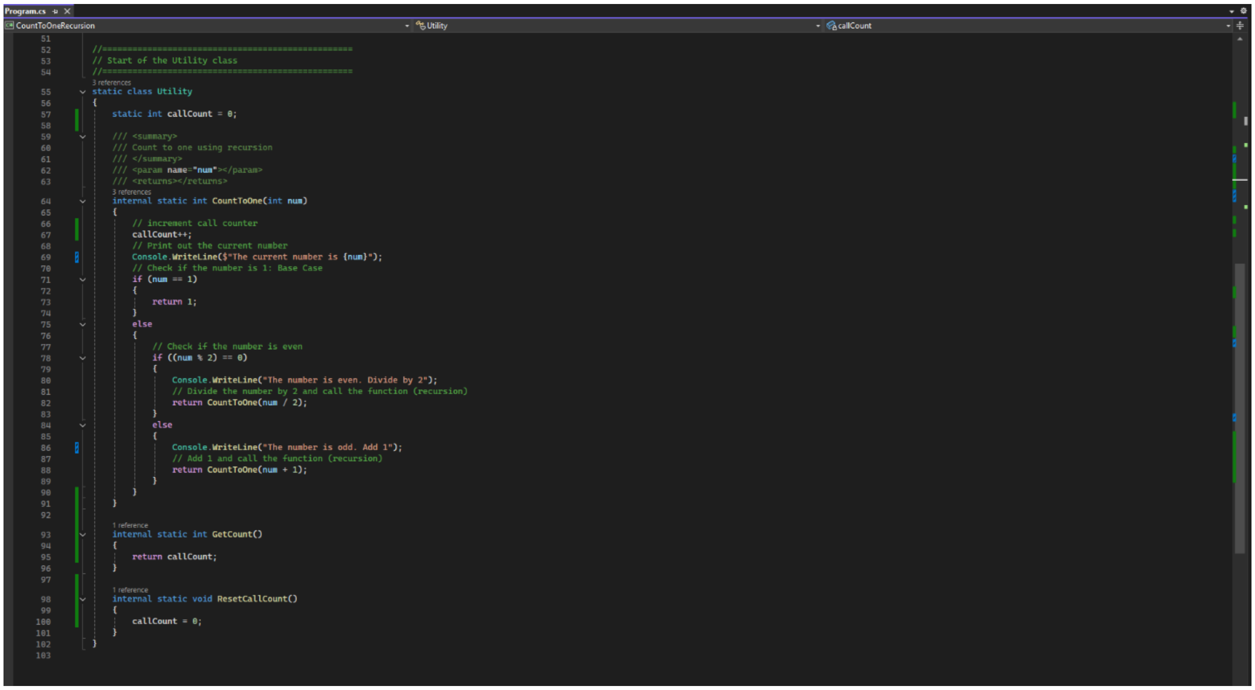


Figure 1-10: Changes to the Utility class for task 1 challenge.

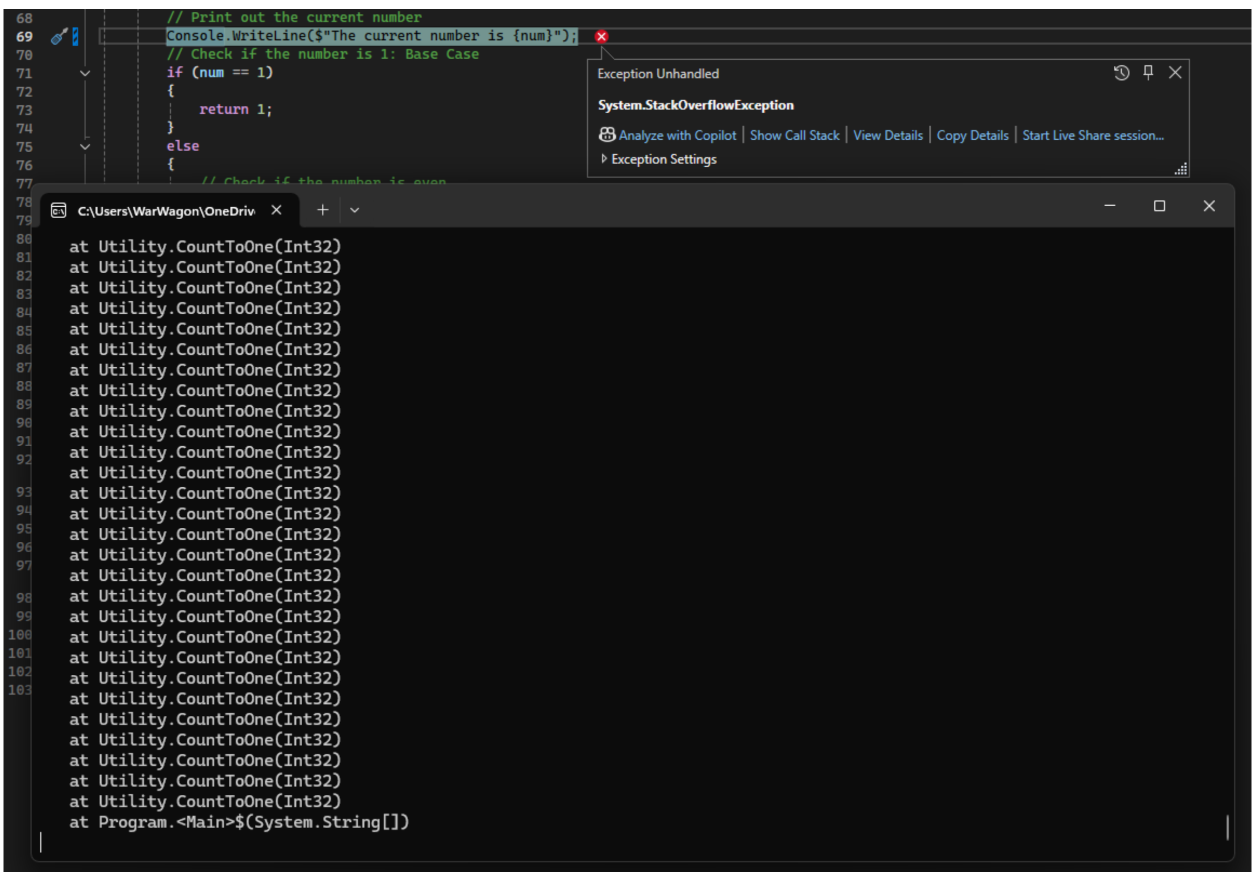


Figure 1-11: Changing to add two to odd numbers causes a stack overflow. Adding 2 to an odd number keeps the number odd forever and cause the recursion to loop until it crashes

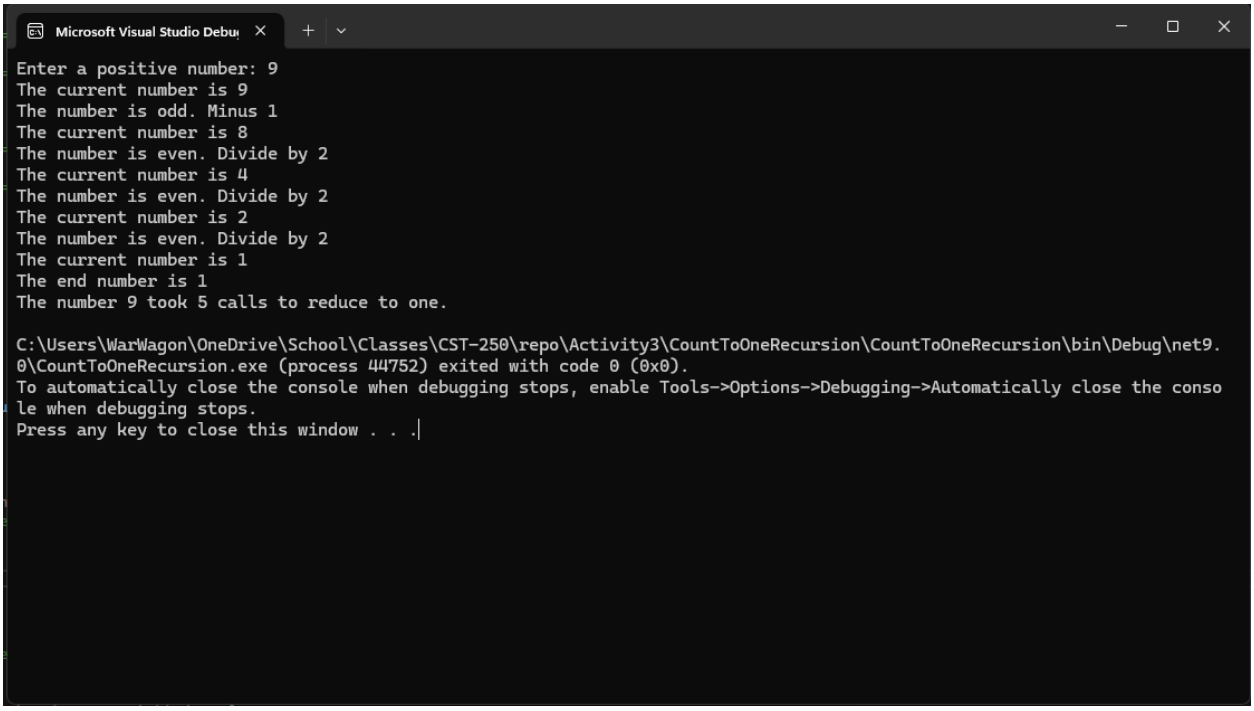


Figure 1-12: Changing to “minus one” runs successfully but the number of calls is reduced. In previous runs entering “9” would make eight calls using the plus one odd number adjustments. Using minus one does not adversely affect the programs operation but it reduced those eight calls to five. With larger numbers (35,972) this variation is less significant at ~ 25% but here in cases of low numbers it is an almost ~38% reduction in call overhead.

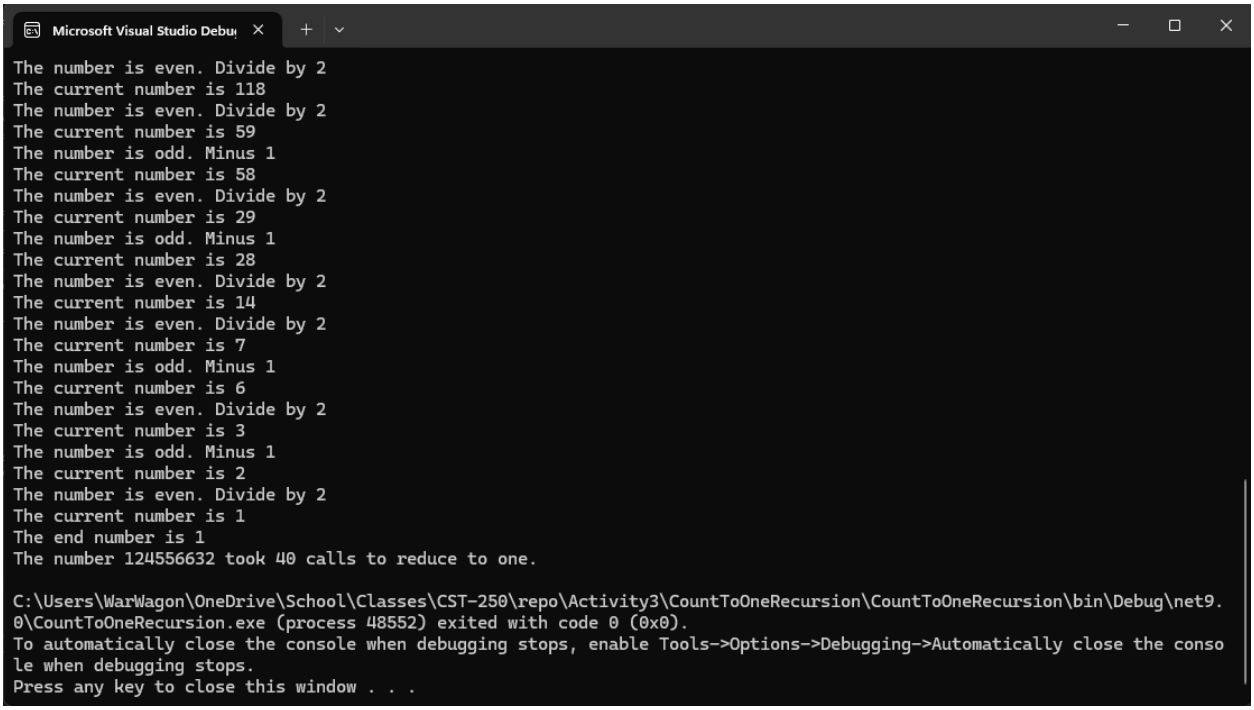


Figure 1-13: Strangely, in the cases of 124,556,632 the number of calls using minus one increased by one compared to using plus one.

One thing that Figure 1-12 and 1-13 clearly demonstrate is don't be afraid to try different methods of adjusting outliers in your recursion loop. A simple thing like flipping a plus sign to minus had a massive effect on some of the lower number's computational efficiency.

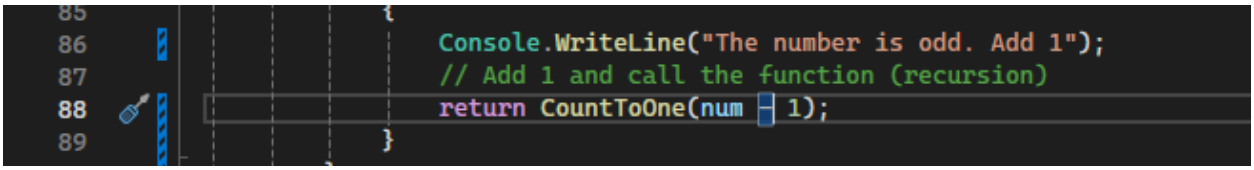


Figure 1-14: The only value changed to go from crashing to a reduction in calls was the number at the tail end of the return call to CountToOne

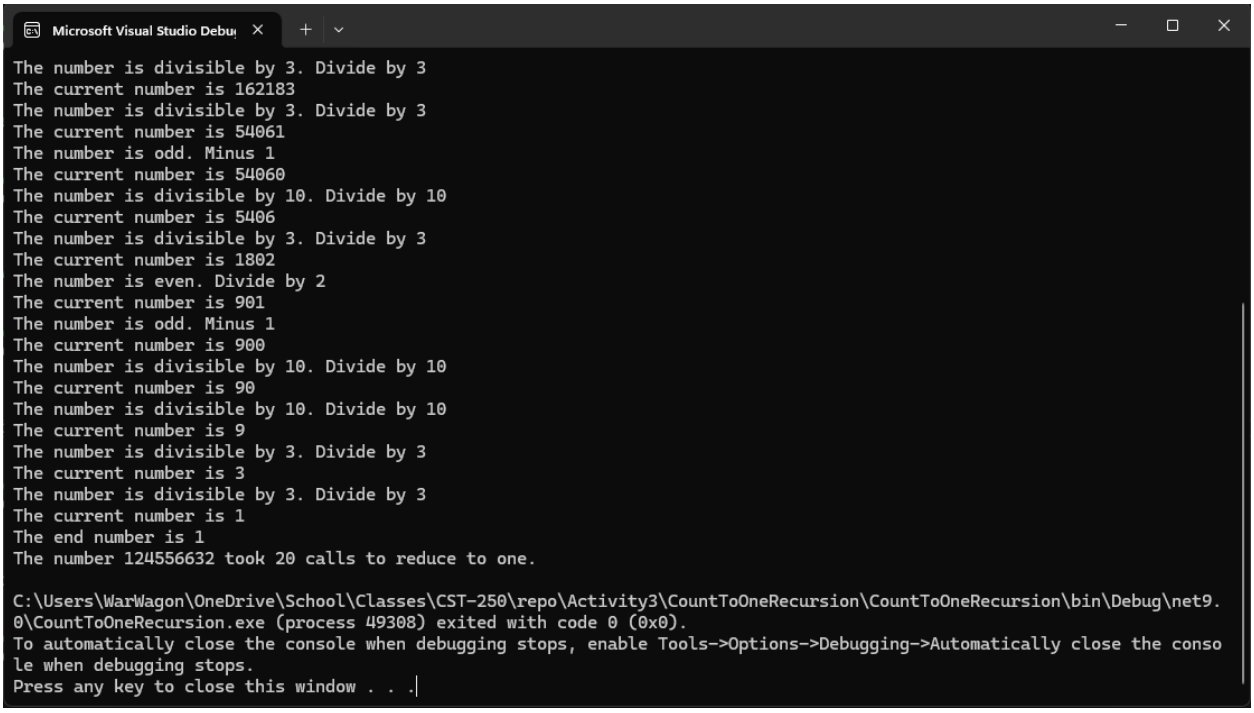
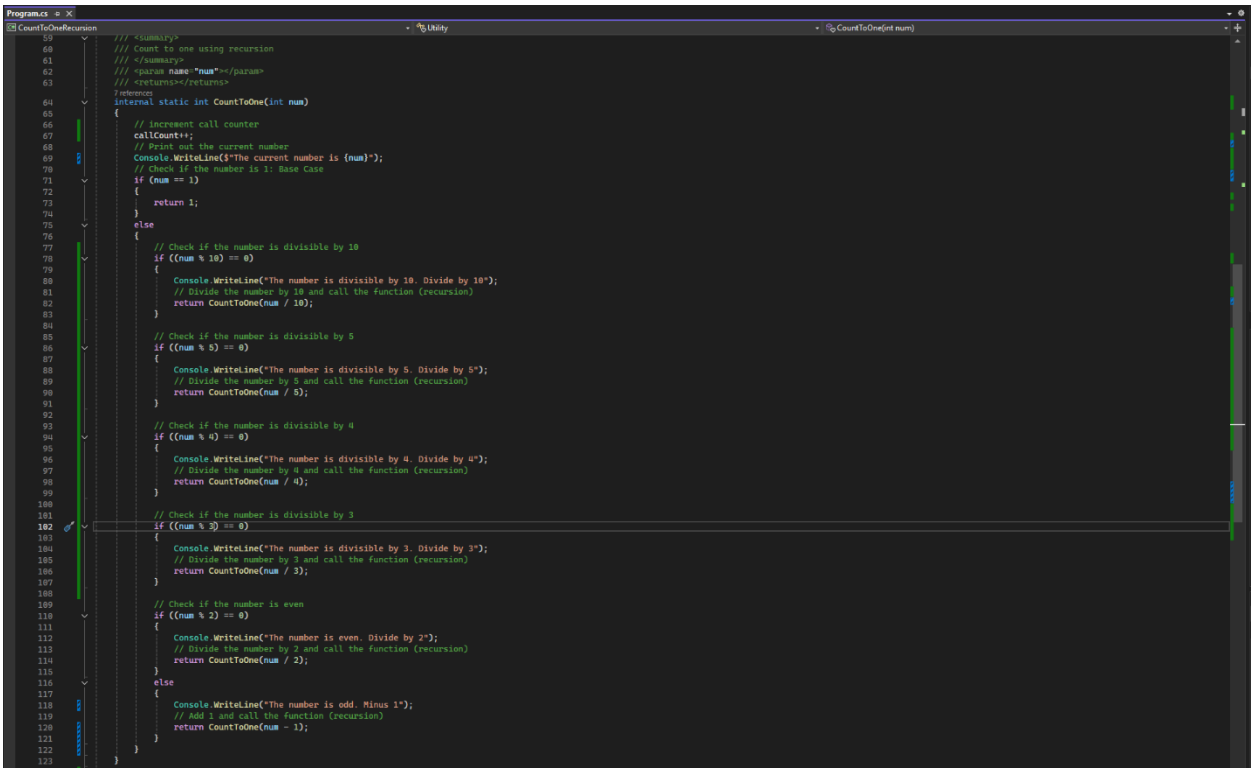
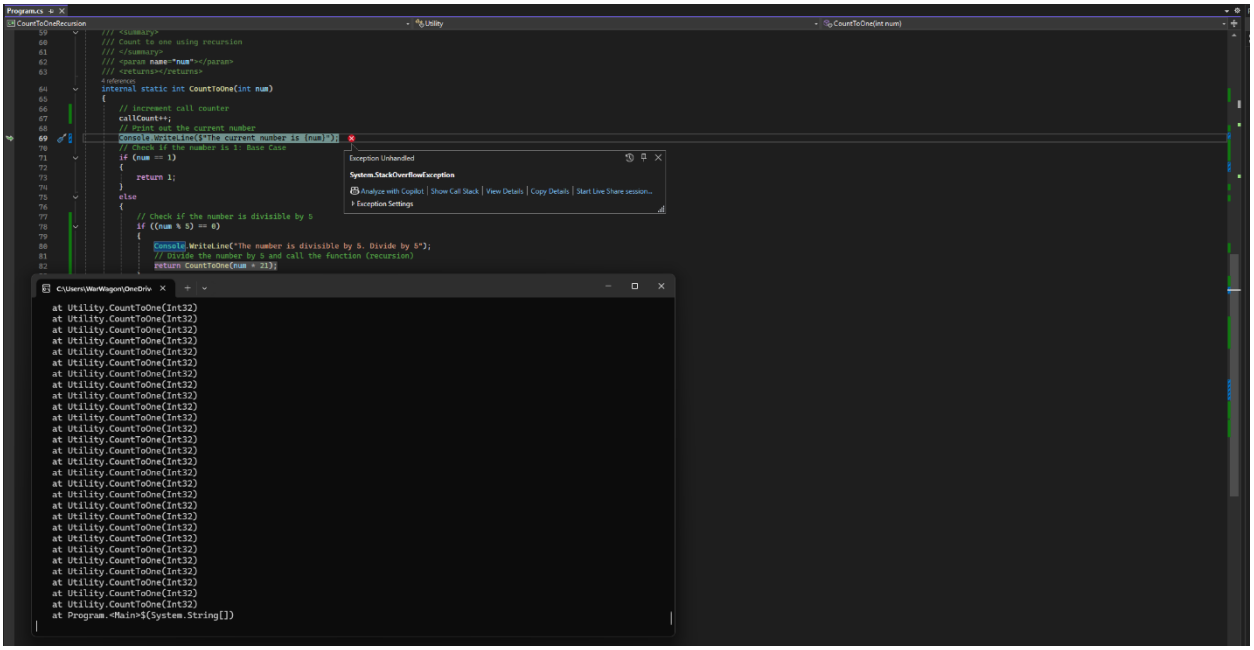


Figure 1-15: Extra Divisions added at mod 10, 5, 4, and 3 reduced the method calls to 20 for task 2 in the challenges list.



```
/// <summary>
/// Count to one using recursion
/// </summary>
/// <param name="num"></param>
/// <returns></returns>
internal static int CountToOne(int num)
{
    // increment call counter
    callCount++;
    // Print out the current number
    Console.WriteLine($"The current number is {num}");
    // Check if the number is 1: Base Case
    if (num == 1)
    {
        return 1;
    }
    else
    {
        // Check if the number is divisible by 10
        if ((num % 10) == 0)
        {
            Console.WriteLine("The number is divisible by 10. Divide by 10");
            // Divide the number by 10 and call the function (recursion)
            return CountToOne(num / 10);
        }

        // Check if the number is divisible by 5
        if ((num % 5) == 0)
        {
            Console.WriteLine("The number is divisible by 5. Divide by 5");
            // Divide the number by 5 and call the function (recursion)
            return CountToOne(num / 5);
        }

        // Check if the number is divisible by 4
        if ((num % 4) == 0)
        {
            Console.WriteLine("The number is divisible by 4. Divide by 4");
            // Divide the number by 4 and call the function (recursion)
            return CountToOne(num / 4);
        }

        // Check if the number is divisible by 3
        if ((num % 3) == 0)
        {
            Console.WriteLine("The number is divisible by 3. Divide by 3");
            // Divide the number by 3 and call the function (recursion)
            return CountToOne(num / 3);
        }

        // Check if the number is even
        if ((num % 2) == 0)
        {
            Console.WriteLine("The number is even. Divide by 2");
            // Divide the number by 2 and call the function (recursion)
            return CountToOne(num / 2);
        }
        else
        {
            Console.WriteLine("The number is odd. Minus 1");
            // Add 1 and call the function (recursion)
            return CountToOne(num - 1);
        }
    }
}
```

Figure 1-16: Code changes to get the extra divisions running in the CountToOne() method.



```
/// <summary>
/// Count to one using recursion
/// </summary>
/// <param name="num"></param>
/// <returns></returns>
internal static int CountToOne(int num)
{
    // increment call counter
    callCount++;
    // Print out the current number
    Console.WriteLine($"The current number is {num}");
    // Check if the number is 1: Base Case
    if (num == 1)
    {
        return 1;
    }
    else
    {
        // Check if the number is divisible by 5
        if ((num % 5) == 0)
        {
            Console.WriteLine("The number is divisible by 5. Divide by 5");
            // Divide the number by 5 and call the function (recursion)
            return CountToOne(num * 21);
```

Exception Unhandled

System.StackOverflowException

```
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Utility.CountToOne(Int32)
   at Program.<Main>$(System.String[])
```

Figure 1-17: Trying task four we run into a stack overflow error, same as we did when we tried adding two to odd numbers. When the number becomes divisible by 5 it is then multiplied by a factor of 21 and it is then still divisible by 5 so the recursion loop runs indefinitely until we get an overflow. We need to be careful of these cases in recursion.

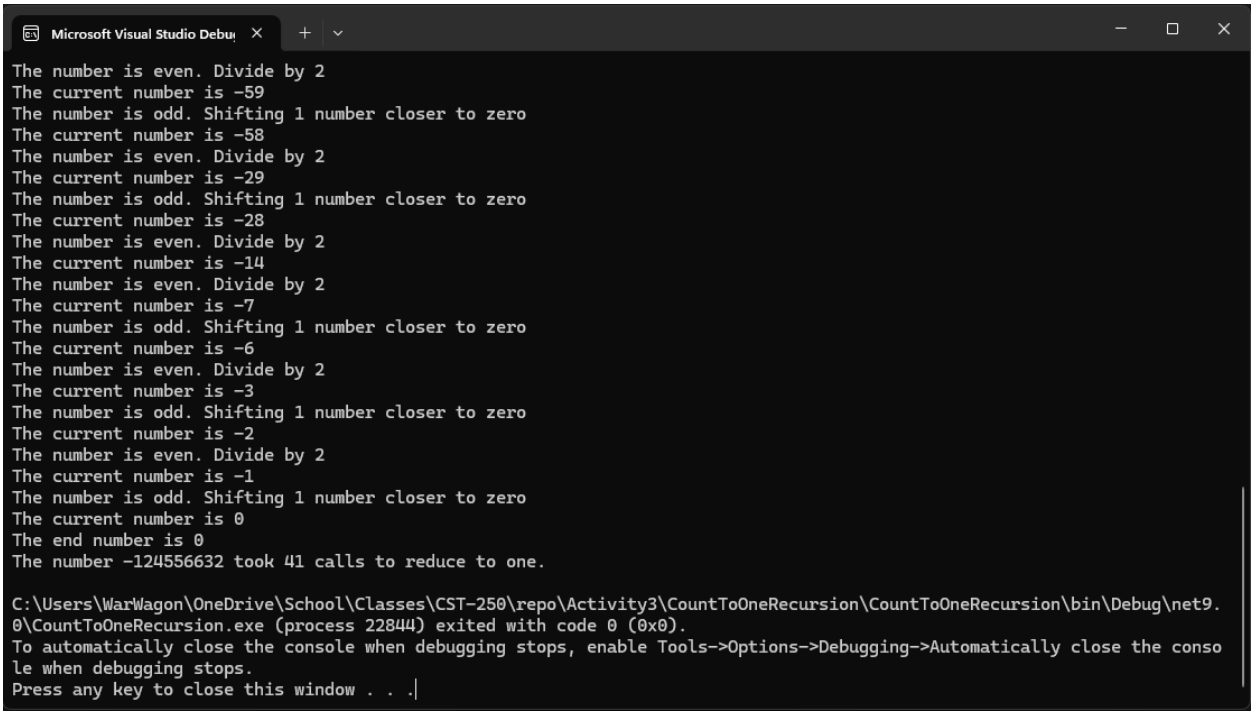


Figure 1-18: Running with negative number and zero tolerance.

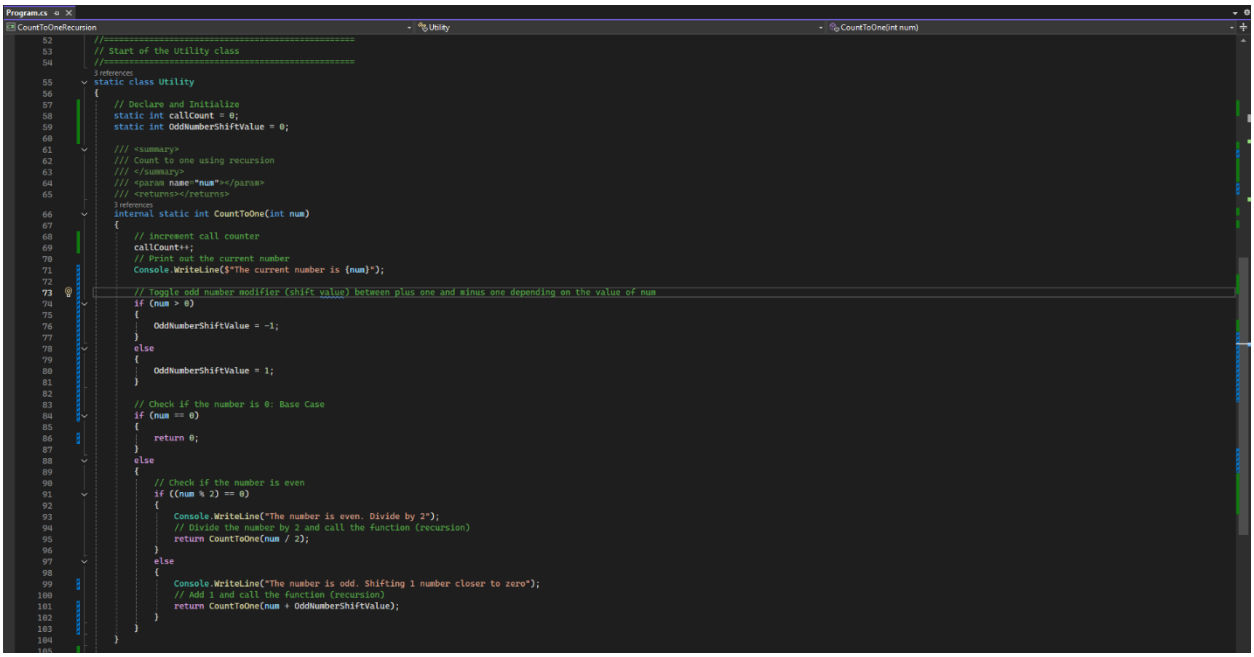


Figure 1-19: Code changes to count to one for negative numbers

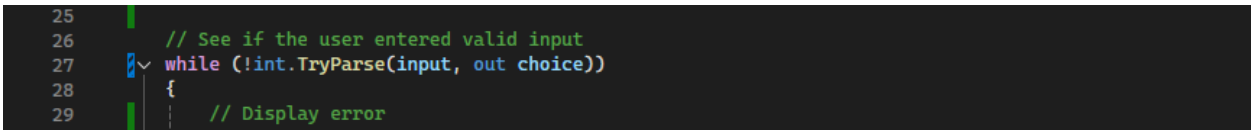


Figure 1-20: Slightly altered error checking in main method to allow negative numbers

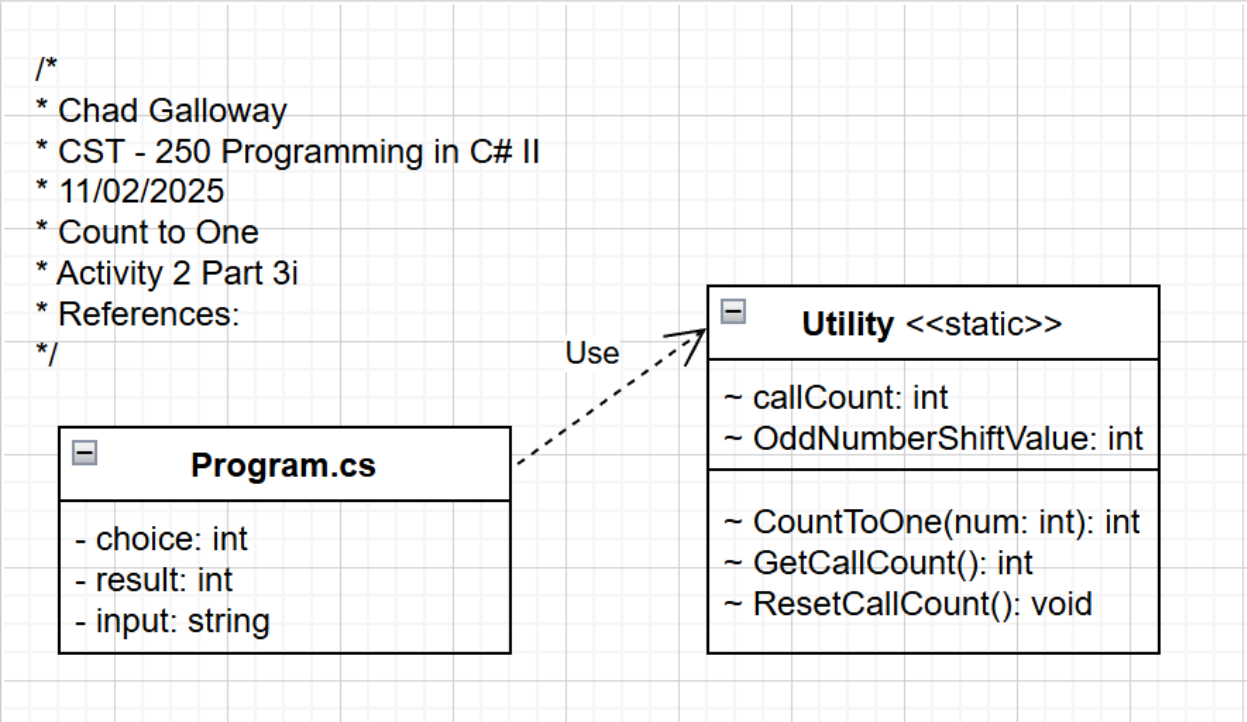


Figure 1-21: Final UML for the count to one console application

PART 2

Flow Chart

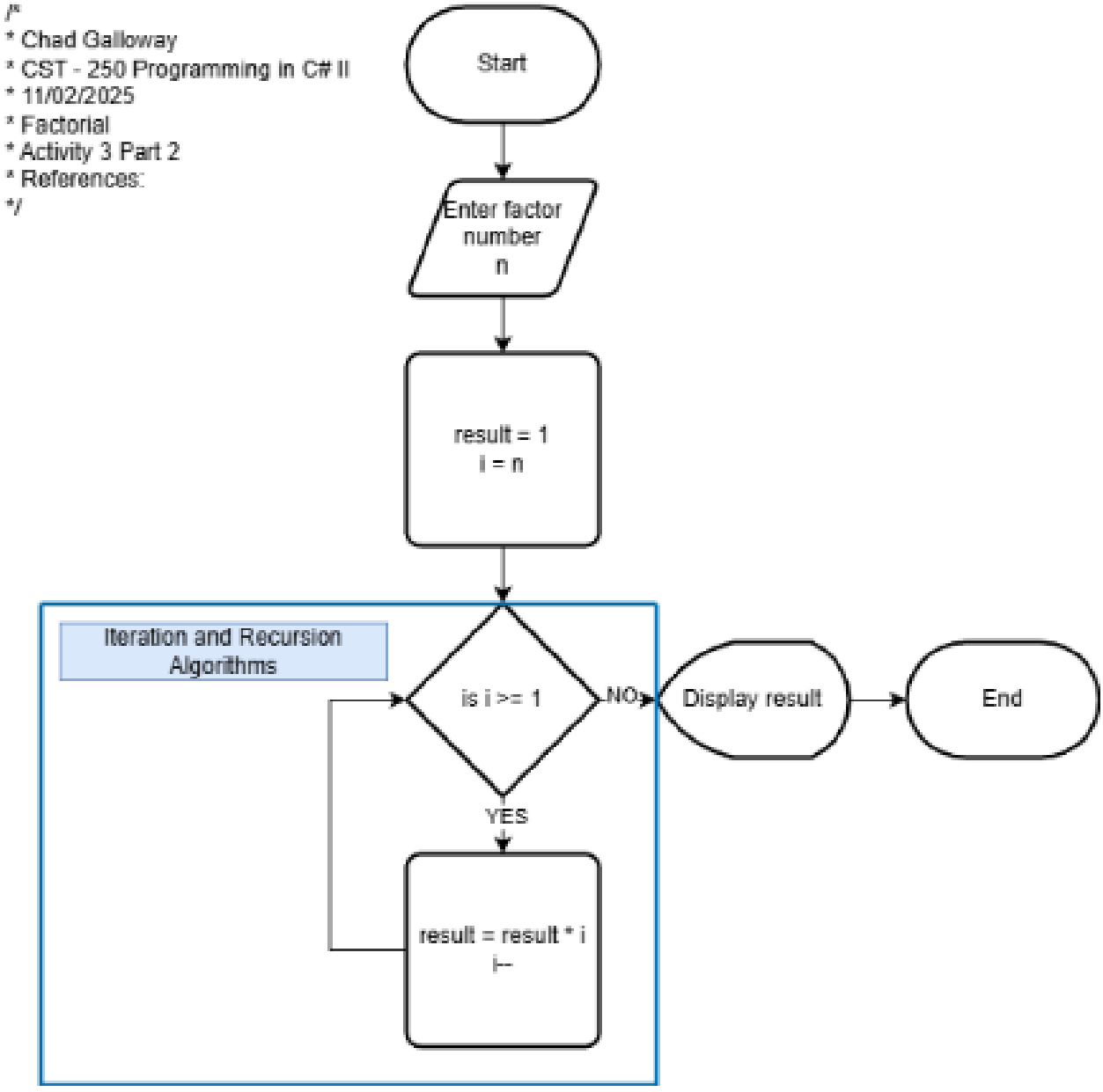


Figure 2-1: Flow chart of Activity 3 Part 2 Factorial recursion

UML Class Diagram

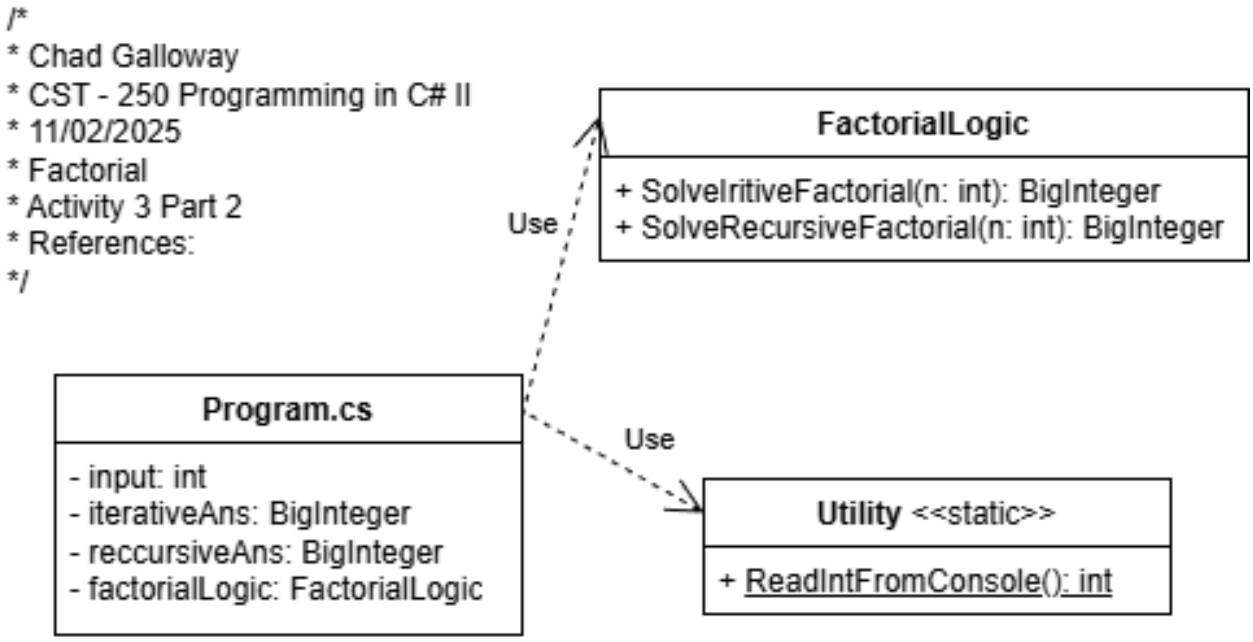


Figure 2-2: UML Class Diagram of Activity 3 Part 2 Factorial recursion

Screen Shots

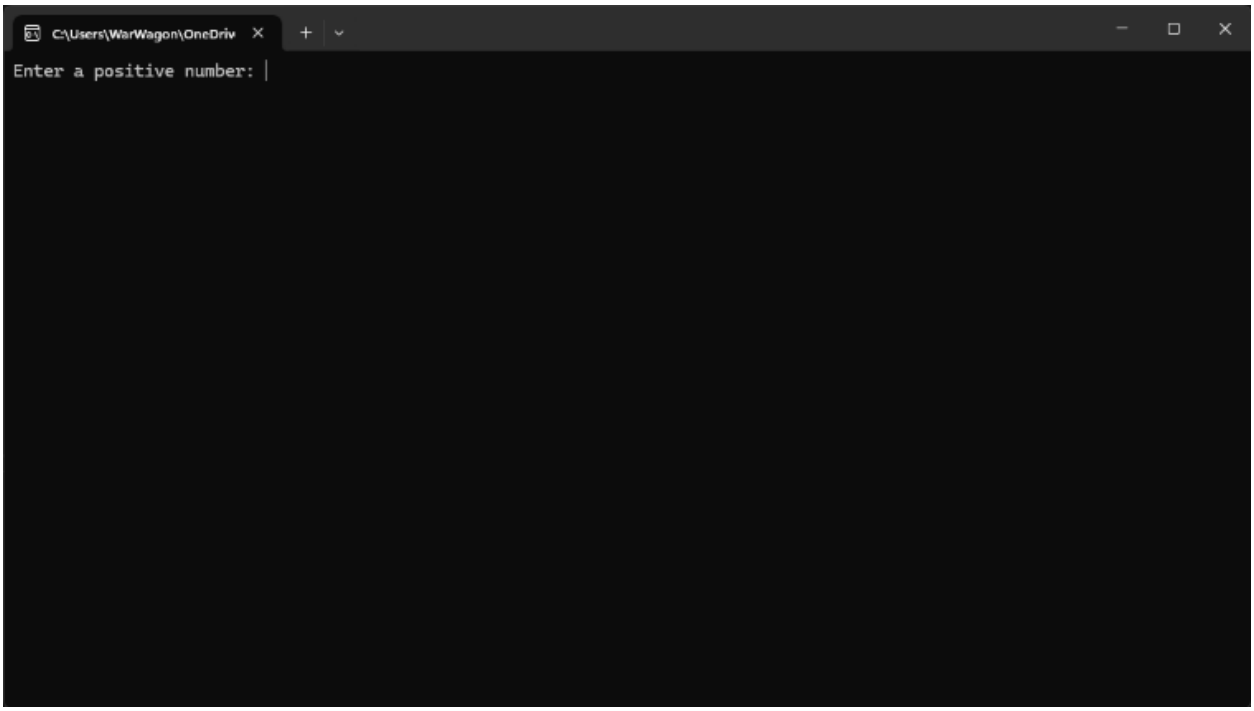


Figure 2-3: Part 2 App Running initial load state

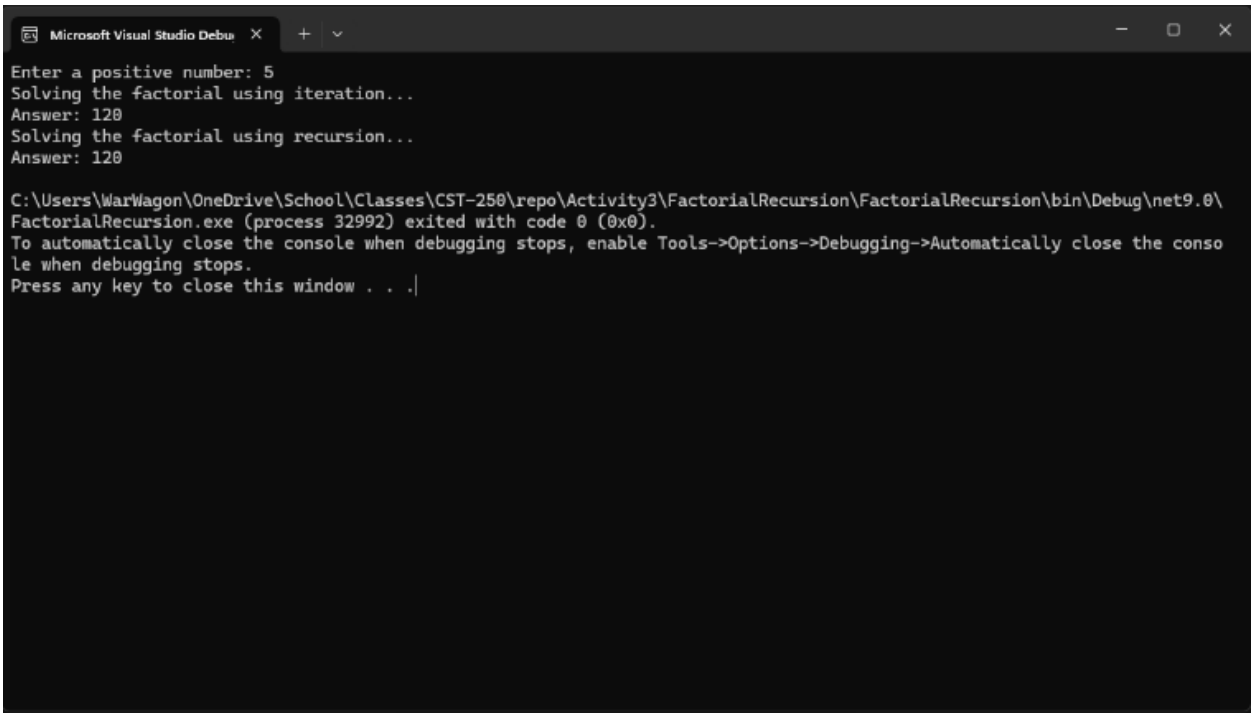


Figure 2-4: Screenshot of successful output Factoring 5

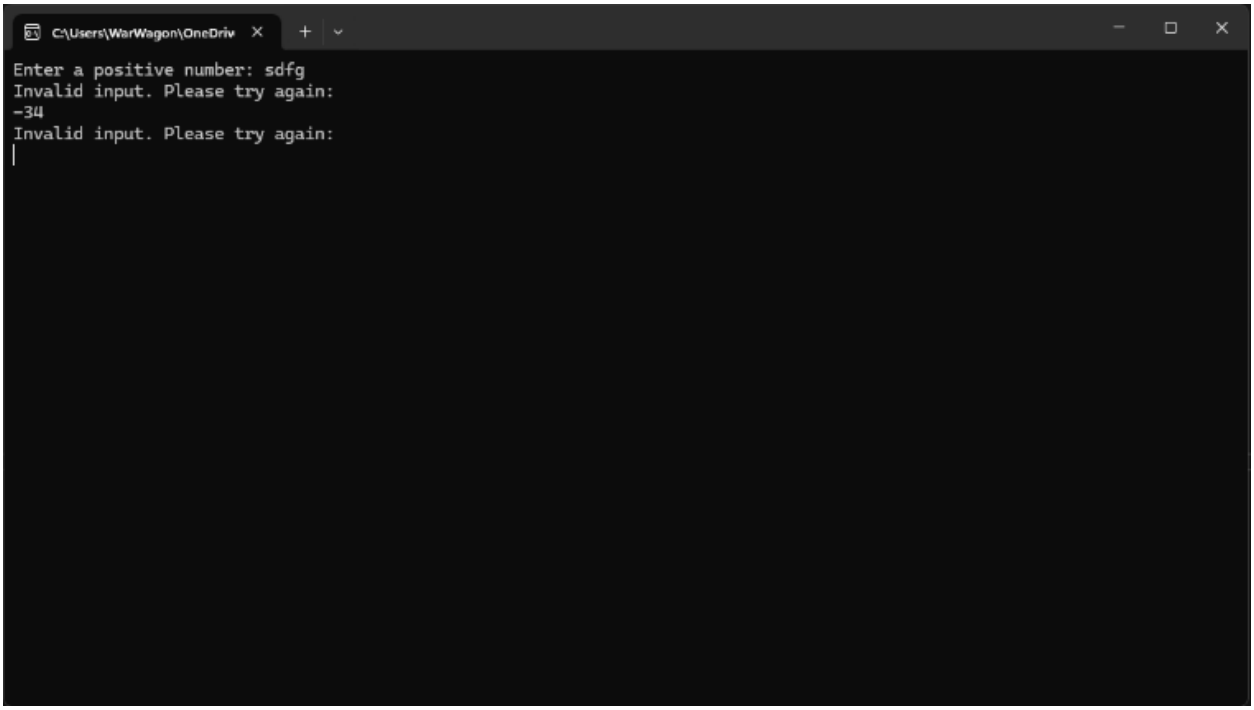


Figure 2-5: Screenshot of Error Handling non integer numbers

Code

```
/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/09/2020
 * Factorial recursion
 * Activity 3 Part 2
 * References:
 */

//---------------------------------------------
// Start of the Main Method
//---------------------------------------------

// Declare and Initialize
using FactorialRecursion.Services.BusinessLogicLayer;
using System.Numerics;

FactorialLogic factorialLogic = new FactorialLogic();
int input = 0;
BigInteger iterativeAns = 0, recursiveAns = 0;

// Prompt the user
Console.Write("Enter a positive number: ");

// Get the users input
input = Utility.ReadIntFromConsole();

// Solve the factorial using iteration
Console.WriteLine("Solving the factorial using iteration...");
iterativeAns = factorialLogic.SolveIterativeFactorial(input);
Console.WriteLine($"Answer: {iterativeAns}");

// Solve the factorial using recursion
Console.WriteLine("Solving the factorial using recursion...");
recursiveAns = factorialLogic.SolveRecursiveFactorial(input);
Console.WriteLine($"Answer: {recursiveAns}");

//---------------------------------------------
// End of the Main Method
//---------------------------------------------
```

Figure 2-6: Code Citations and the main method.

```
//---------------------------------------------
// Start of the Utility class
//---------------------------------------------

/// <summary>
/// Read integer input from the console
/// </summary>
/// <returns></returns>
1 reference
static class Utility
{
    1 reference
    internal static int ReadIntFromConsole()
    {
        // Declare and Initialize
        string input = "";
        int number = -1;

        // Get the current input
        input = Console.ReadLine();

        // Check if the input is valid
        while ( !(int.TryParse(input, out number) && number >0) )
        {
            // Show the user an error message and prompt them again
            Console.WriteLine("Invalid input. Please try again: ");

            // Get the new input
            input = Console.ReadLine();
        }

        // Return the resulting integer from the user
        return number;
    }
}

//---------------------------------------------
// End of the Utility class
//---------------------------------------------
```

Figure 2-7: Utility class and the read int from console method.

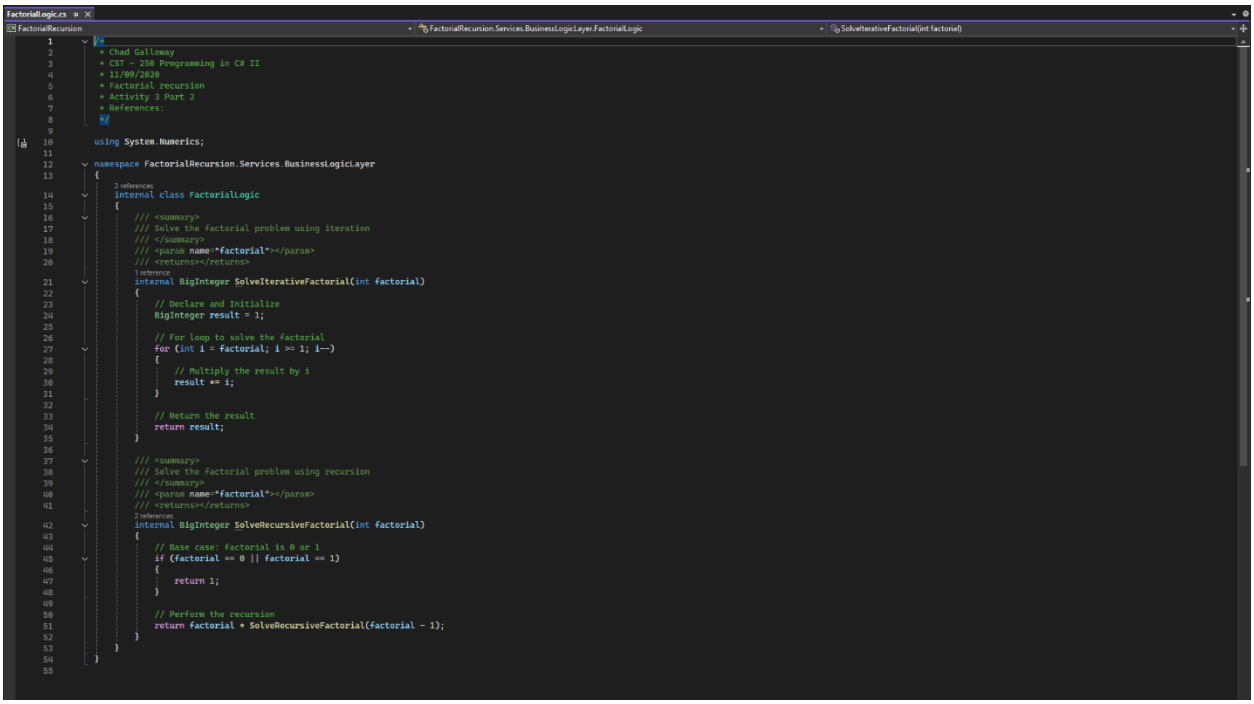
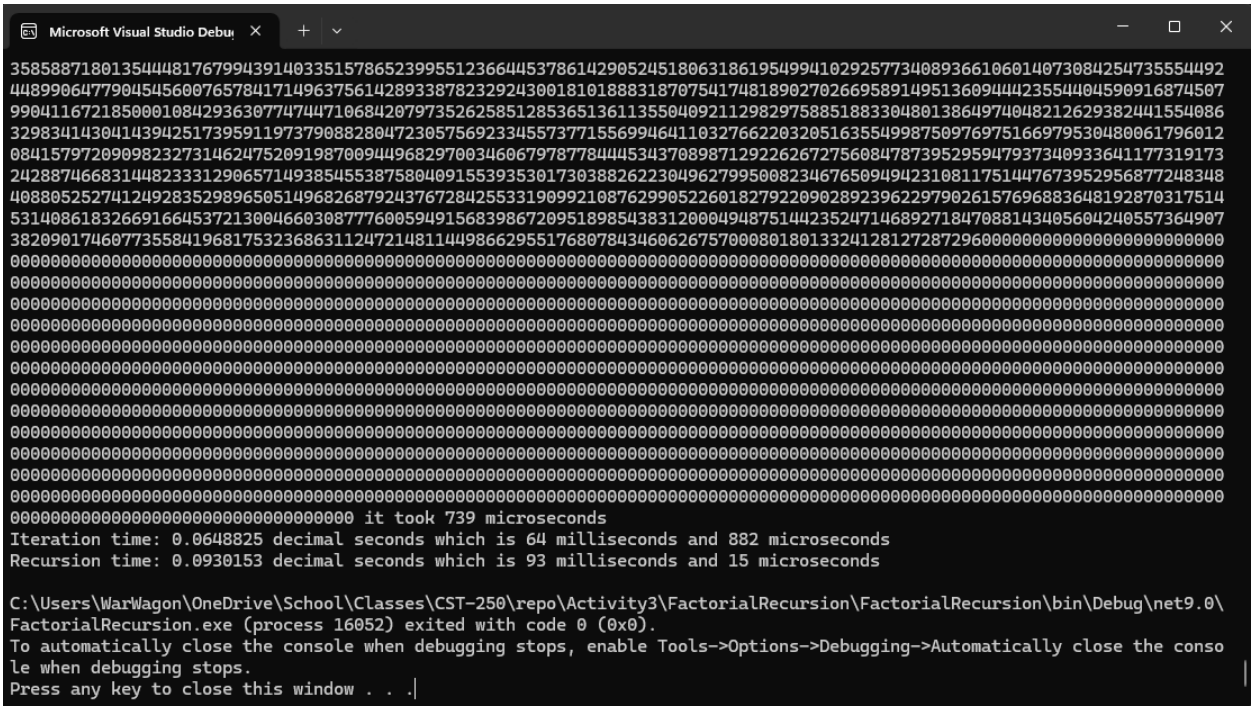


Figure 2-8: Factorial logic class with the two methods for iteration and recursion present

PART 2 CHALLENGES

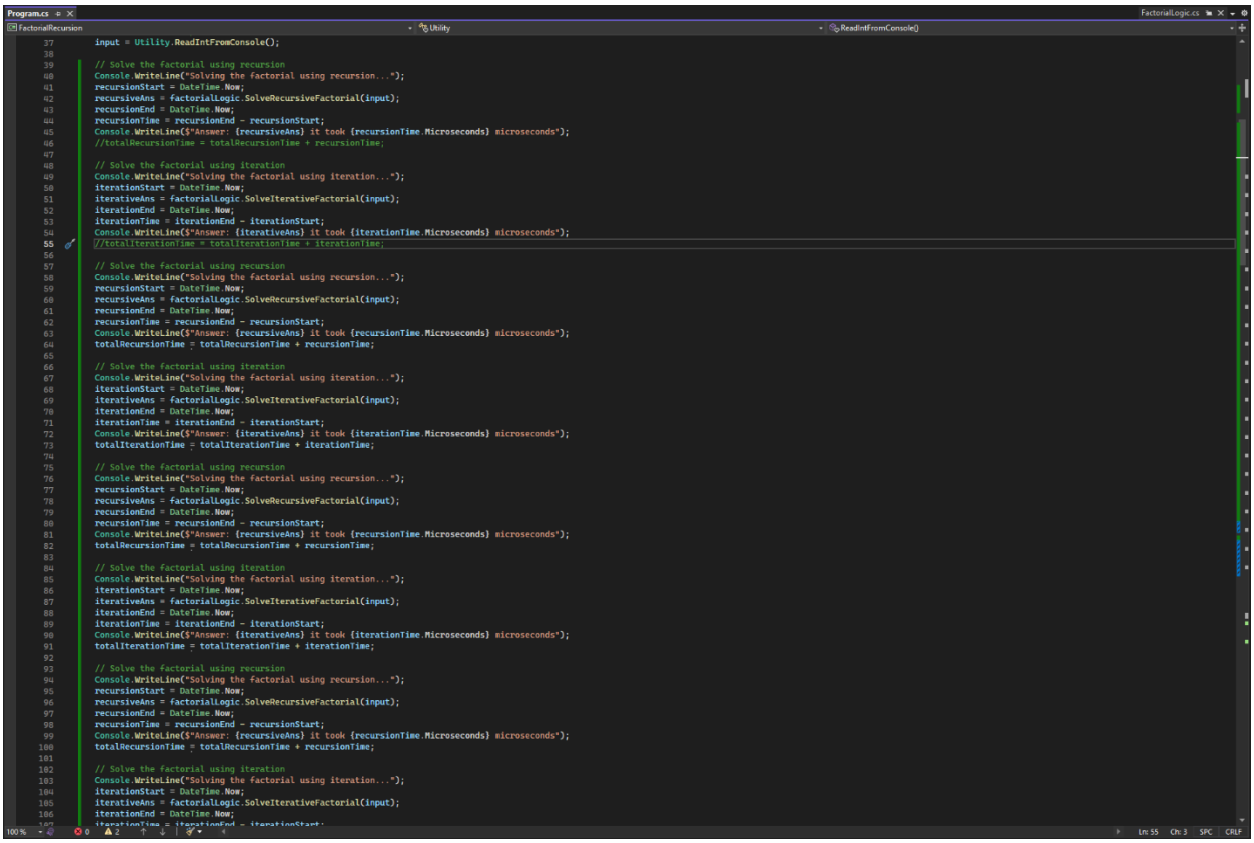
Screen Shots



```
358588718013544481767994391403351578652399551236644537861429052451806318619549941029257734089366106014073084254735554492
448990647790454560076578417149637561428933878232924300181018883187075417481890270266958914951360944423554404590916874507
990411672185000108429363077474471068420797352625851285365136113550409211298297588518833048013864974048212629382441554086
329834143041439425173959119737908828047230575692334557377155699464110327662203205163554998750976975166979530480061796012
084157972090982327314624752091987009449682970034606797877844453437089871292262672756084787395295947937340933641177319173
242887466831448233312906571493854553875804091553935301730388262230496279950082346765094942310811751447673952956877248348
408805252741249283529896505149682687924376728425533190992108762990522601827922090289239622979026157696883648192870317514
531408618326691664537213004660308777600594915683986720951898543831200049487514423524714689271847088143405604240557364907
382090174607735584196817532368631124721481144986629551768078434606267570008018013324128127287296000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000 it took 739 microseconds
Iteration time: 0.0648825 decimal seconds which is 64 milliseconds and 882 microseconds
Recursion time: 0.0930153 decimal seconds which is 93 milliseconds and 15 microseconds

C:\Users\WarWagon\OneDrive\School\Classes\CST-250\repo\Activity3\FactorialRecursion\FactorialRecursion\bin\Debug\net9.0\
FactorialRecursion.exe (process 16052) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

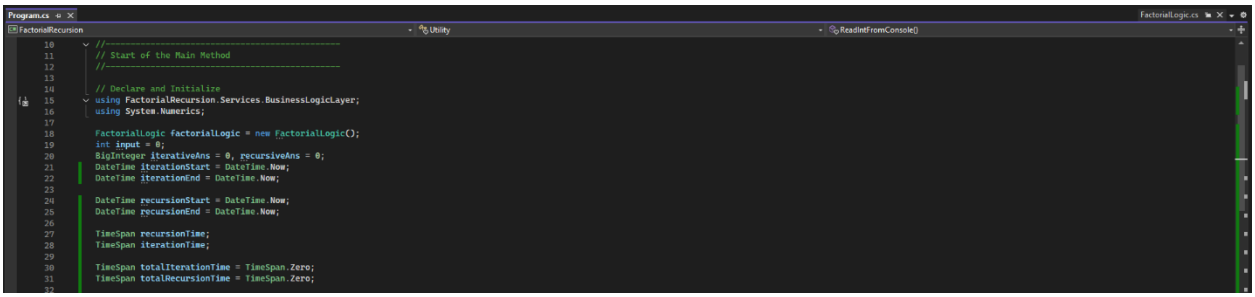
Figure 2-9: I created my own challenge. I wanted to know if one was faster than the other. I created some timing variables. Then started running some tests using 6000! as the value every time. Whichever method variation ran first took the most amount of time. It would seem that creating the first call, whether it was iteration or recursion always took the most time. To eliminate this outlier and continue testing I ran a few dozen (12 to each) calls to each method and discarded both of the first calls. Doing this started producing a trend, every single time iteration was fastest when adding up each type’s total time. I thought at first, I was seeing a 5% chance that recursion would be faster but it just turned out to be rounding errors in my code. After modifying my code to track the decimal time in seconds and do math off of that, iteration won every time.

Code



```
input = Utility.ReadIntFromConsole();

// Solve the factorial using recursion
Console.WriteLine("Solving the factorial using recursion...");
recursionStart = DateTime.Now;
recursiveAns = factorialLogic.SolveRecursiveFactorial(input);
recursionEnd = DateTime.Now;
recursionTime = recursionEnd - recursionStart;
Console.WriteLine($"Answer: {recursiveAns} it took {recursionTime.Microseconds} microseconds");
//totalRecursionTime = totalRecursionTime + recursionTime;

// Solve the factorial using iteration
Console.WriteLine("Solving the factorial using iteration...");
iterationStart = DateTime.Now;
iterativeAns = factorialLogic.SolveIterativeFactorial(input);
iterationEnd = DateTime.Now;
iterationTime = iterationEnd - iterationStart;
Console.WriteLine($"Answer: {iterativeAns} it took {iterationTime.Microseconds} microseconds");
//totalIterationTime = totalIterationTime + iterationTime;

// Solve the factorial using recursion
Console.WriteLine("Solving the factorial using recursion...");
recursionStart = DateTime.Now;
recursiveAns = factorialLogic.SolveRecursiveFactorial(input);
recursionEnd = DateTime.Now;
recursionTime = recursionEnd - recursionStart;
Console.WriteLine($"Answer: {recursiveAns} it took {recursionTime.Microseconds} microseconds");
totalRecursionTime = totalRecursionTime + recursionTime;

// Solve the factorial using iteration
Console.WriteLine("Solving the factorial using iteration...");
iterationStart = DateTime.Now;
iterativeAns = factorialLogic.SolveIterativeFactorial(input);
iterationEnd = DateTime.Now;
iterationTime = iterationEnd - iterationStart;
Console.WriteLine($"Answer: {iterativeAns} it took {iterationTime.Microseconds} microseconds");
totalIterationTime = totalIterationTime + iterationTime;

// Solve the factorial using recursion
Console.WriteLine("Solving the factorial using recursion...");
recursionStart = DateTime.Now;
recursiveAns = factorialLogic.SolveRecursiveFactorial(input);
recursionEnd = DateTime.Now;
recursionTime = recursionEnd - recursionStart;
Console.WriteLine($"Answer: {recursiveAns} it took {recursionTime.Microseconds} microseconds");
totalRecursionTime = totalRecursionTime + recursionTime;

// Solve the factorial using iteration
Console.WriteLine("Solving the factorial using iteration...");
iterationStart = DateTime.Now;
iterativeAns = factorialLogic.SolveIterativeFactorial(input);
iterationEnd = DateTime.Now;
iterationTime = iterationEnd - iterationStart;
Console.WriteLine($"Answer: {iterativeAns} it took {iterationTime.Microseconds} microseconds");
totalIterationTime = totalIterationTime + iterationTime;

// Solve the factorial using recursion
Console.WriteLine("Solving the factorial using recursion...");
recursionStart = DateTime.Now;
recursiveAns = factorialLogic.SolveRecursiveFactorial(input);
recursionEnd = DateTime.Now;
recursionTime = recursionEnd - recursionStart;
Console.WriteLine($"Answer: {recursiveAns} it took {recursionTime.Microseconds} microseconds");
totalRecursionTime = totalRecursionTime + recursionTime;

// Solve the factorial using iteration
Console.WriteLine("Solving the factorial using iteration...");
iterationStart = DateTime.Now;
iterativeAns = factorialLogic.SolveIterativeFactorial(input);
iterationEnd = DateTime.Now;
```

Figure 2-10: Code for the increased number of method calls. I even tried a structure where the calls happened in a different order and that just widened the gap between the two different structure types. So based off of all my tests at 6000! Iteration has a clear edge of about 30% efficiency.



```
//-----------------------------------------------------
// Start of the Main Method
//-----------------------------------------------------

// Declare and Initialize
using FactorialRecursion.Services.BusinessLogicLayer;
using System.Numerics;

FactorialLogic factorialLogic = new FactorialLogic();
int input = 0;
BigInteger iterativeAns = 0, recursiveAns = 0;
DateTime iterationStart = DateTime.Now;
DateTime iterationEnd = DateTime.Now;

DateTime recursionStart = DateTime.Now;
DateTime recursionEnd = DateTime.Now;

TimeSpan recursionTime;
TimeSpan iterationTime;

TimeSpan totalIterationTime = TimeSpan.Zero;
TimeSpan totalRecursionTime = TimeSpan.Zero;
```

Figure 2-11: The code for the tracking variables I created.

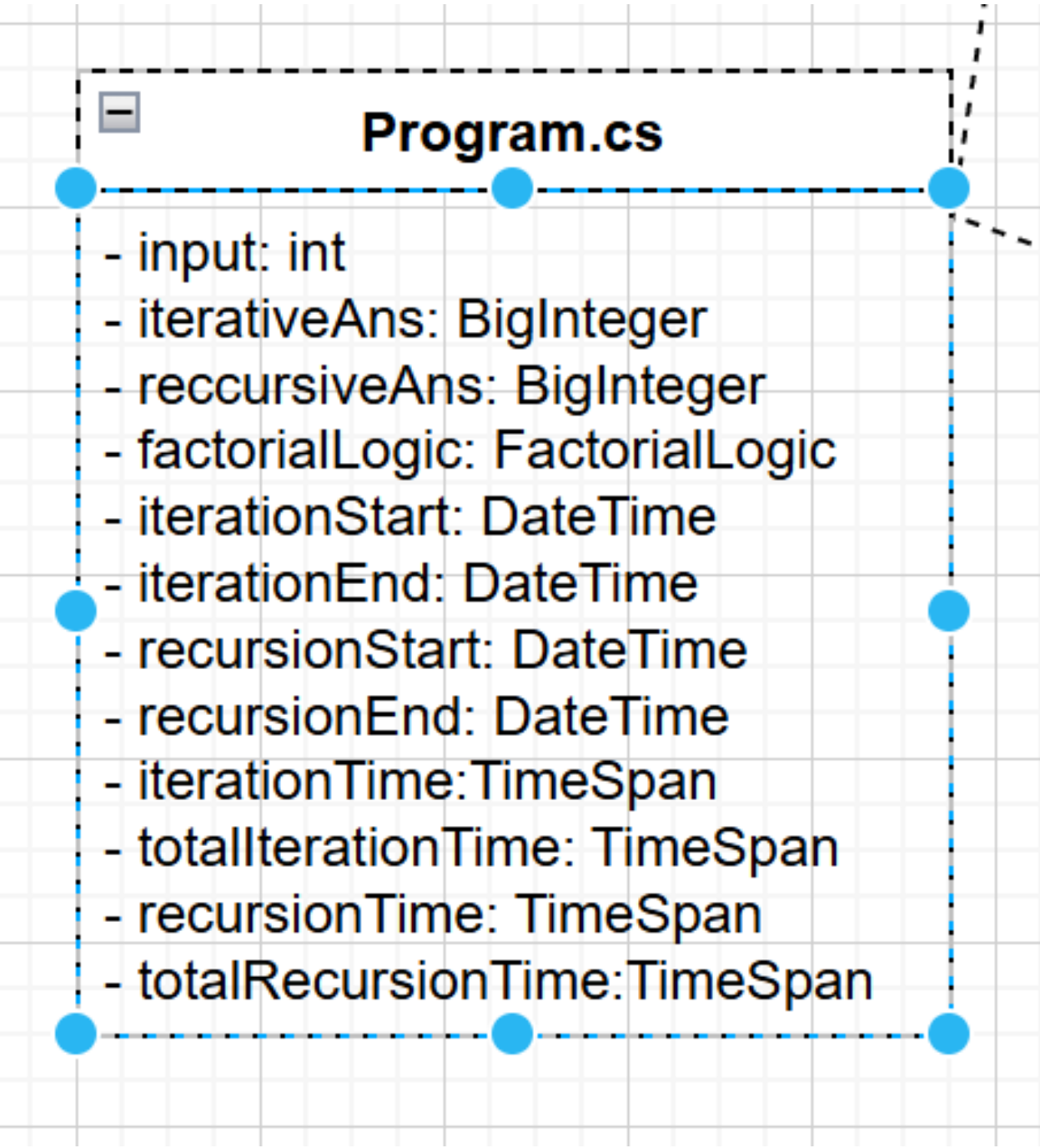


Figure 2-12: Updated UML for my timing tests

PART 3

Flow Chart

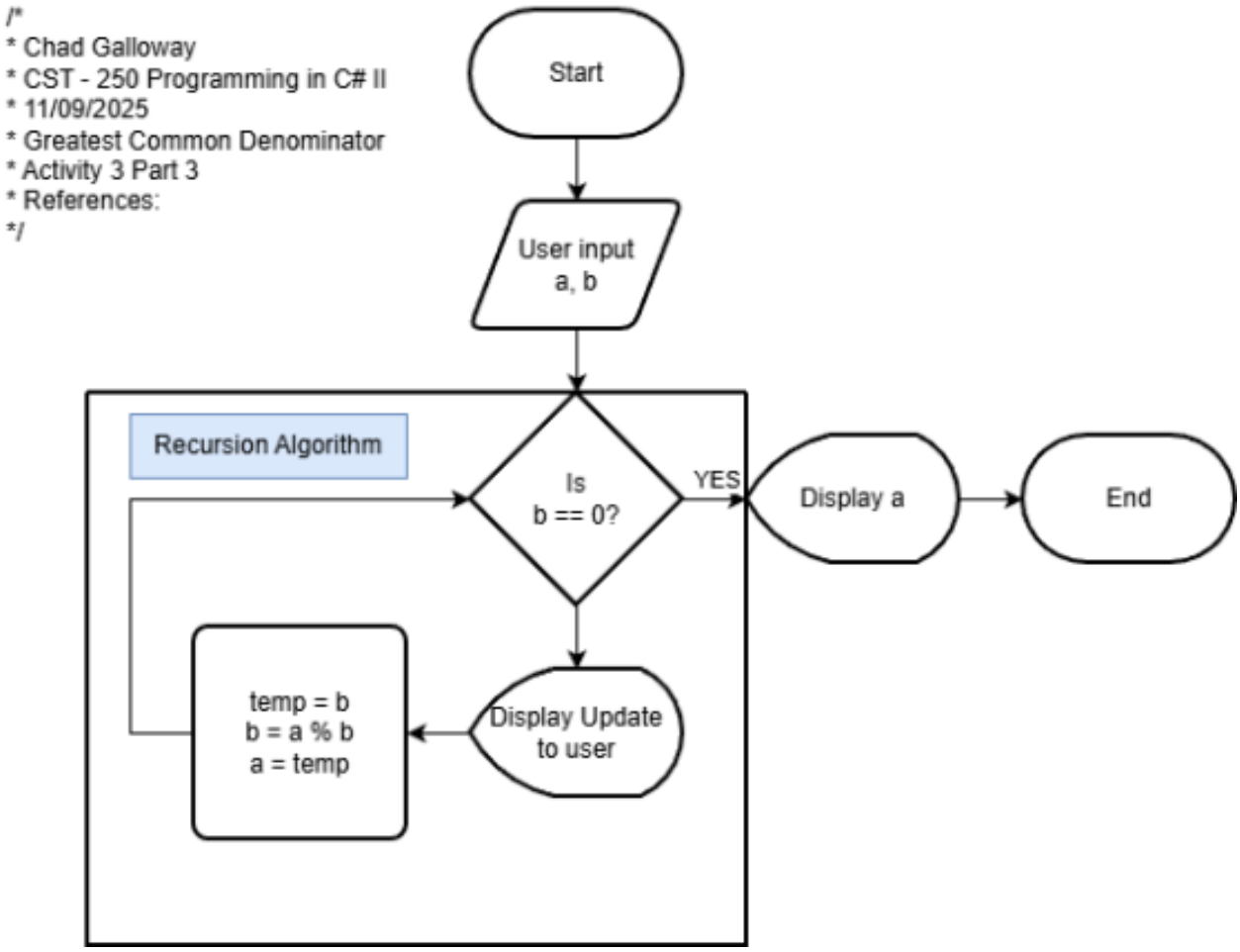


Figure 3-1: Flow chart of Activity 3 Part 3 greatest common divisor

UML Class Diagram

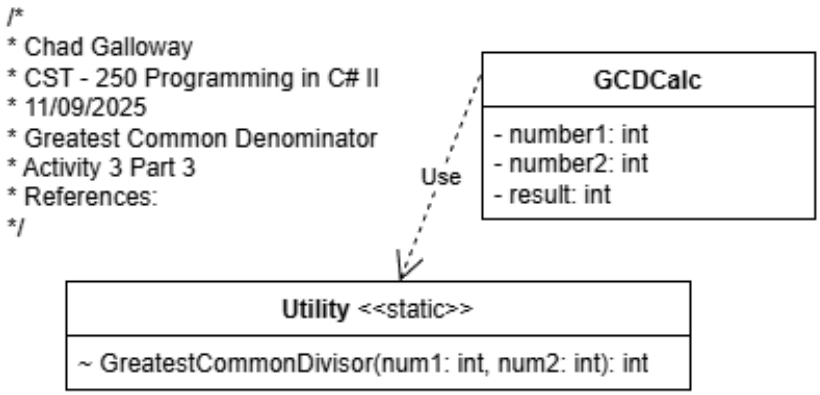


Figure 3-2: UML Class Diagram of Activity 3 Part 3 greatest common divisor

Screen Shots

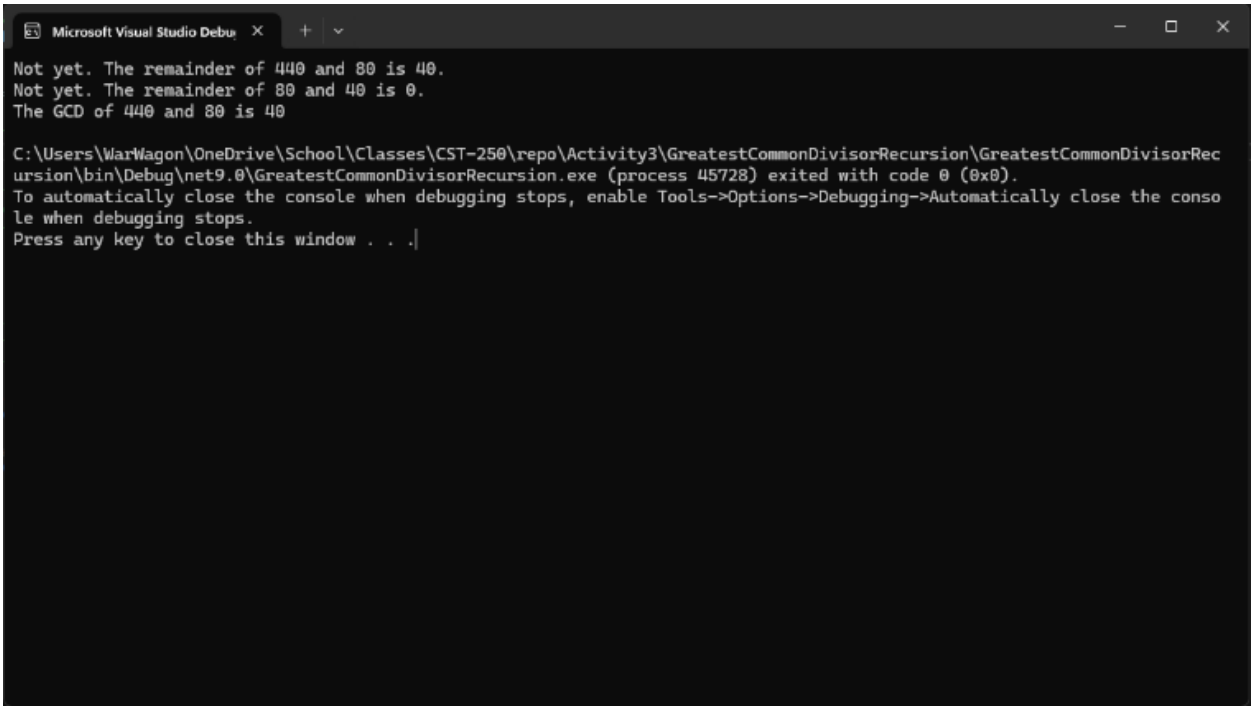
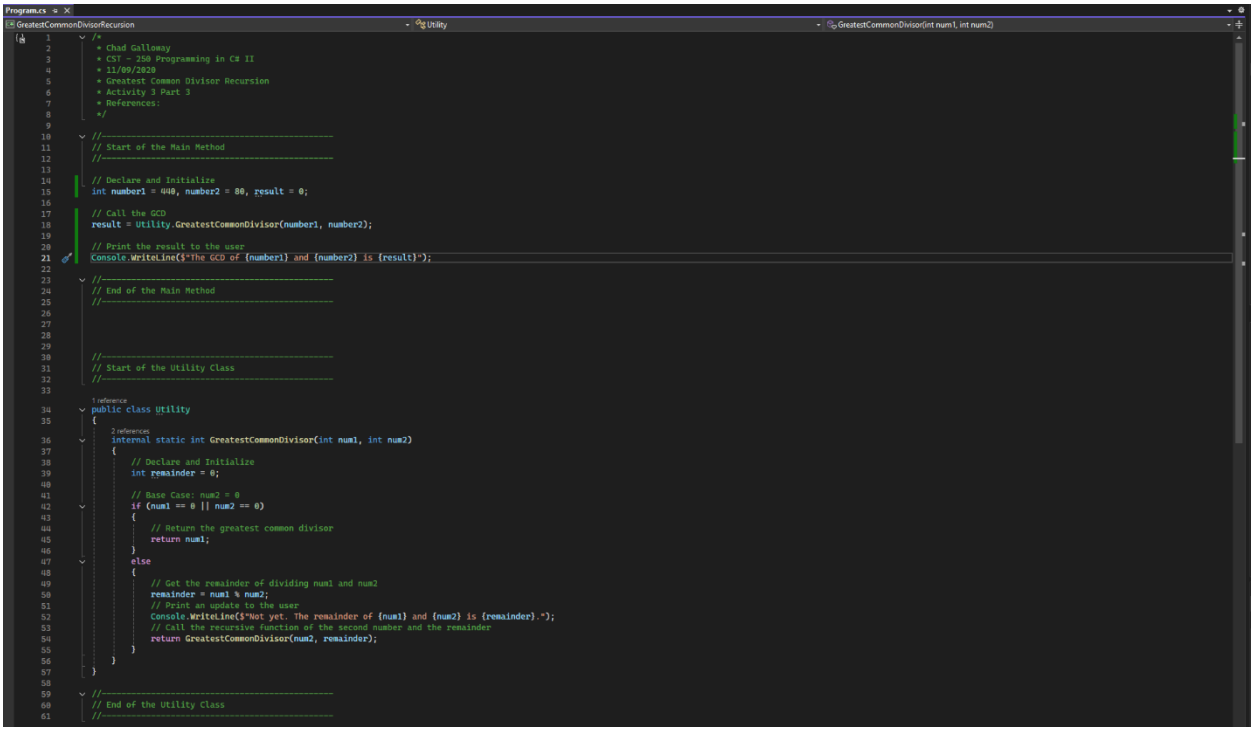


Figure 3-3: Screenshot of successful output

Code



```
Program.cs
GreatestCommonDivisorRecursion | Utility | GreatestCommonDivisor(int num1, int num2)

/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/09/2020
 * Greatest Common Divisor Recursion
 * Activity 3 Part 3
 * References:
 */

//-----------------------------------------------
// Start of the Main Method
//-----------------------------------------------

// Declare and Initialize
int number1 = 448, number2 = 80, result = 0;

// Call the GCD
result = Utility.GreatestCommonDivisor(number1, number2);

// Print the result to the user
Console.WriteLine($"The GCD of {number1} and {number2} is {result}");

//-----------------------------------------------
// End of the Main Method
//-----------------------------------------------



//-----------------------------------------------
// Start of the Utility Class
//-----------------------------------------------

1 reference
public class Utility
{
    2 references
    internal static int GreatestCommonDivisor(int num1, int num2)
    {
        // Declare and Initialize
        int remainder = 0;

        // Base Case: num2 = 0
        if (num1 == 0 || num2 == 0)
        {
            // Return the greatest common divisor
            return num1;
        }
        else
        {
            // Get the remainder of dividing num1 and num2
            remainder = num1 % num2;
            // Print an update to the user
            Console.WriteLine($"Not yet. The remainder of {num1} and {num2} is {remainder}.");
            // Call the recursive function of the second number and the remainder
            return GreatestCommonDivisor(num2, remainder);
        }
    }
}

//-----------------------------------------------
// End of the Utility Class
//-----------------------------------------------
```

Figure 3-4: Code Citations and the main method.

PART 3 CHALLENGES

Screen Shots

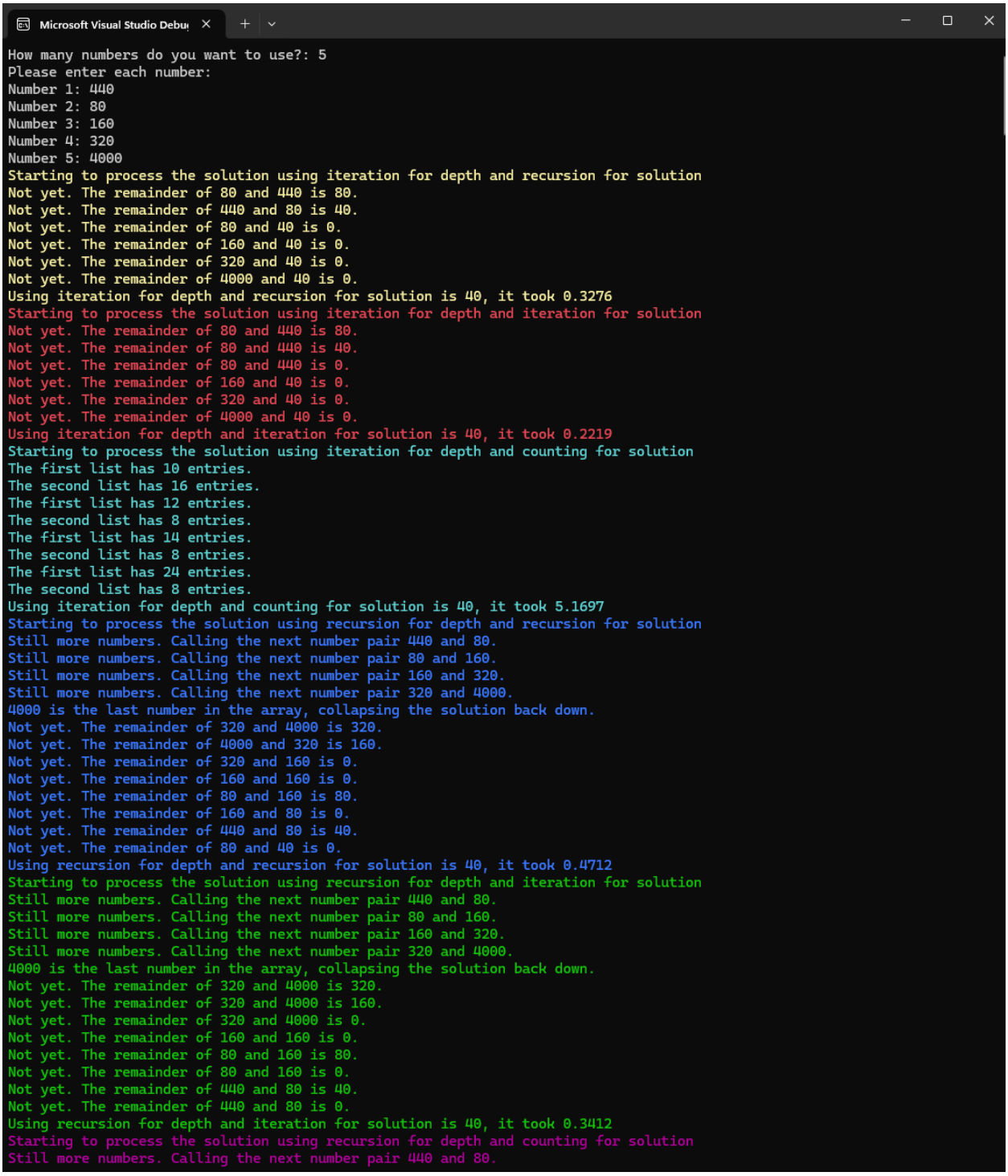


Figure 3-5 Screen shot of the user entry on the challenges. Completing number one of the challenges

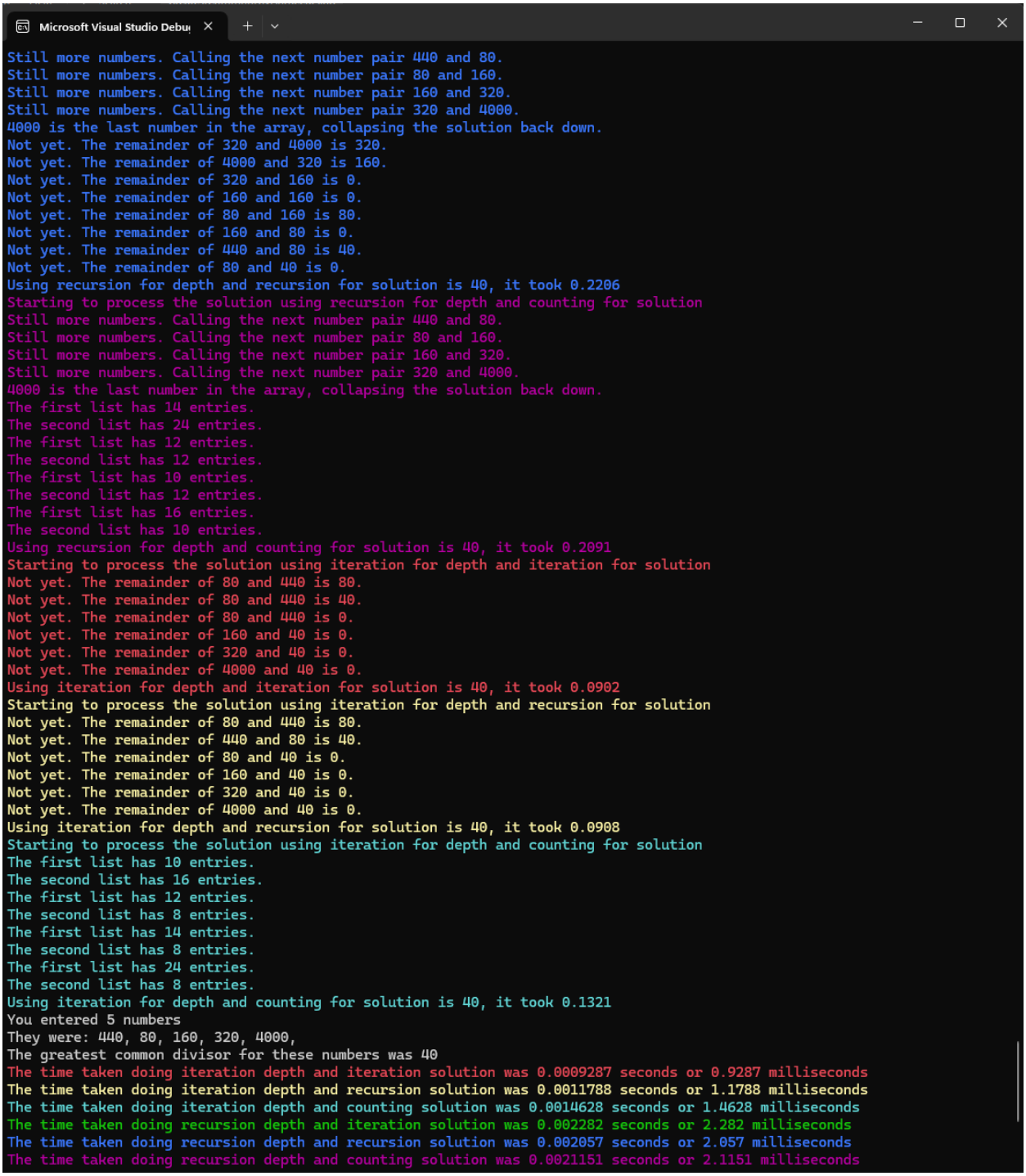


Figure 3-6 Screen shot of the successful completion of the challenges. Every different type of method call is color coded. It made back tracking when one type of operation had an unusually high time or even an incorrect result when I worked on making negative numbers and zero function properly. This screen shot depicts challenges 2-4

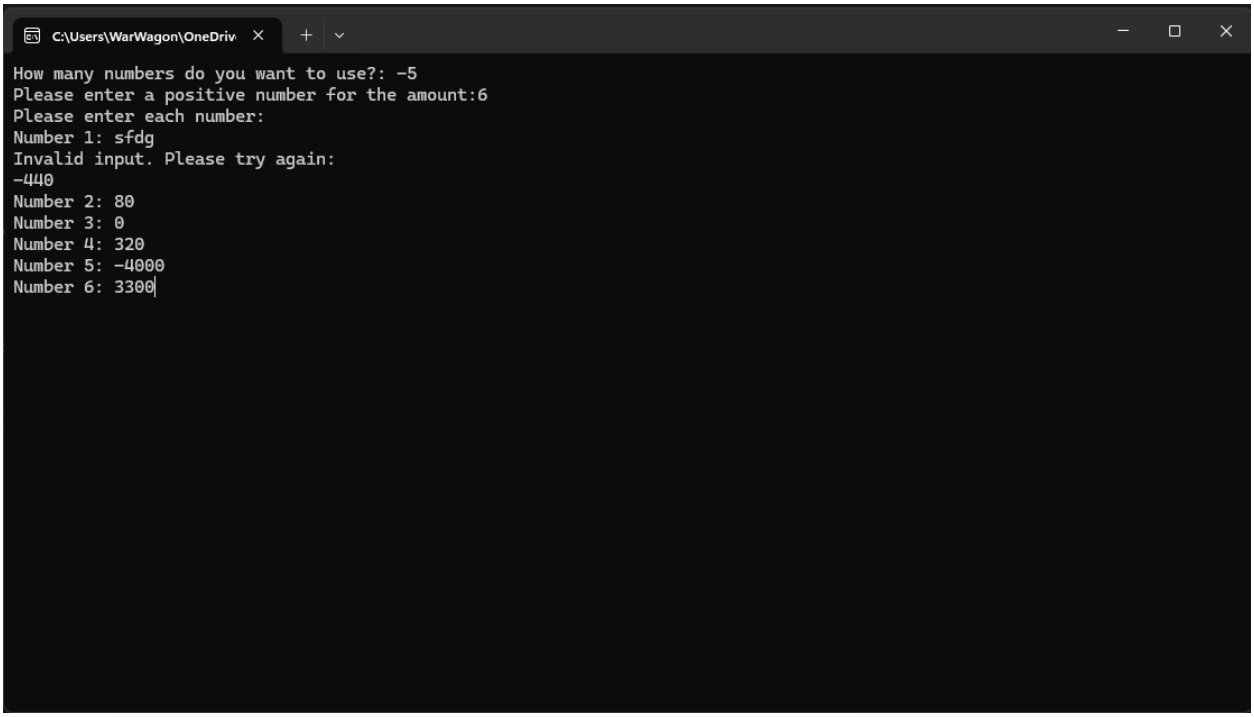


Figure 3-7 Screen shot of the error handling capabilities of the application.

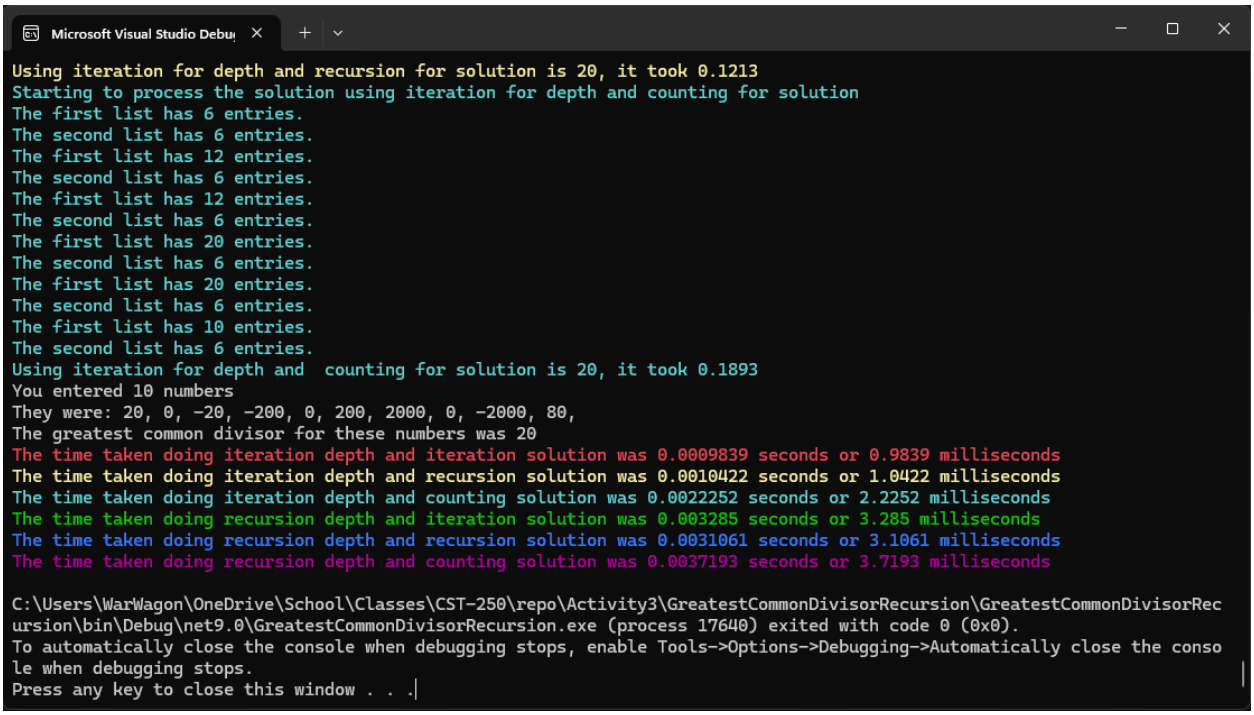
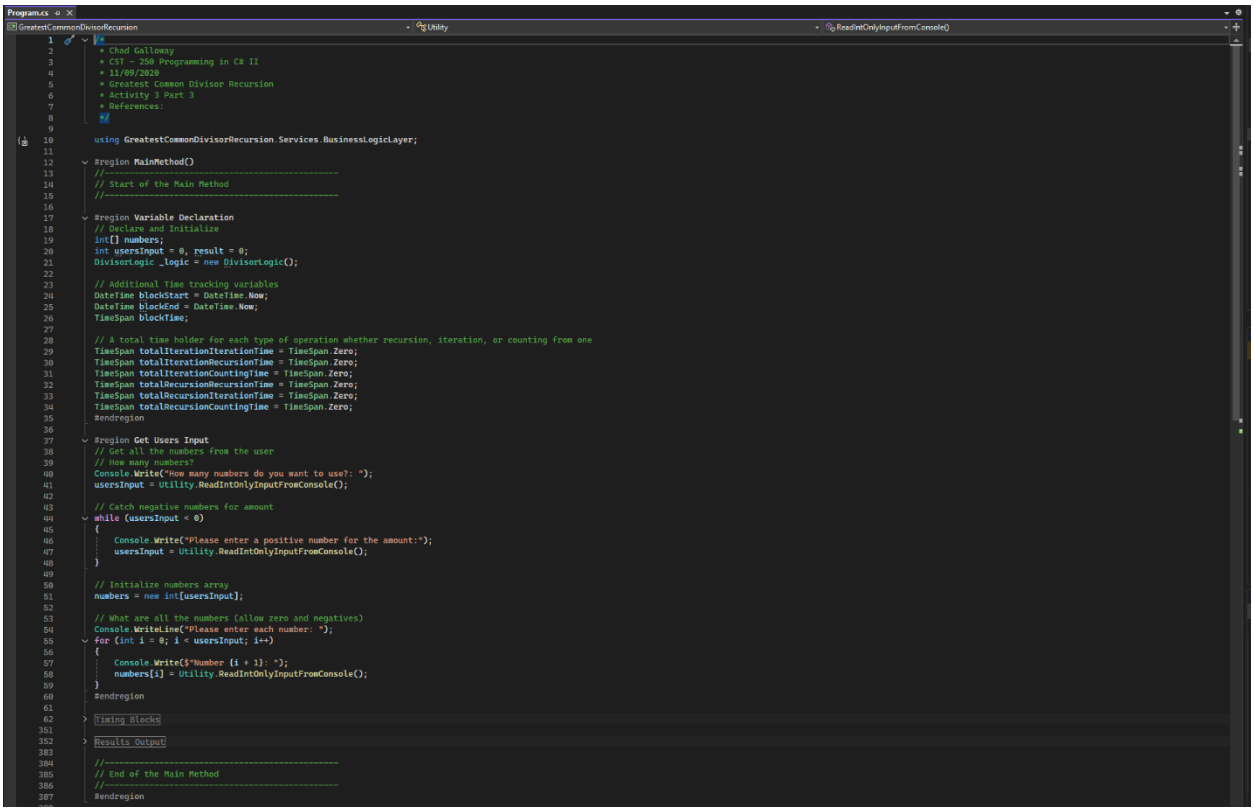


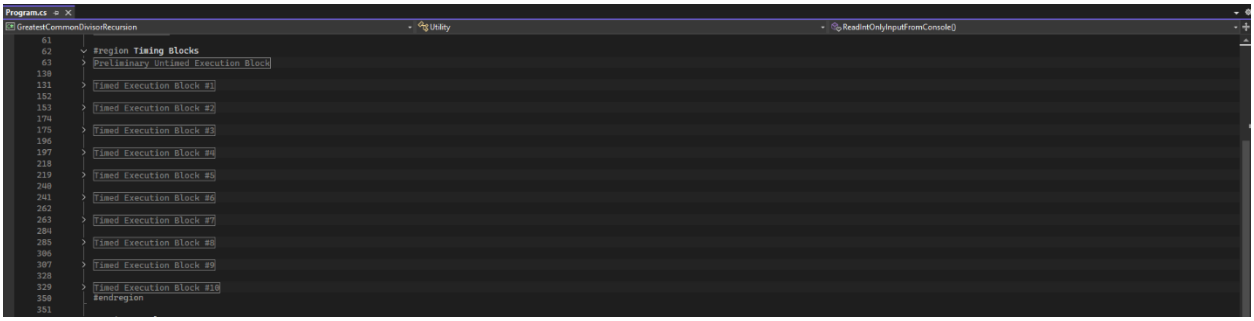
Figure 3-8 Screen shot of the zero and negative handling of the application. Figure 3-7 and 3-8 satisfy number 5 of the part three challenges.

Code



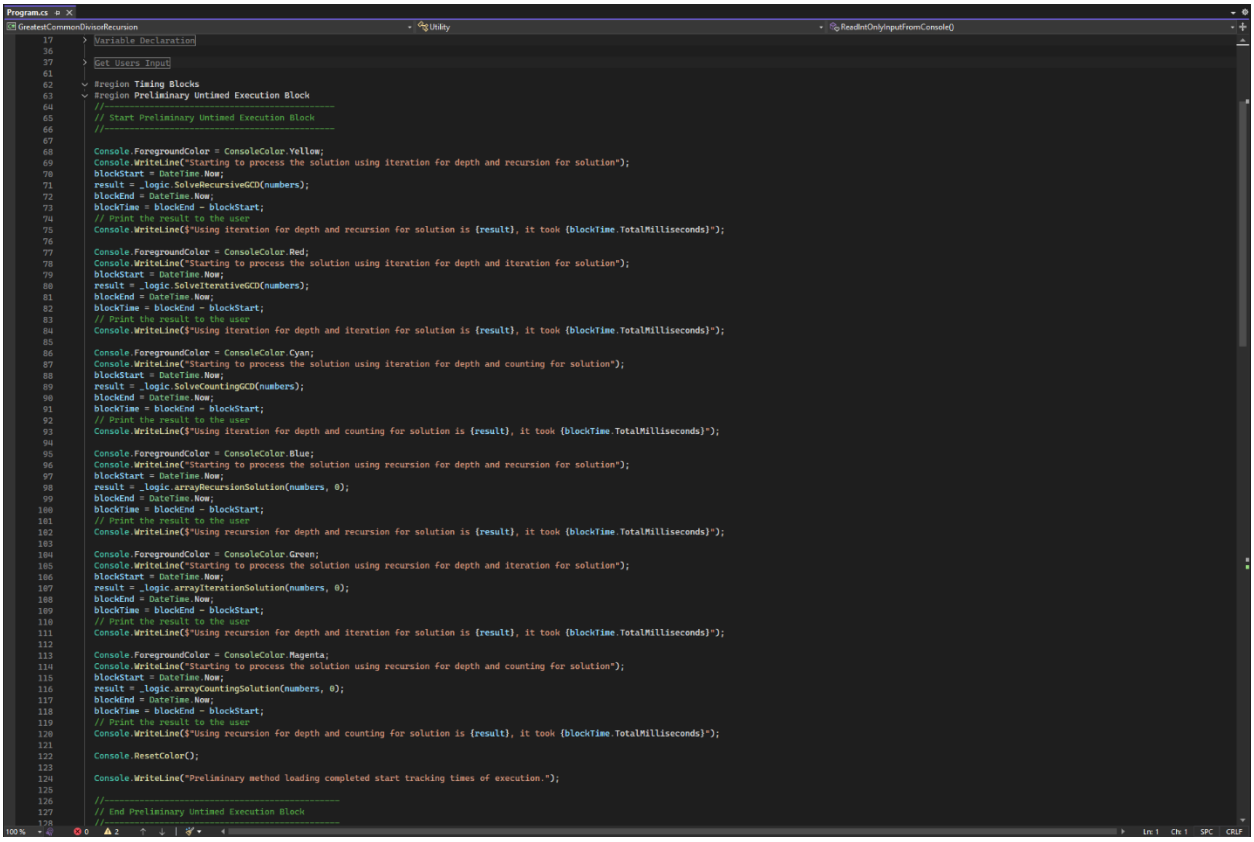
```
Programs.cs
GreatestCommonDivisorRecursion  Utility  ReadIntOnlyInputFromConsole()

/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/09/2020
 * Greatest Common Divisor Recursion
 * Activity 3 Part 3
 * References:
 */

using GreatestCommonDivisorRecursion.Services.BusinessLogicLayer;

#region MainMethod()
//-----------------------------------------------
// Start of the Main Method
//-----------------------------------------------

#region Variable Declaration
// Declare and Initialize
int[] numbers;
int usersInput = 0, result = 0;
DivisorLogic _logic = new DivisorLogic();

// Additional Time tracking variables
DateTime blockStart = DateTime.Now;
DateTime blockEnd = DateTime.Now;
TimeSpan blockTime;

// A total time holder for each type of operation whether recursion, iteration, or counting from one
TimeSpan totalIterationIterationTime = TimeSpan.Zero;
TimeSpan totalIterationRecursionTime = TimeSpan.Zero;
TimeSpan totalIterationCountingTime = TimeSpan.Zero;
TimeSpan totalRecursionRecursionTime = TimeSpan.Zero;
TimeSpan totalRecursionIterationTime = TimeSpan.Zero;
TimeSpan totalRecursionCountingTime = TimeSpan.Zero;
#endregion

#region Get Users Input
// Get all the numbers from the user
// How many numbers?
Console.Write("How many numbers do you want to use?: ");
usersInput = Utility.ReadIntOnlyInputFromConsole();

// Catch negative numbers for amount
while (usersInput < 0)
{
    Console.Write("Please enter a positive number for the amount:");
    usersInput = Utility.ReadIntOnlyInputFromConsole();
}

// Initialize numbers array
numbers = new int[usersInput];

// What are all the numbers (allow zero and negatives)
Console.WriteLine("Please enter each number: ");
for (int i = 0; i < usersInput; i++)
{
    Console.Write($"Number {i + 1}: ");
    numbers[i] = Utility.ReadIntOnlyInputFromConsole();
}
#endregion

Timing Blocks

Results Output

//-----------------------------------------------
// End of the Main Method
//-----------------------------------------------
#endregion
```

Figure 3-9: Code Citations and the main method.



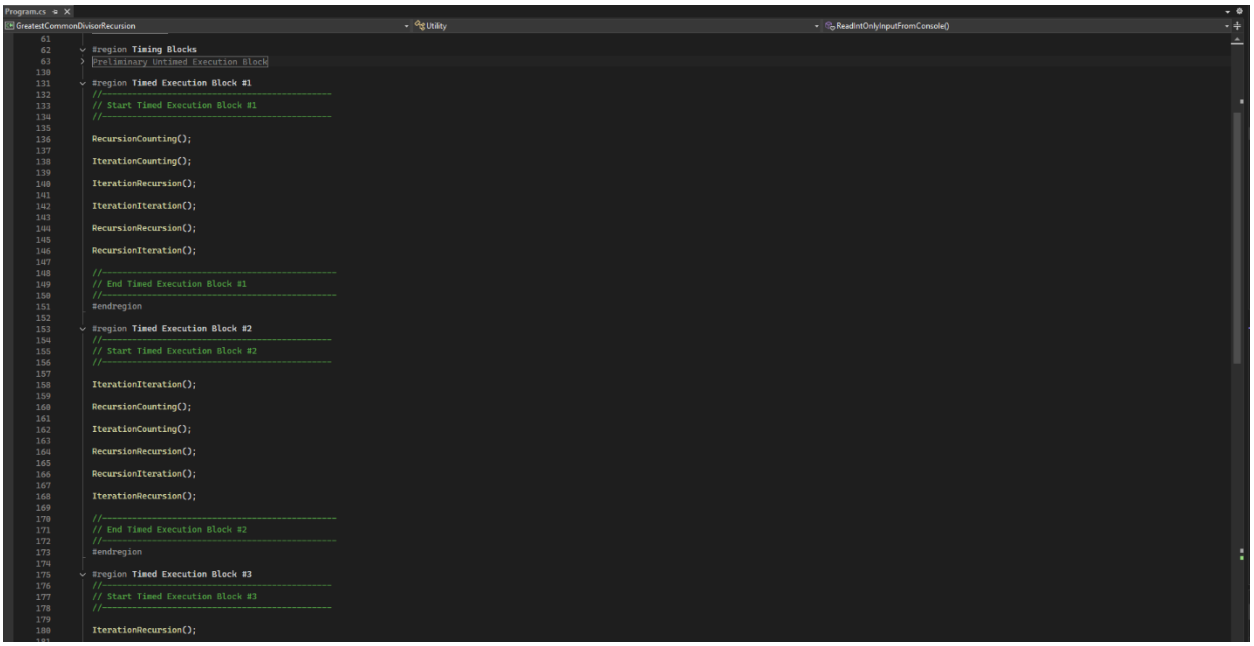
```
Programs.cs
GreatestCommonDivisorRecursion  Utility  ReadIntOnlyInputFromConsole()

#region Timing Blocks
Preliminary Untimed Execution Block

Timed Execution Block #1

Timed Execution Block #2

Timed Execution Block #3

Timed Execution Block #4

Timed Execution Block #5

Timed Execution Block #6

Timed Execution Block #7

Timed Execution Block #8

Timed Execution Block #9

Timed Execution Block #10
#endregion
```

Figure 3-10: A collapsed region of all the different code blocks used to track time on multiple method calls



```
Program.cs
GreatestCommonDivisorRecursion
Utility
ReadIntOnlyInputFromConsole()
Variable Declaration
Get Users Input
#region Timing Blocks
#region Preliminary Untimed Execution Block
//------------------------------------------------
// Start Preliminary Untimed Execution Block
//------------------------------------------------

Console.ForegroundColor = ConsoleColor.Yellow;
Console.WriteLine("Starting to process the solution using iteration for depth and recursion for solution");
blockStart = DateTime.Now;
result = _logic.SolveRecursiveGCD(numbers);
blockEnd = DateTime.Now;
blockTime = blockEnd - blockStart;
// Print the result to the user
Console.WriteLine($"Using iteration for depth and recursion for solution is {result}, it took {blockTime.TotalMilliseconds}");

Console.ForegroundColor = ConsoleColor.Red;
Console.WriteLine("Starting to process the solution using iteration for depth and iteration for solution");
blockStart = DateTime.Now;
result = _logic.SolveIterativeGCD(numbers);
blockEnd = DateTime.Now;
blockTime = blockEnd - blockStart;
// Print the result to the user
Console.WriteLine($"Using iteration for depth and iteration for solution is {result}, it took {blockTime.TotalMilliseconds}");

Console.ForegroundColor = ConsoleColor.Cyan;
Console.WriteLine("Starting to process the solution using iteration for depth and counting for solution");
blockStart = DateTime.Now;
result = _logic.SolveCountingGCD(numbers);
blockEnd = DateTime.Now;
blockTime = blockEnd - blockStart;
// Print the result to the user
Console.WriteLine($"Using iteration for depth and counting for solution is {result}, it took {blockTime.TotalMilliseconds}");

Console.ForegroundColor = ConsoleColor.Blue;
Console.WriteLine("Starting to process the solution using recursion for depth and recursion for solution");
blockStart = DateTime.Now;
result = _logic.arrayRecursionSolution(numbers, 0);
blockEnd = DateTime.Now;
blockTime = blockEnd - blockStart;
// Print the result to the user
Console.WriteLine($"Using recursion for depth and recursion for solution is {result}, it took {blockTime.TotalMilliseconds}");

Console.ForegroundColor = ConsoleColor.Green;
Console.WriteLine("Starting to process the solution using recursion for depth and iteration for solution");
blockStart = DateTime.Now;
result = _logic.arrayIterationSolution(numbers, 0);
blockEnd = DateTime.Now;
blockTime = blockEnd - blockStart;
// Print the result to the user
Console.WriteLine($"Using recursion for depth and iteration for solution is {result}, it took {blockTime.TotalMilliseconds}");

Console.ForegroundColor = ConsoleColor.Magenta;
Console.WriteLine("Starting to process the solution using recursion for depth and counting for solution");
blockStart = DateTime.Now;
result = _logic.arrayCountingSolution(numbers, 0);
blockEnd = DateTime.Now;
blockTime = blockEnd - blockStart;
// Print the result to the user
Console.WriteLine($"Using recursion for depth and counting for solution is {result}, it took {blockTime.TotalMilliseconds}");

Console.ResetColor();

Console.WriteLine("Preliminary method loading completed start tracking times of execution.");

//------------------------------------------------
// End Preliminary Untimed Execution Block
//------------------------------------------------
```

Figure 3-11: Preliminary timing block. This block runs all the methods but does not time them as in my tests I found that the first run method had extra overhead and accrued extra time for some reason so I removed them from the results for better consistency.



```
Program.cs
GreatestCommonDivisorRecursion
Utility
ReadIntOnlyInputFromConsole()
#region Timing Blocks
Preliminary Untimed Execution Block

#region Timed Execution Block #1
//------------------------------------------------
// Start Timed Execution Block #1
//------------------------------------------------

RecursionCounting();

IterationCounting();

IterationRecursion();

IterationIteration();

RecursionRecursion();

RecursionIteration();

//------------------------------------------------
// End Timed Execution Block #1
//------------------------------------------------
#endregion

#region Timed Execution Block #2
//------------------------------------------------
// Start Timed Execution Block #2
//------------------------------------------------

IterationIteration();

RecursionCounting();

IterationCounting();

RecursionRecursion();

RecursionIteration();

IterationRecursion();

//------------------------------------------------
// End Timed Execution Block #2
//------------------------------------------------
#endregion

#region Timed Execution Block #3
//------------------------------------------------
// Start Timed Execution Block #3
//------------------------------------------------

IterationRecursion();
```

Figure 3-12: Screen shot of the first few timing blocks expanded. They are in different orders to avoid any weird call order funniness. I tried to keep the results as pure as I could.

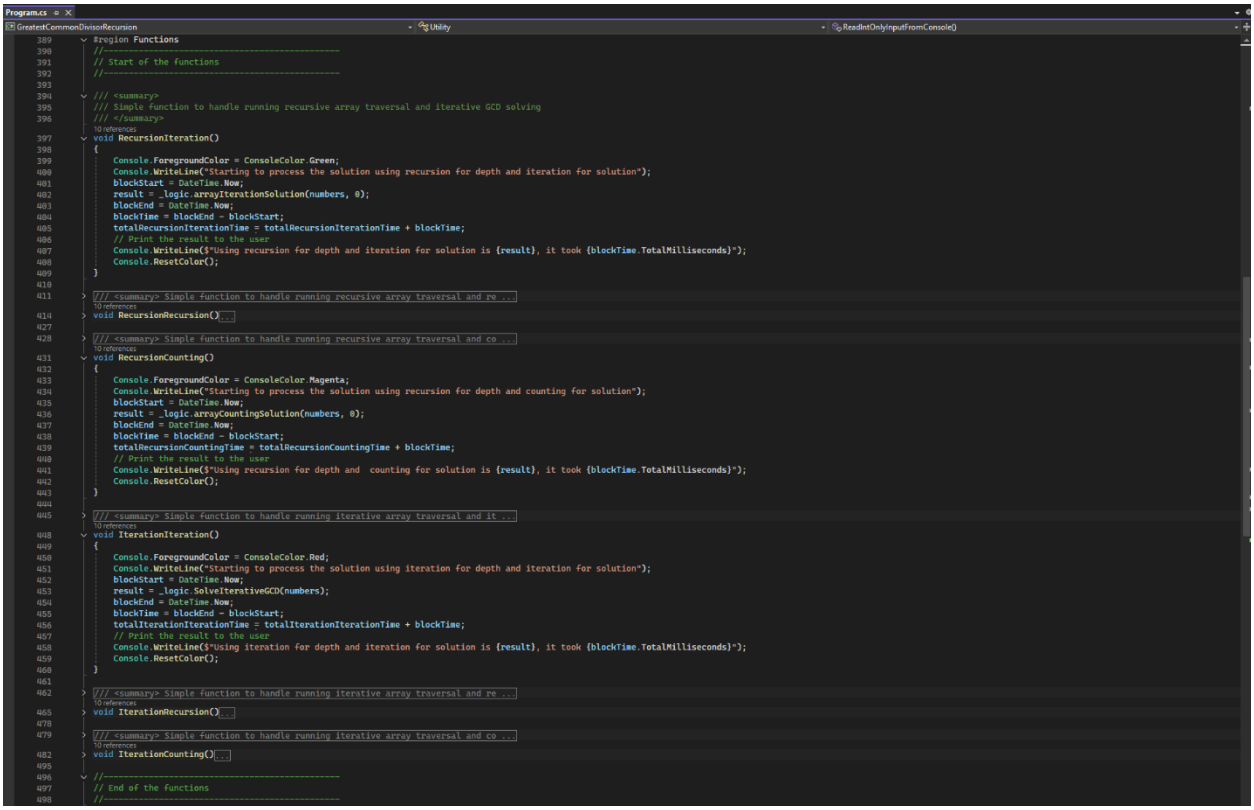


Figure 3-13: Screen shot of functions placed outside of the main method. Some of them are collapsed for space but they are all written almost the exact same way.

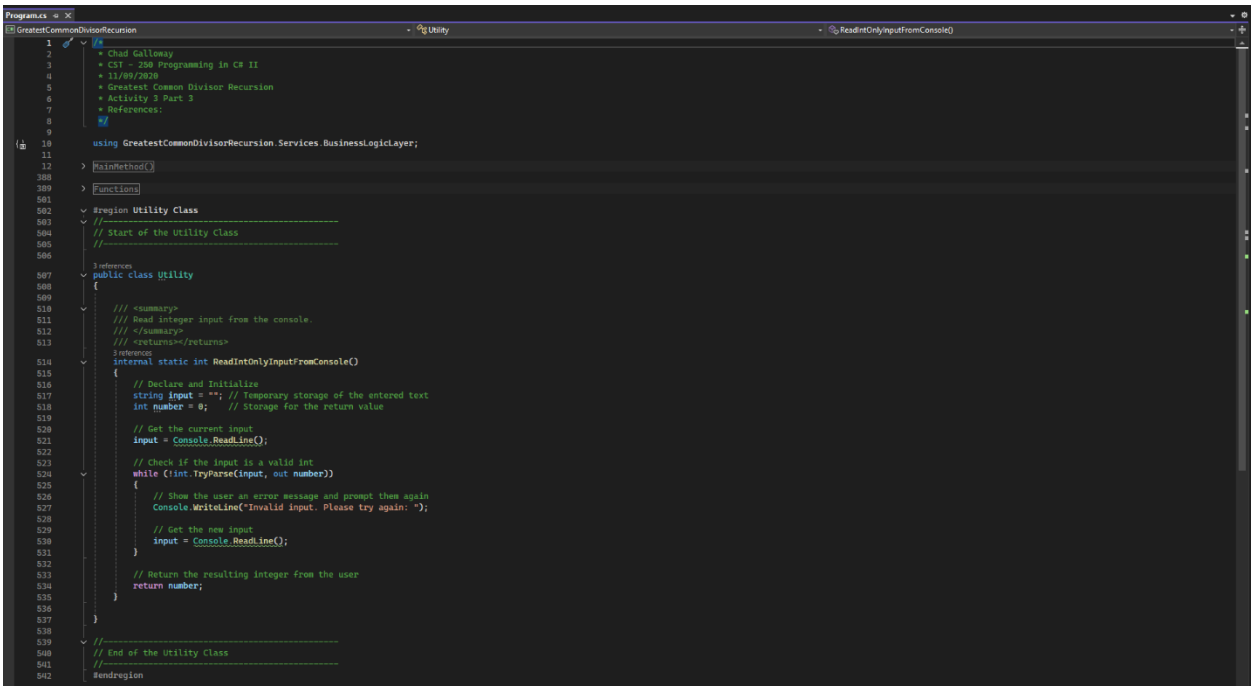
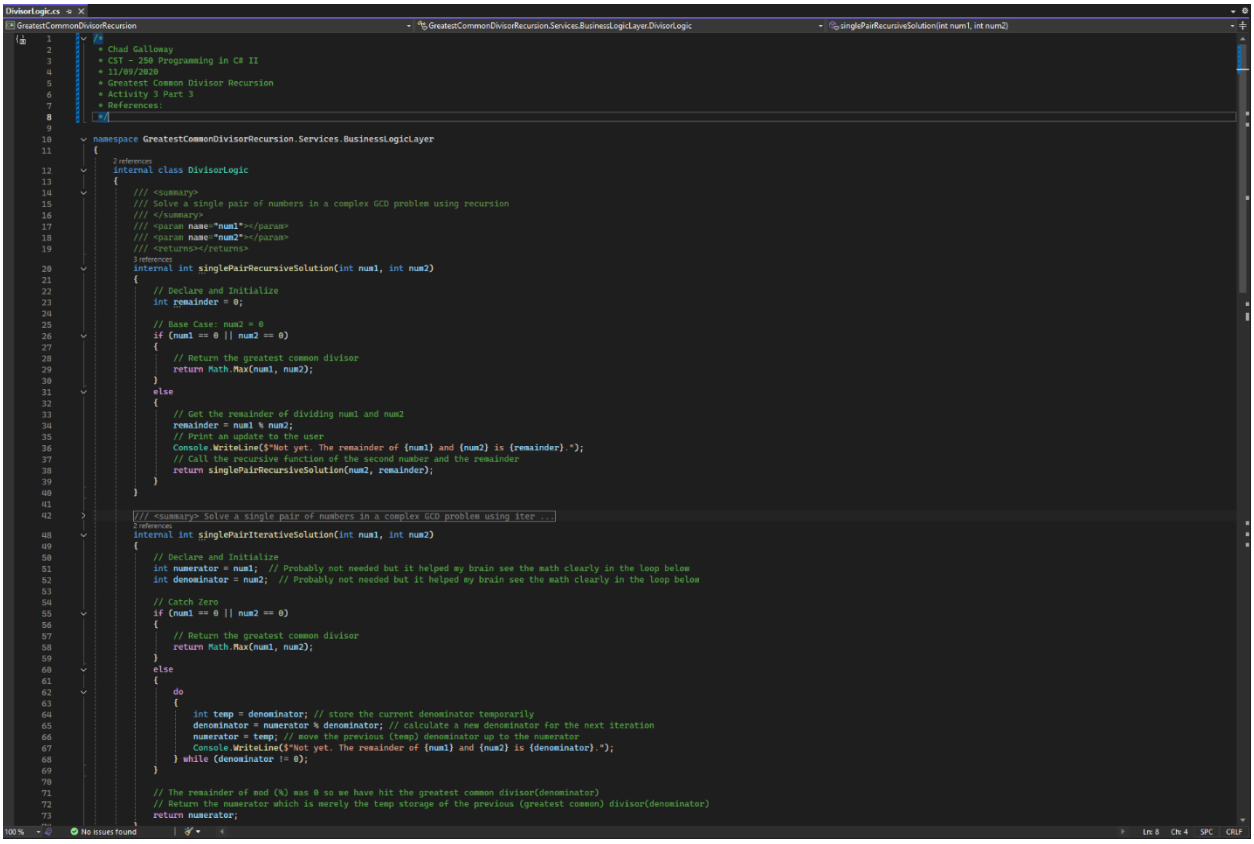
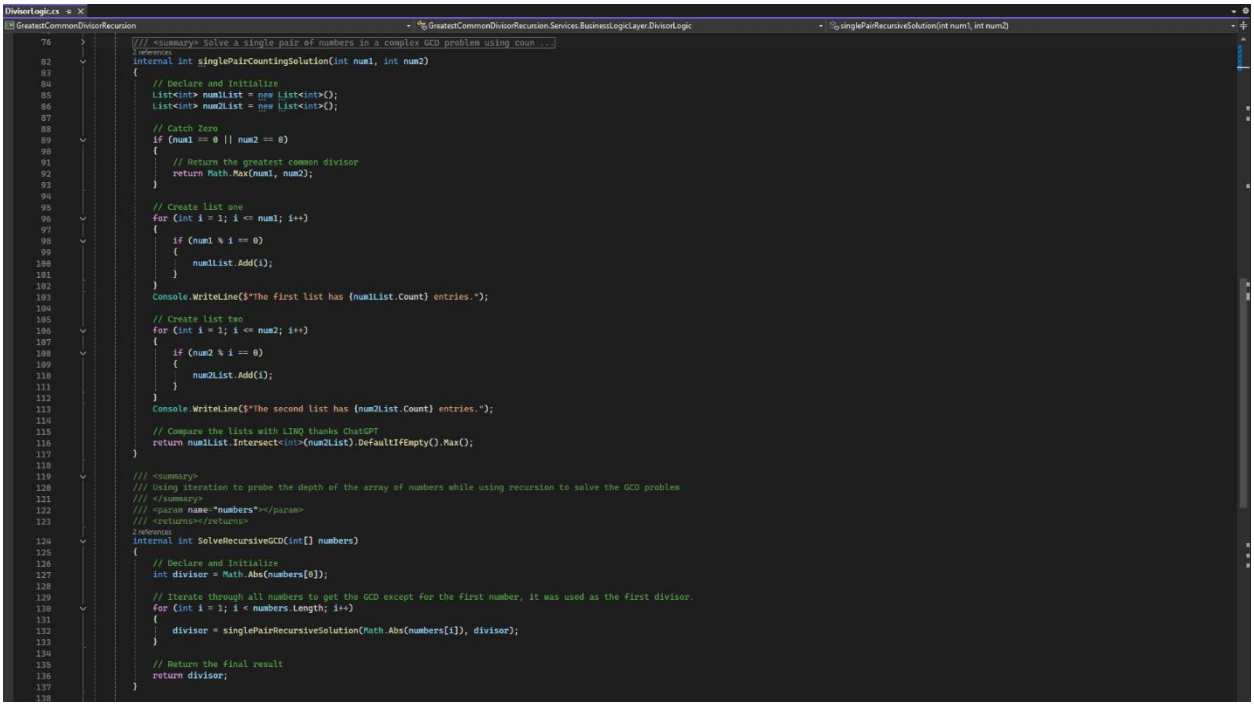


Figure 3-14: Screen shot of the utility class used to get users inputs.



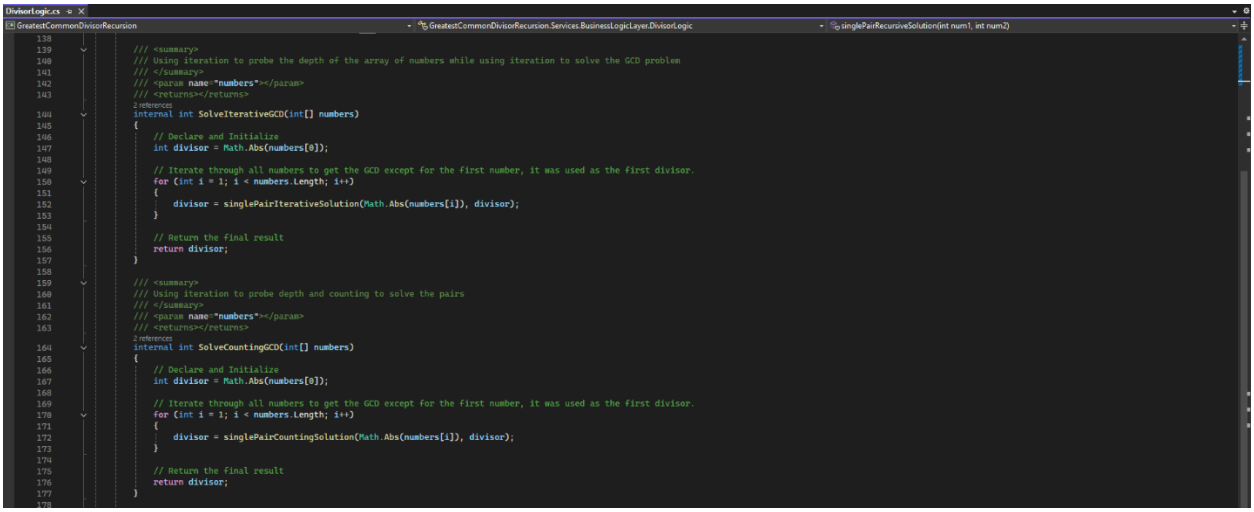
```
/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/09/2020
 * Greatest Common Divisor Recursion
 * Activity 3 Part 3
 * References:
 */

namespace GreatestCommonDivisorRecursion.Services.BusinessLogicLayer
{
    2 references
    internal class DivisorLogic
    {
        /// <summary>
        /// Solve a single pair of numbers in a complex GCD problem using recursion
        /// </summary>
        /// <param name="num1"></param>
        /// <param name="num2"></param>
        /// <returns></returns>
        3 references
        internal int singlePairRecursiveSolution(int num1, int num2)
        {
            // Declare and Initialize
            int remainder = 0;

            // Base Case: num2 = 0
            if (num1 == 0 || num2 == 0)
            {
                // Return the greatest common divisor
                return Math.Max(num1, num2);
            }
            else
            {
                // Get the remainder of dividing num1 and num2
                remainder = num1 % num2;
                // Print an update to the user
                Console.WriteLine($"Not yet. The remainder of {num1} and {num2} is {remainder}.");
                // Call the recursive function of the second number and the remainder
                return singlePairRecursiveSolution(num2, remainder);
            }
        }

        /// <summary> Solve a single pair of numbers in a complex GCD problem using iter ...
        2 references
        internal int singlePairIterativeSolution(int num1, int num2)
        {
            // Declare and Initialize
            int numerator = num1;  // Probably not needed but it helped my brain see the math clearly in the loop below
            int denominator = num2;  // Probably not needed but it helped my brain see the math clearly in the loop below

            // Catch Zero
            if (num1 == 0 || num2 == 0)
            {
                // Return the greatest common divisor
                return Math.Max(num1, num2);
            }
            else
            {
                do
                {
                    int temp = denominator; // store the current denominator temporarily
                    denominator = numerator % denominator; // calculate a new denominator for the next iteration
                    numerator = temp; // move the previous (temp) denominator up to the numerator
                    Console.WriteLine($"Not yet. The remainder of {num1} and {num2} is {denominator}.");
                } while (denominator != 0);
            }

            // The remainder of mod (%) was 0 so we have hit the greatest common divisor(denominator)
            // Return the numerator which is merely the temp storage of the previous (greatest common) divisor(denominator)
            return numerator;
```

Figure 3-15: Screen shot of the single pair GCD solution algorithms for recursion and iteration.



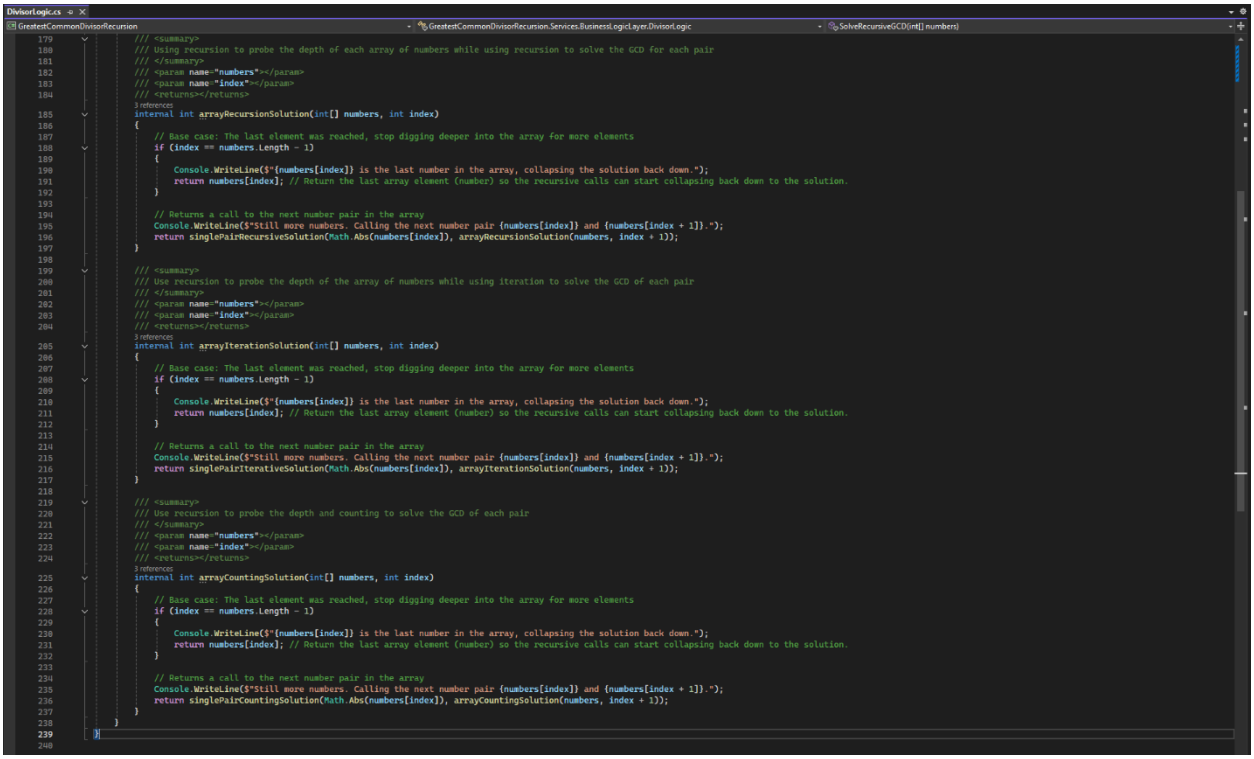
```
        /// <summary> Solve a single pair of numbers in a complex GCD problem using coun ...
        2 references
        internal int singlePairCountingSolution(int num1, int num2)
        {
            // Declare and Initialize
            List<int> num1List = new List<int>();
            List<int> num2List = new List<int>();

            // Catch Zero
            if (num1 == 0 || num2 == 0)
            {
                // Return the greatest common divisor
                return Math.Max(num1, num2);
            }

            // Create list one
            for (int i = 1; i <= num1; i++)
            {
                if (num1 % i == 0)
                {
                    num1List.Add(i);
                }
            }
            Console.WriteLine($"The first list has {num1List.Count} entries.");

            // Create list two
            for (int i = 1; i <= num2; i++)
            {
                if (num2 % i == 0)
                {
                    num2List.Add(i);
                }
            }
            Console.WriteLine($"The second list has {num2List.Count} entries.");

            // Compare the lists with LINQ thanks ChatGPT
            return num1List.Intersect<int>(num2List).DefaultIfEmpty().Max();
        }

        /// <summary>
        /// Using iteration to probe the depth of the array of numbers while using recursion to solve the GCD problem
        /// </summary>
        /// <param name="numbers"></param>
        /// <returns></returns>
        2 references
        internal int SolveRecursiveGCD(int[] numbers)
        {
            // Declare and Initialize
            int divisor = Math.Abs(numbers[0]);

            // Iterate through all numbers to get the GCD except for the first number, it was used as the first divisor.
            for (int i = 1; i < numbers.Length; i++)
            {
                divisor = singlePairRecursiveSolution(Math.Abs(numbers[i]), divisor);
            }

            // Return the final result
            return divisor;
        }
```

Figure 3-16 Screen shot of the final pair solution using counting from one, as per the worksheet, and the first iteration depth method for recursion pairs.



```
/// <summary>
/// Using iteration to probe the depth of the array of numbers while using iteration to solve the GCD problem
/// </summary>
/// <param name="numbers"></param>
/// <returns></returns>
2 references
internal int SolveIterativeGCD(int[] numbers)
{
    // Declare and Initialize
    int divisor = Math.Abs(numbers[0]);

    // Iterate through all numbers to get the GCD except for the first number, it was used as the first divisor.
    for (int i = 1; i < numbers.Length; i++)
    {
        divisor = singlePairIterativeSolution(Math.Abs(numbers[i]), divisor);
    }

    // Return the final result
    return divisor;
}

/// <summary>
/// Using iteration to probe depth and counting to solve the pairs
/// </summary>
/// <param name="numbers"></param>
/// <returns></returns>
2 references
internal int SolveCountingGCD(int[] numbers)
{
    // Declare and Initialize
    int divisor = Math.Abs(numbers[0]);

    // Iterate through all numbers to get the GCD except for the first number, it was used as the first divisor.
    for (int i = 1; i < numbers.Length; i++)
    {
        divisor = singlePairCountingSolution(Math.Abs(numbers[i]), divisor);
    }

    // Return the final result
    return divisor;
}
```

Figure 3-17: Screen shot of the final two iteration depth methods for iteration and counting pairs methods.



```
/// <summary>
/// Using recursion to probe the depth of each array of numbers while using recursion to solve the GCD for each pair
/// </summary>
/// <param name="numbers"></param>
/// <param name="index"></param>
/// <returns></returns>
3 references
internal int arrayRecursionSolution(int[] numbers, int index)
{
    // Base case: The last element was reached, stop digging deeper into the array for more elements
    if (index == numbers.Length - 1)
    {
        Console.WriteLine($"{numbers[index]} is the last number in the array, collapsing the solution back down.");
        return numbers[index]; // Return the last array element (number) so the recursive calls can start collapsing back down to the solution.
    }

    // Returns a call to the next number pair in the array
    Console.WriteLine($"Still more numbers. Calling the next number pair {numbers[index]} and {numbers[index + 1]}.");
    return singlePairRecursiveSolution(Math.Abs(numbers[index]), arrayRecursionSolution(numbers, index + 1));
}

/// <summary>
/// Use recursion to probe the depth of the array of numbers while using iteration to solve the GCD of each pair
/// </summary>
/// <param name="numbers"></param>
/// <param name="index"></param>
/// <returns></returns>
3 references
internal int arrayIterationSolution(int[] numbers, int index)
{
    // Base case: The last element was reached, stop digging deeper into the array for more elements
    if (index == numbers.Length - 1)
    {
        Console.WriteLine($"{numbers[index]} is the last number in the array, collapsing the solution back down.");
        return numbers[index]; // Return the last array element (number) so the recursive calls can start collapsing back down to the solution.
    }

    // Returns a call to the next number pair in the array
    Console.WriteLine($"Still more numbers. Calling the next number pair {numbers[index]} and {numbers[index + 1]}.");
    return singlePairIterativeSolution(Math.Abs(numbers[index]), arrayIterationSolution(numbers, index + 1));
}

/// <summary>
/// Use recursion to probe the depth and counting to solve the GCD of each pair
/// </summary>
/// <param name="numbers"></param>
/// <param name="index"></param>
/// <returns></returns>
3 references
internal int arrayCountingSolution(int[] numbers, int index)
{
    // Base case: The last element was reached, stop digging deeper into the array for more elements
    if (index == numbers.Length - 1)
    {
        Console.WriteLine($"{numbers[index]} is the last number in the array, collapsing the solution back down.");
        return numbers[index]; // Return the last array element (number) so the recursive calls can start collapsing back down to the solution.
    }

    // Returns a call to the next number pair in the array
    Console.WriteLine($"Still more numbers. Calling the next number pair {numbers[index]} and {numbers[index + 1]}.");
    return singlePairCountingSolution(Math.Abs(numbers[index]), arrayCountingSolution(numbers, index + 1));
}
}
```

Figure 3-18: Screen shot of the three recursive depth methods for iteration, recursion, and counting pairs.

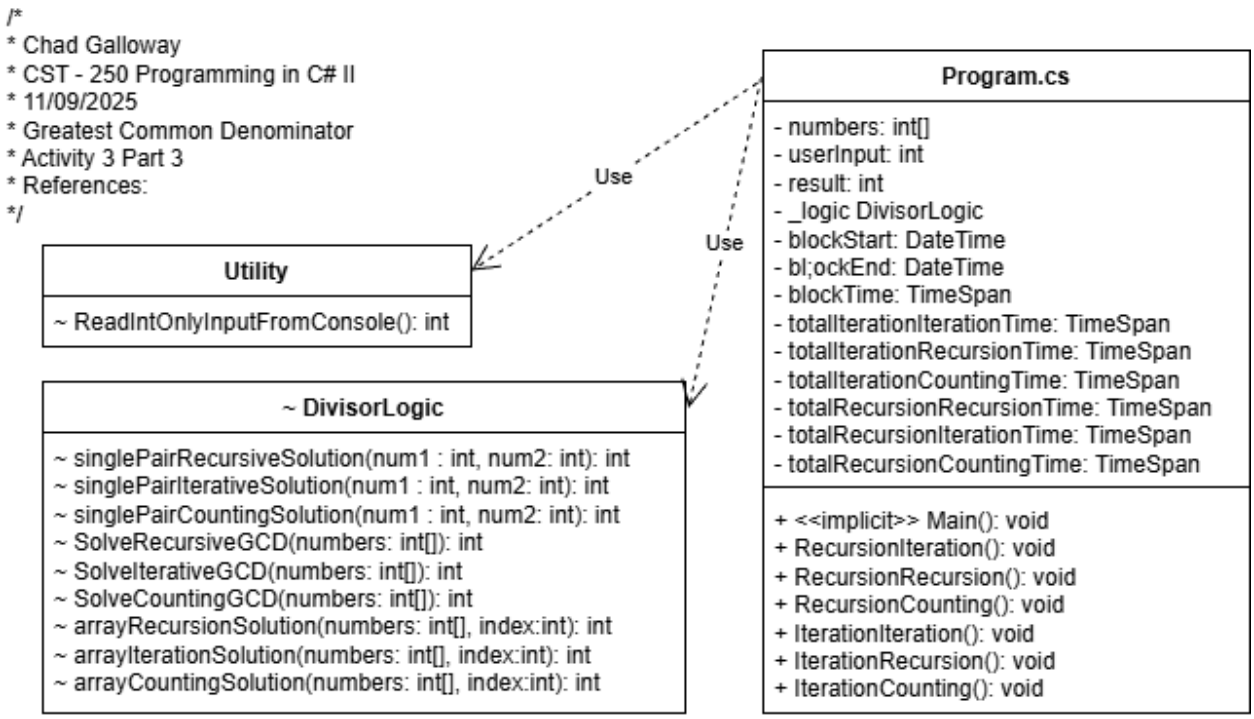


Figure 3-19: UML Changes for Part 3 GCD

Observations & Thinking

My train of thought on this set of challenges was based off of the tests I had done in the previous part when I was curious about the speeds. Adding the element of an array of numbers of unknown size made things interesting. I started by making the methods for finding the GCD of two numbers. I called these my single pairs. I created one of each type; recursion, iteration, and the counting up from one implementation described in the worksheet. With those sorted I had to figure out arrays so the user could enter more than two numbers. I knew I could easily loop through the array with a for loop and call whichever pair method I wanted to after that. What I was unsure of was whether I could use recursion for array traversal. As it turns out you can. By calling a nested recursion loop you can recursively traverse the array and at each step call the recursion method for figuring out pairs of numbers. After playing around with different methodologies I decided to run two branches, an iteration branch that would iterate through the array of numbers and call each pair function I had created, and a recursion branch that would call one of the pair methods with a call to itself inside one of the parameters for that particular pair method. Basically, the recursion traversal method would call the pair method which would need to call the traversal method which would call the pair method which would need to call the traversal method…. You get the point. This would continue until the last element in the array was reached and the traversal method would quit calling more pair methods and the stack would start collapsing solving the problem of GCD one pair at a time. After sorting out both array traversal methodologies I moved my focus to tracking time. My work in the previous part helped a great deal. I found, then, that the first method calls, for some reason (I guess the cost and time to allocate resources in Windows), take a considerable amount of time more and for this reason I discarded the times of each of the first calls for each method chain. I threw away the one call and then called each method type (there were six) 10 more times in mixed up orders to try and get the results as random and distributed as I could. I added in some color coding on the output to help tracking results and error better. What I found thru a lot of testing is that almost every time iteration depth with recursion pair solving was the fastest. Some times iteration depth and iteration pair solving would squeak out a small win. And on super rare occasions, usually on only two numbers iteration with counting pairs was fastest. What I did observe was that recursion array traversal was never the fastest method. Recursion traversal was consistently almost twice as slow as the iteration counter part on traversal. Traversal seems to matter a great deal on the time it takes but different pair solutions are a bit more forgiving on choice of solution method when time is the measuring stick.

PART 4

Flow Chart

```
/*
* Chad Galloway
* CST - 250 Programming in C# II
* 11/09/2025
* Flood Fill
* Activity 3 Part 4
* References:
*/
```

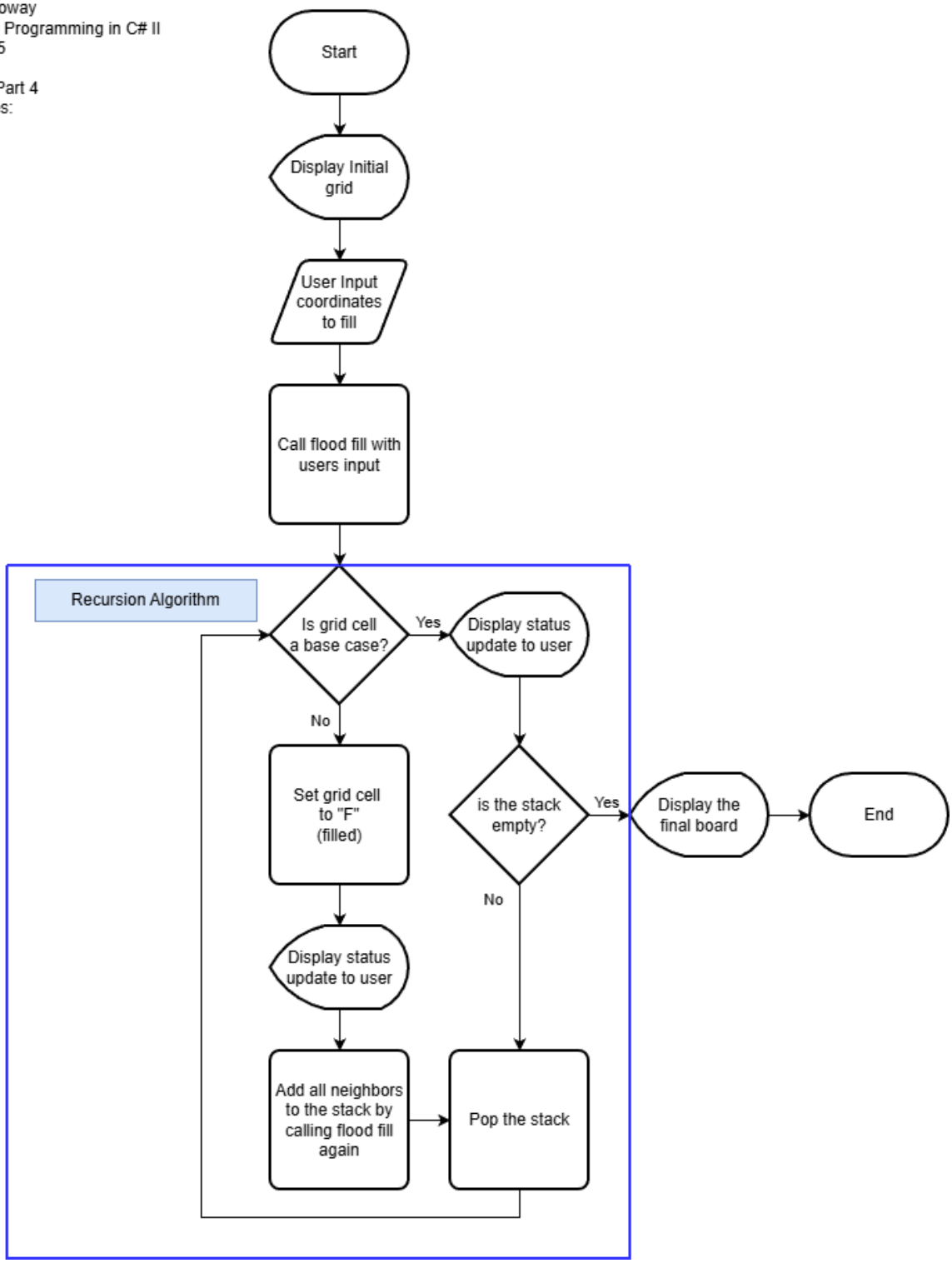


Figure 4-1: Flow chart of Activity 3 Part 4 flood fill recursion

UML Class Diagram

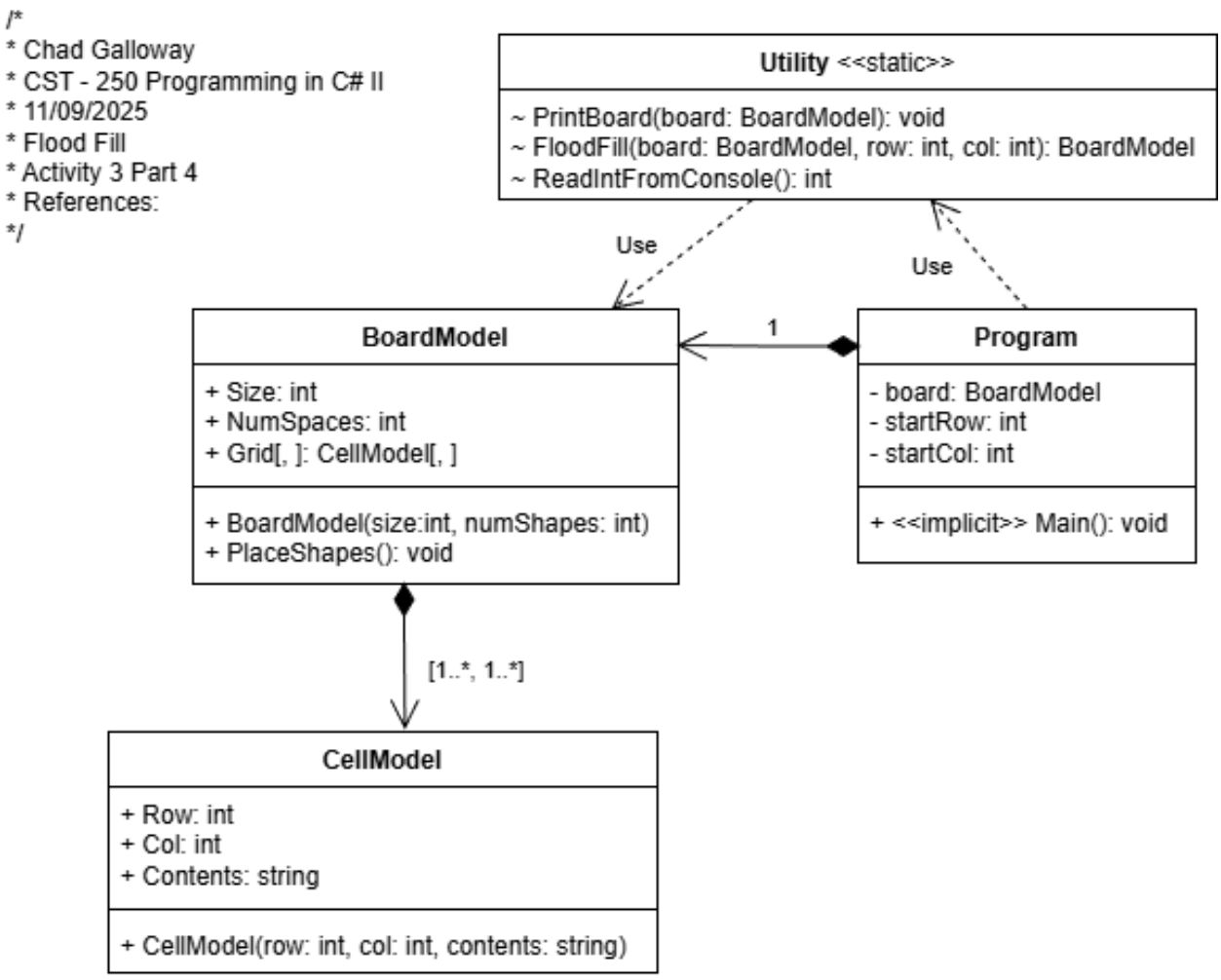


Figure X: UML Class Diagram for Activity 3 part 4

Screen Shots

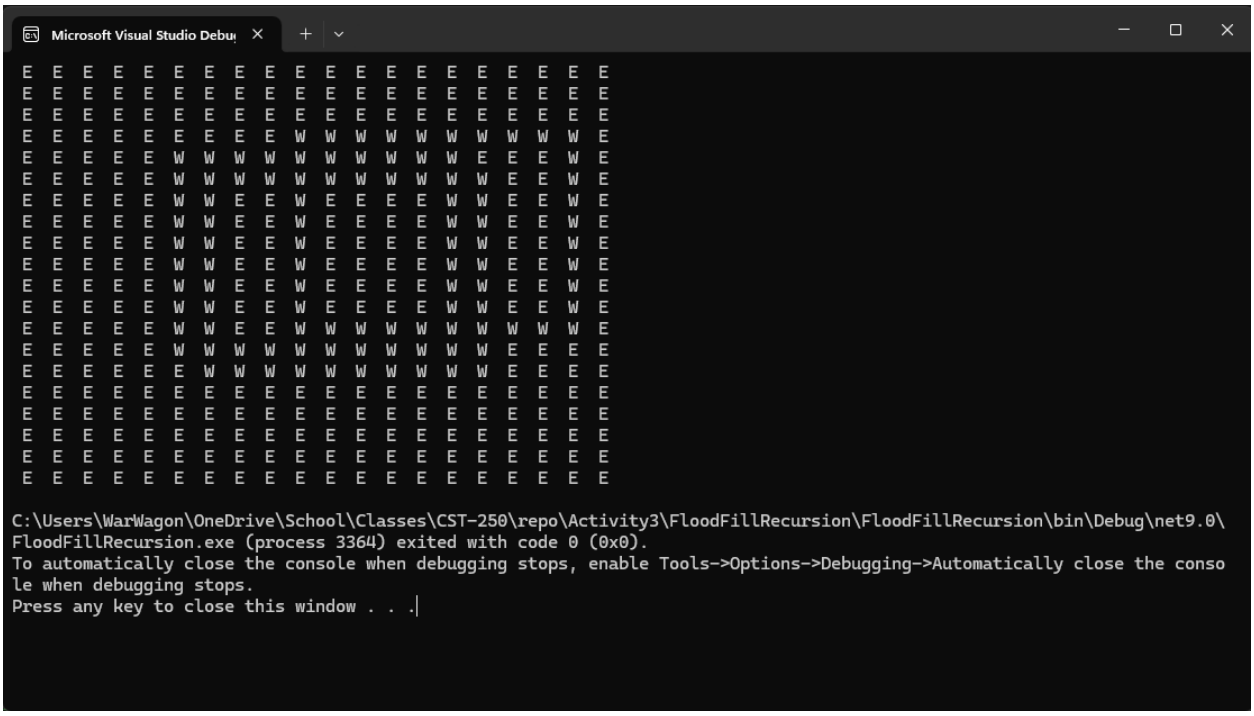


Figure 4-3: Part 4 App Running for the first time

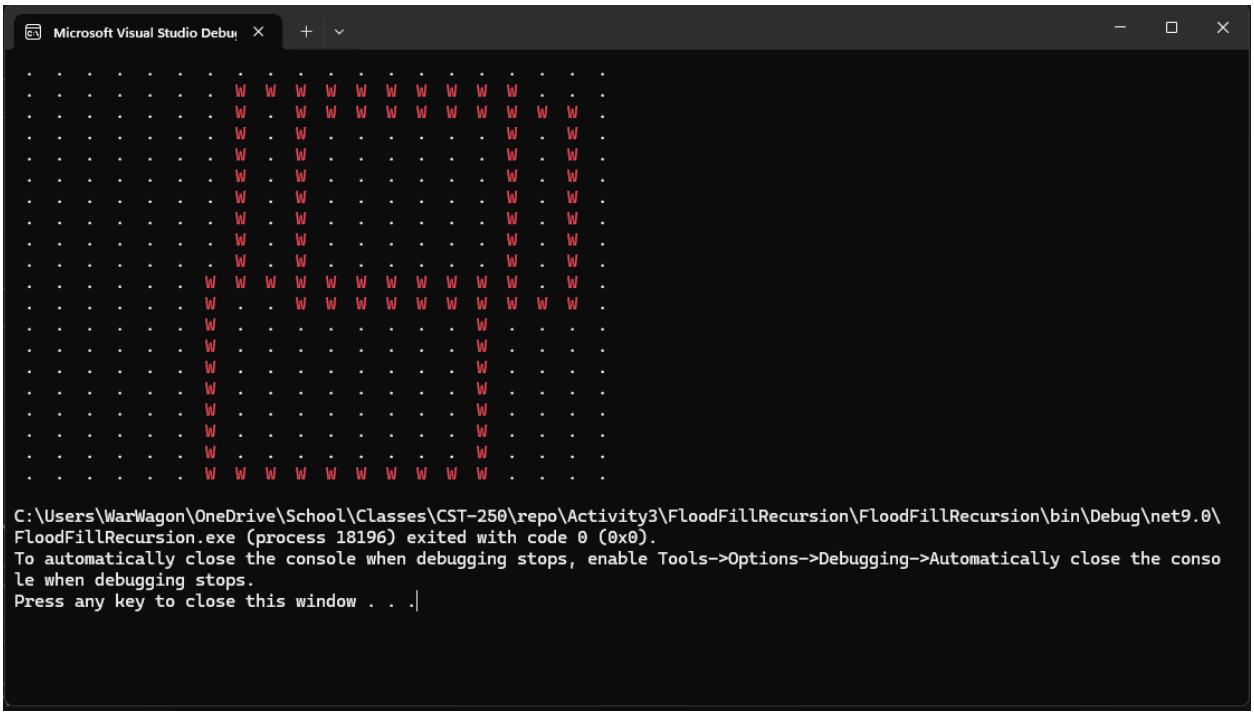


Figure 4-4: Part 4 App Running for the second time with colored shapes.

Figure 4-5: The addition of row and column number.

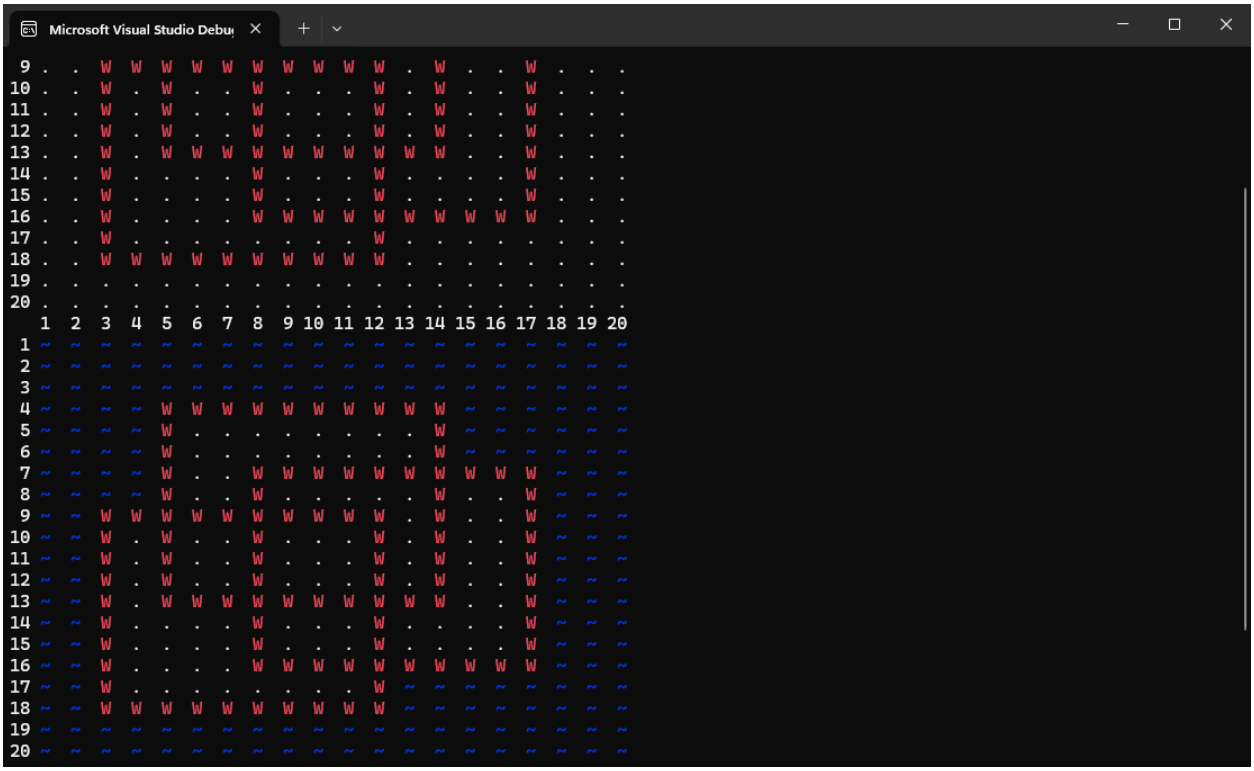
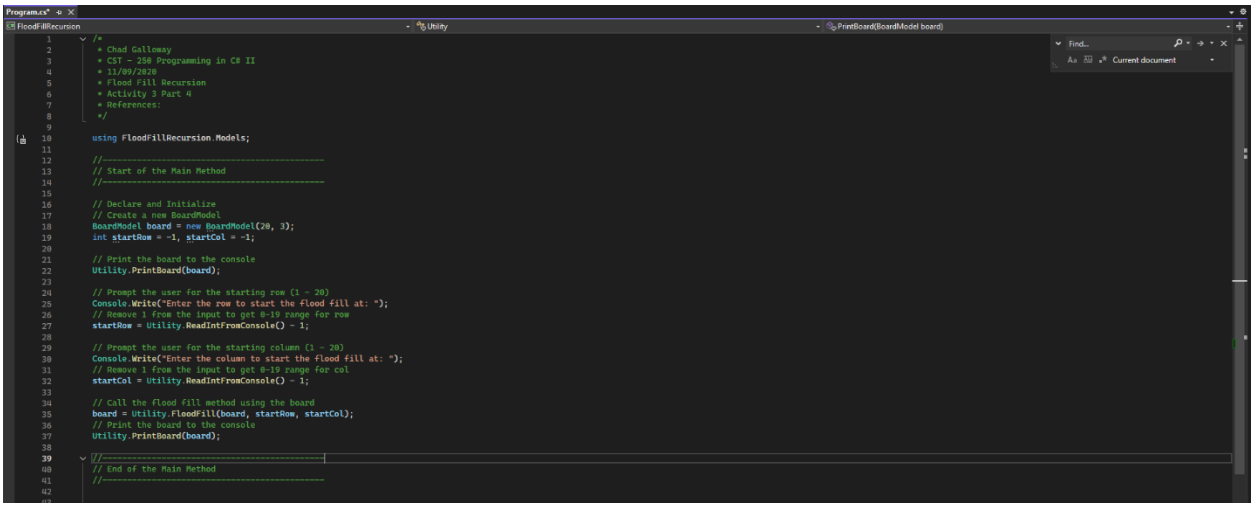


Figure 4-6: Flood fill implemented with blue tilde.

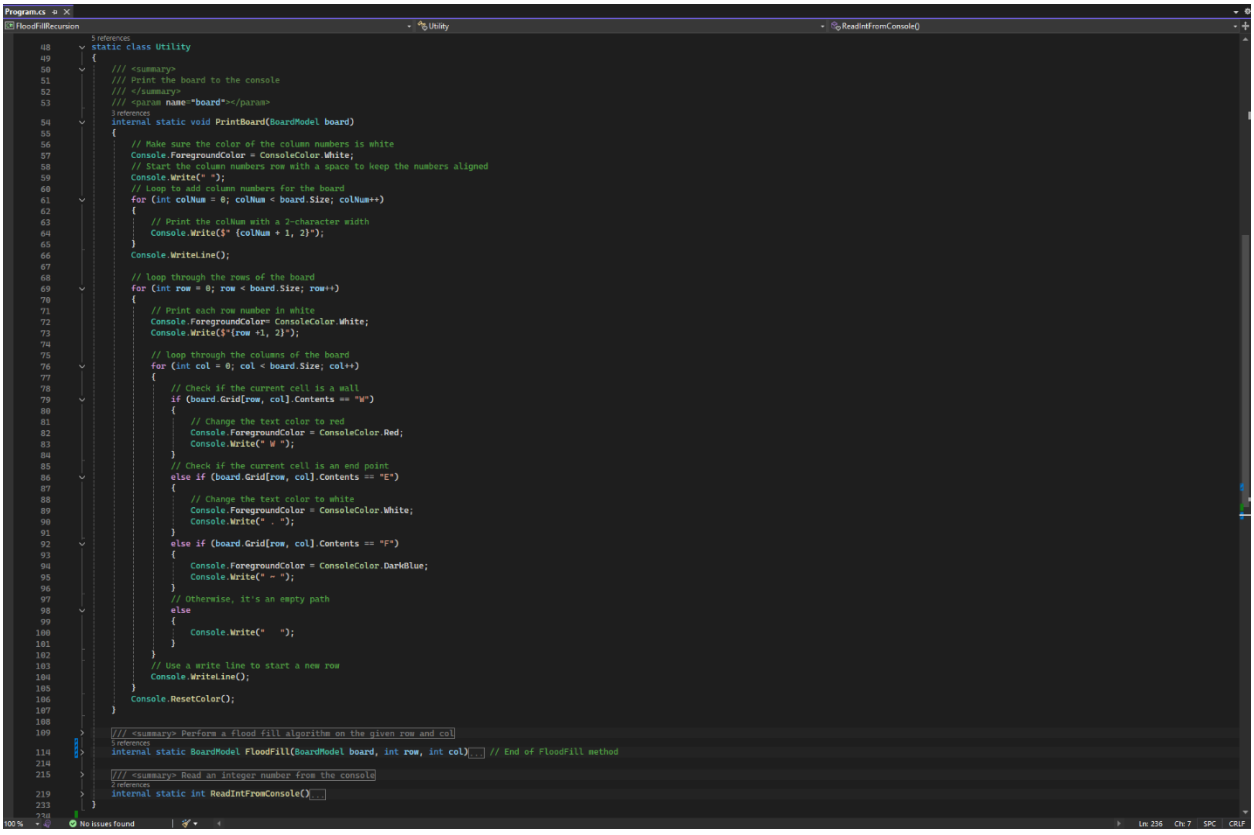
Figure 4-7: Completed implementing user selected flood fill start location

Code



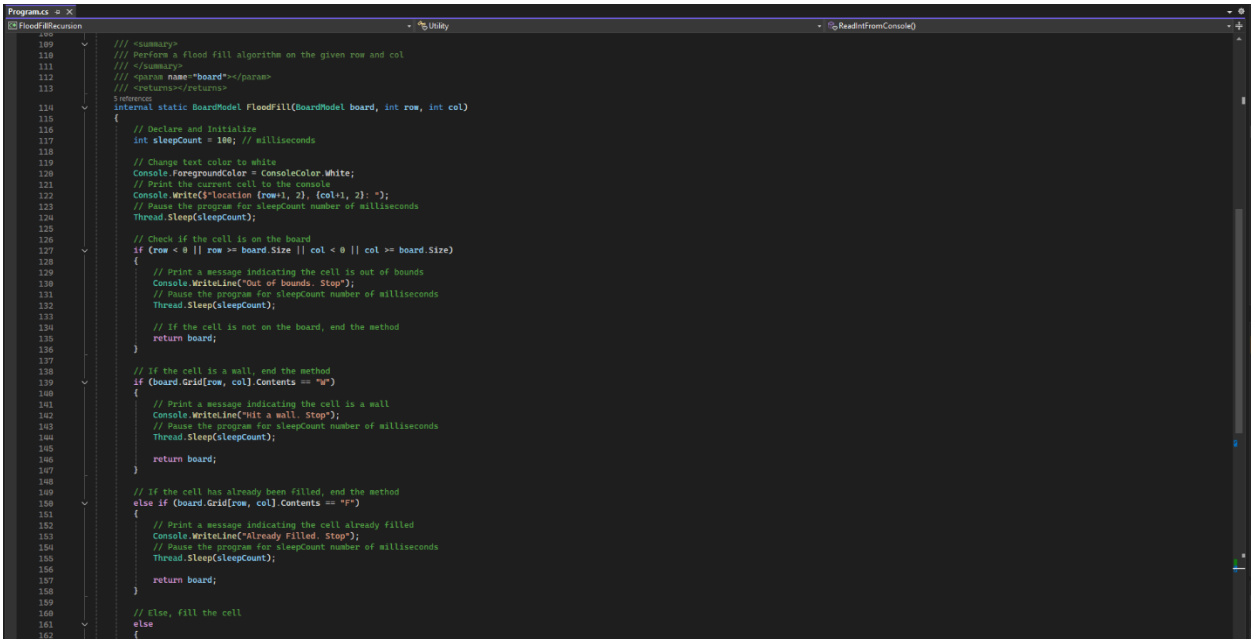
```
/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/09/2020
 * Flood Fill Recursion
 * Activity 3 Part 4
 * References:
 */

using FloodFillRecursion.Models;

//---------------------------------------------
// Start of the Main Method
//---------------------------------------------

// Declare and Initialize
// Create a new BoardModel
BoardModel board = new BoardModel(20, 3);
int startRow = -1, startCol = -1;

// Print the board to the console
Utility.PrintBoard(board);

// Prompt the user for the starting row (1 - 20)
Console.Write("Enter the row to start the flood fill at: ");
// Remove 1 from the input to get 0-19 range for row
startRow = Utility.ReadIntFromConsole() - 1;

// Prompt the user for the starting column (1 - 20)
Console.Write("Enter the column to start the flood fill at: ");
// Remove 1 from the input to get 0-19 range for col
startCol = Utility.ReadIntFromConsole() - 1;

// Call the flood fill method using the board
board = Utility.FloodFill(board, startRow, startCol);
// Print the board to the console
Utility.PrintBoard(board);

//---------------------------------------------
// End of the Main Method
//---------------------------------------------
```

Figure 4-8: Code Citations and the main method.



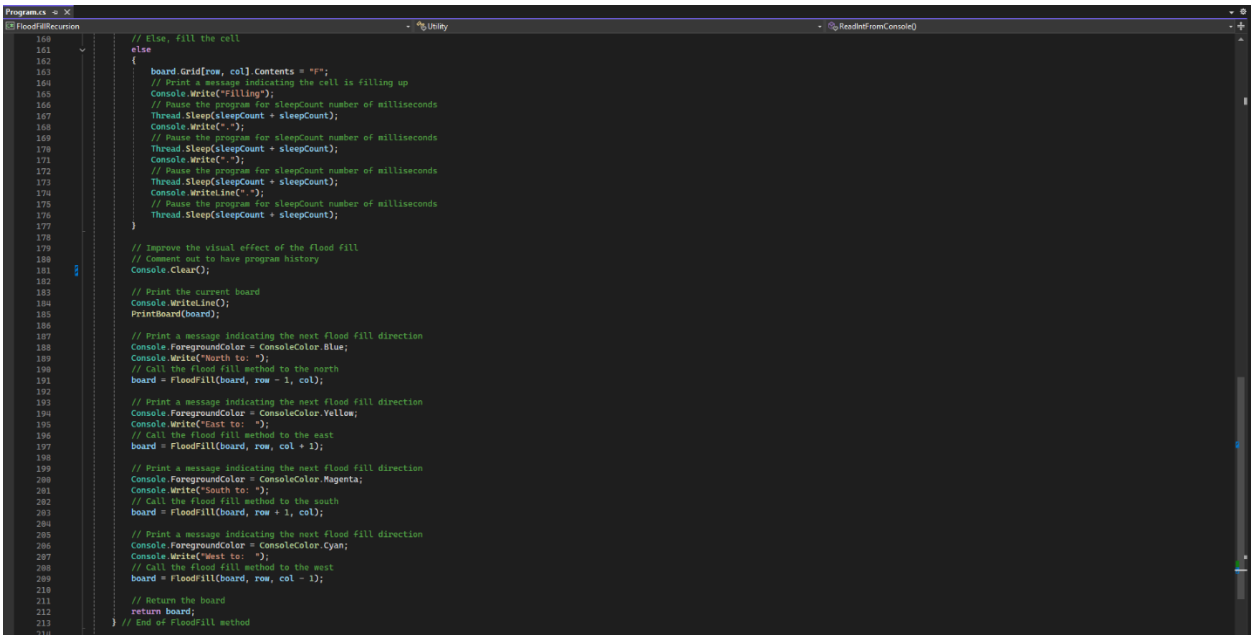
```
static class Utility
{
    /// <summary>
    /// Print the board to the console
    /// </summary>
    /// <param name="board"></param>
    internal static void PrintBoard(BoardModel board)
    {
        // Make sure the color of the column numbers is white
        Console.ForegroundColor = ConsoleColor.White;
        // Start the colum numbers row with a space to keep the numbers aligned
        Console.Write(" ");
        // Loop to add column numbers for the board
        for (int colNum = 0; colNum < board.Size; colNum++)
        {
            // Print the colNum with a 2-character width
            Console.Write($" {colNum + 1, 2}");
        }
        Console.WriteLine();

        // loop through the rows of the board
        for (int row = 0; row < board.Size; row++)
        {
            // Print each row number in white
            Console.ForegroundColor= ConsoleColor.White;
            Console.Write($"{row +1, 2}");

            // loop through the columns of the board
            for (int col = 0; col < board.Size; col++)
            {
                // Check if the current cell is a wall
                if (board.Grid[row, col].Contents == "W")
                {
                    // Change the text color to red
                    Console.ForegroundColor = ConsoleColor.Red;
                    Console.Write(" W ");
                }
                // Check if the current cell is an end point
                else if (board.Grid[row, col].Contents == "E")
                {
                    // Change the text color to white
                    Console.ForegroundColor = ConsoleColor.White;
                    Console.Write(" . ");
                }
                else if (board.Grid[row, col].Contents == "F")
                {
                    Console.ForegroundColor = ConsoleColor.DarkBlue;
                    Console.Write(" ~ ");
                }
                // Otherwise, it's an empty path
                else
                {
                    Console.Write("   ");
                }
            }
            // Use a write line to start a new row
            Console.WriteLine();
        }
        Console.ResetColor();
    }

    /// <summary> Perform a flood fill algorithm on the given row and col
    internal static BoardModel FloodFill(BoardModel board, int row, int col) // End of FloodFill method

    /// <summary> Read an integer number from the console
    internal static int ReadIntFromConsole()
}
```

Figure 4-9: Utility class and the print board method.



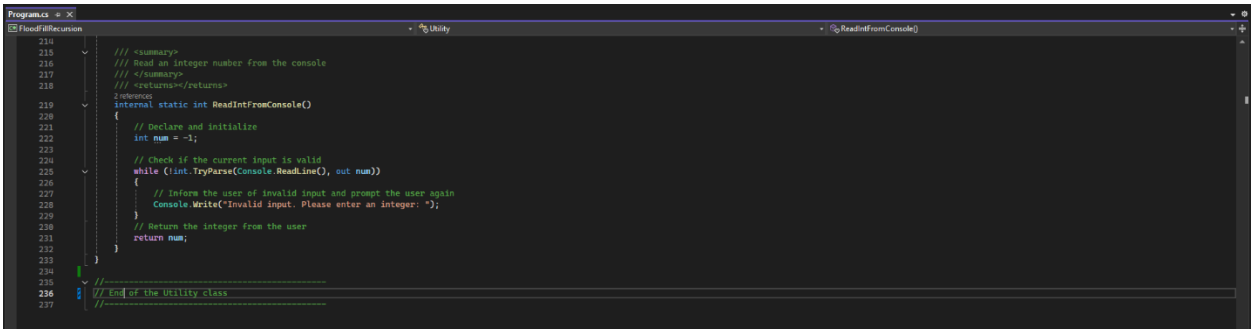
```
Programs.cs
FloodFillRecursion
Utility
ReadIntFromConsole()
109     /// <summary>
110     /// Perform a flood fill algorithm on the given row and col
111     /// </summary>
112     /// <param name="board"></param>
113     /// <returns></returns>
        5 references
114     internal static BoardModel FloodFill(BoardModel board, int row, int col)
115     {
116         // Declare and Initialize
117         int sleepCount = 100; // milliseconds
118
119         // Change text color to white
120         Console.ForegroundColor = ConsoleColor.White;
121         // Print the current cell to the console
122         Console.Write($"location {row+1, 2}, {col+1, 2}: ");
123         // Pause the program for sleepCount number of milliseconds
124         Thread.Sleep(sleepCount);
125
126         // Check if the cell is on the board
127         if (row < 0 || row >= board.Size || col < 0 || col >= board.Size)
128         {
129             // Print a message indicating the cell is out of bounds
130             Console.WriteLine("Out of bounds. Stop");
131             // Pause the program for sleepCount number of milliseconds
132             Thread.Sleep(sleepCount);
133
134             // If the cell is not on the board, end the method
135             return board;
136         }
137
138         // If the cell is a wall, end the method
139         if (board.Grid[row, col].Contents == "W")
140         {
141             // Print a message indicating the cell is a wall
142             Console.WriteLine("Hit a wall. Stop");
143             // Pause the program for sleepCount number of milliseconds
144             Thread.Sleep(sleepCount);
145
146             return board;
147         }
148
149         // If the cell has already been filled, end the method
150         else if (board.Grid[row, col].Contents == "F")
151         {
152             // Print a message indicating the cell already filled
153             Console.WriteLine("Already Filled. Stop");
154             // Pause the program for sleepCount number of milliseconds
155             Thread.Sleep(sleepCount);
156
157             return board;
158         }
159
160         // Else, fill the cell
161         else
162         {
```

Figure 4-10: Flood Fill Initializations and base cases.



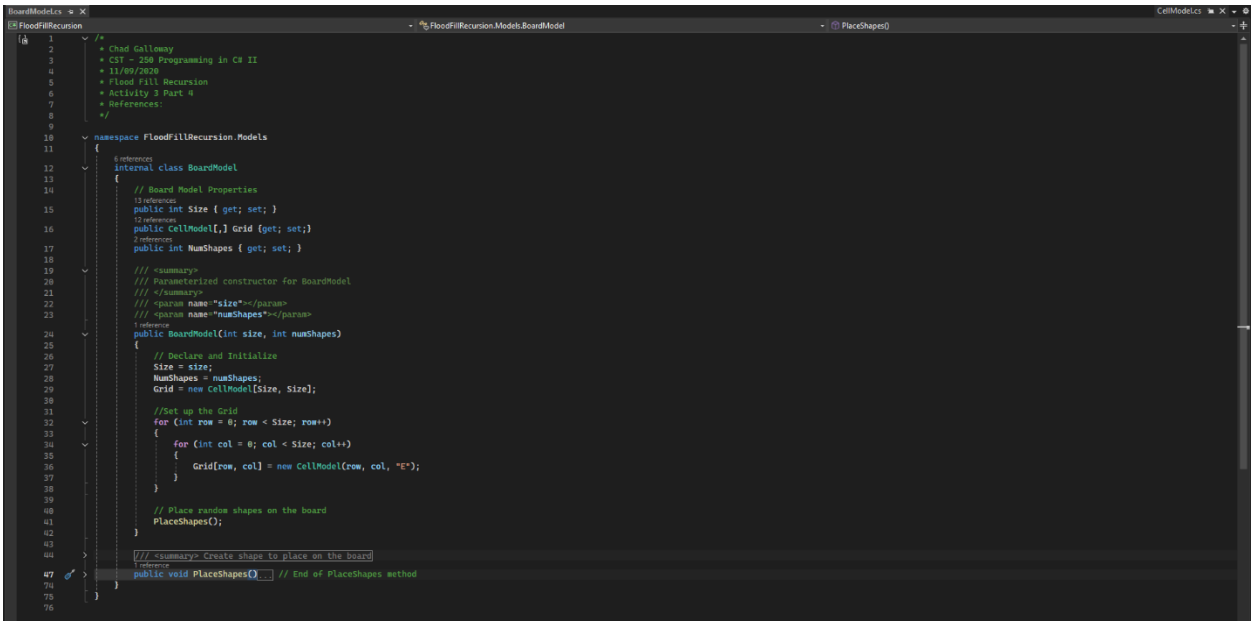
```
Programs.cs
FloodFillRecursion
Utility
ReadIntFromConsole()
160         // Else, fill the cell
161         else
162         {
163             board.Grid[row, col].Contents = "F";
164             // Print a message indicating the cell is filling up
165             Console.Write("Filling");
166             // Pause the program for sleepCount number of milliseconds
167             Thread.Sleep(sleepCount + sleepCount);
168             Console.Write(".");
169             // Pause the program for sleepCount number of milliseconds
170             Thread.Sleep(sleepCount + sleepCount);
171             Console.Write(".");
172             // Pause the program for sleepCount number of milliseconds
173             Thread.Sleep(sleepCount + sleepCount);
174             Console.WriteLine(".");
175             // Pause the program for sleepCount number of milliseconds
176             Thread.Sleep(sleepCount + sleepCount);
177         }
178
179         // Improve the visual effect of the flood fill
180         // Comment out to have program history
181         Console.Clear();
182
183         // Print the current board
184         Console.WriteLine();
185         PrintBoard(board);
186
187         // Print a message indicating the next flood fill direction
188         Console.ForegroundColor = ConsoleColor.Blue;
189         Console.Write("North to: ");
190         // Call the flood fill method to the north
191         board = FloodFill(board, row - 1, col);
192
193         // Print a message indicating the next flood fill direction
194         Console.ForegroundColor = ConsoleColor.Yellow;
195         Console.Write("East to:  ");
196         // Call the flood fill method to the east
197         board = FloodFill(board, row, col + 1);
198
199         // Print a message indicating the next flood fill direction
200         Console.ForegroundColor = ConsoleColor.Magenta;
201         Console.Write("South to: ");
202         // Call the flood fill method to the south
203         board = FloodFill(board, row + 1, col);
204
205         // Print a message indicating the next flood fill direction
206         Console.ForegroundColor = ConsoleColor.Cyan;
207         Console.Write("West to:  ");
208         // Call the flood fill method to the west
209         board = FloodFill(board, row, col - 1);
210
211         // Return the board
212         return board;
213     } // End of FloodFill method
```

Figure 4-11: Flood Fill recursion calls and visual effects



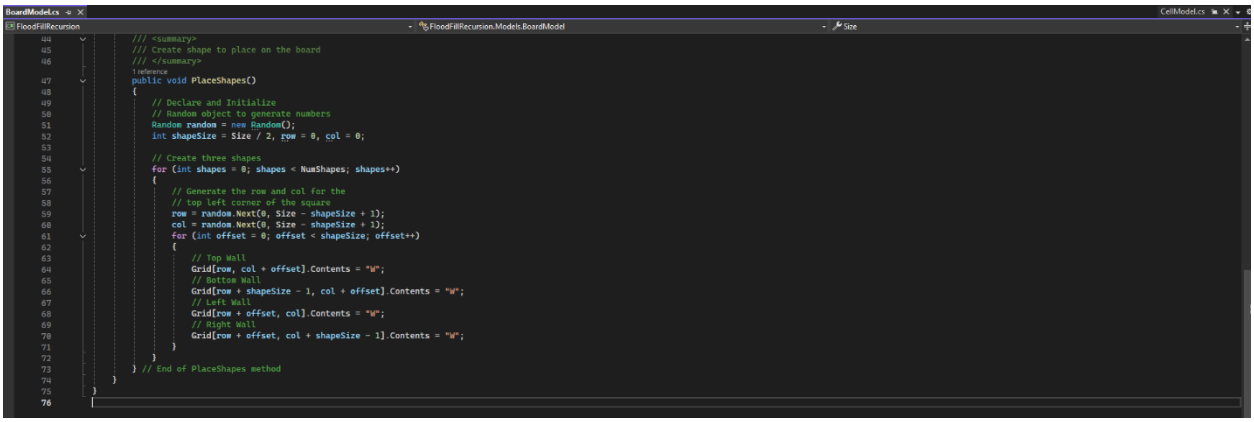
```
Program.cs
FloodFillRecursion
Utility
ReadIntFromConsole()

    /// <summary>
    /// Read an integer number from the console
    /// </summary>
    /// <returns></returns>
    2 references
    internal static int ReadIntFromConsole()
    {
        // Declare and initialize
        int num = -1;

        // Check if the current input is valid
        while (!int.TryParse(Console.ReadLine(), out num))
        {
            // Inform the user of invalid input and prompt the user again
            Console.Write("Invalid input. Please enter an integer: ");
        }
        // Return the integer from the user
        return num;
    }
}

//----------------------------------------
// End of the Utility class
//----------------------------------------
```

Figure 4-12: ReadIntFromConsole() method from the Utility class



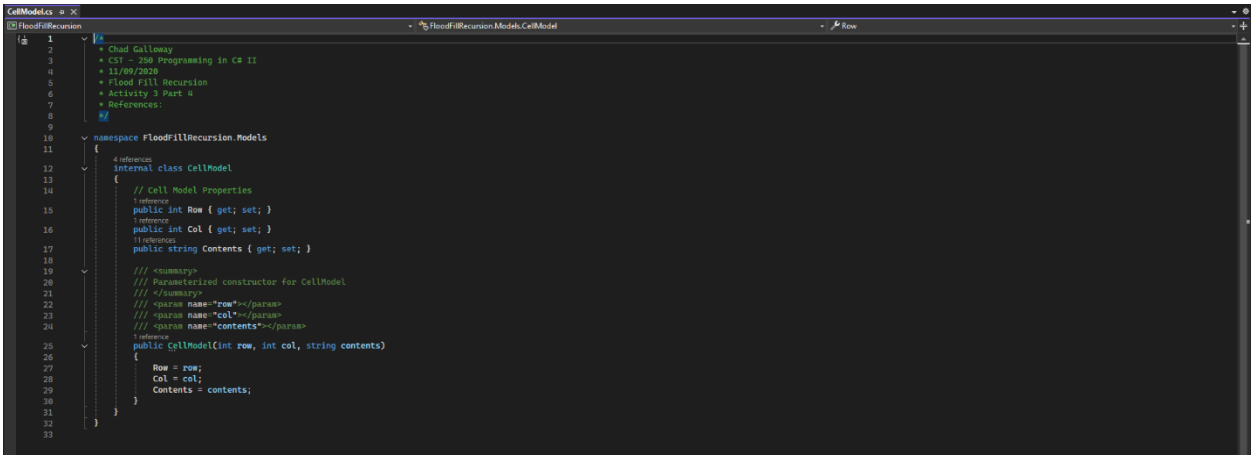
```
BoardModel.cs
CellModel.cs
FloodFillRecursion
FloodFillRecursion.Models.BoardModel
PlaceShapes()

/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/09/2020
 * Flood Fill Recursion
 * Activity 3 Part 4
 * References:
 */

namespace FloodFillRecursion.Models
{
    6 references
    internal class BoardModel
    {
        // Board Model Properties
        13 references
        public int Size { get; set; }
        12 references
        public CellModel[,] Grid {get; set;}
        2 references
        public int NumShapes { get; set; }

        /// <summary>
        /// Parameterized constructor for BoardModel
        /// </summary>
        /// <param name="size"></param>
        /// <param name="numShapes"></param>
        1 reference
        public BoardModel(int size, int numShapes)
        {
            // Declare and Initialize
            Size = size;
            NumShapes = numShapes;
            Grid = new CellModel[Size, Size];

            //Set up the Grid
            for (int row = 0; row < Size; row++)
            {
                for (int col = 0; col < Size; col++)
                {
                    Grid[row, col] = new CellModel(row, col, "E");
                }
            }

            // Place random shapes on the board
            PlaceShapes();
        }

        /// <summary> Create shape to place on the board
        1 reference
        public void PlaceShapes() ... // End of PlaceShapes method
    }
}
```

Figure 4-13: Board model class citations and constructor



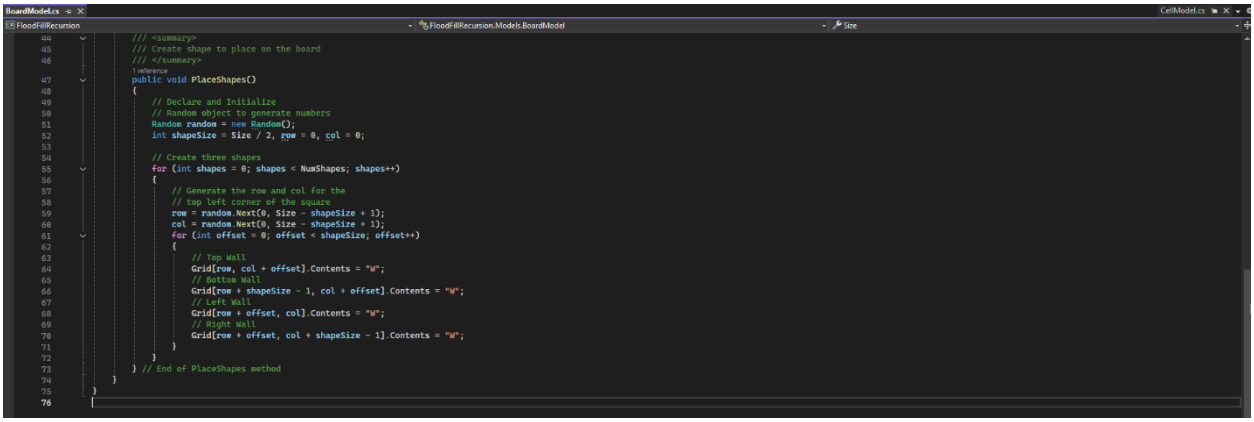
```
BoardModel.cs
CellModel.cs
FloodFillRecursion
FloodFillRecursion.Models.BoardModel
Size

        /// <summary>
        /// Create shape to place on the board
        /// </summary>
        1 reference
        public void PlaceShapes()
        {
            // Declare and Initialize
            // Random object to generate numbers
            Random random = new Random();
            int shapeSize = Size / 2, row = 0, col = 0;

            // Create three shapes
            for (int shapes = 0; shapes < NumShapes; shapes++)
            {
                // Generate the row and col for the
                // top left corner of the square
                row = random.Next(0, Size - shapeSize + 1);
                col = random.Next(0, Size - shapeSize + 1);
                for (int offset = 0; offset < shapeSize; offset++)
                {
                    // Top Wall
                    Grid[row, col + offset].Contents = "W";
                    // Bottom Wall
                    Grid[row + shapeSize - 1, col + offset].Contents = "W";
                    // Left Wall
                    Grid[row + offset, col].Contents = "W";
                    // Right Wall
                    Grid[row + offset, col + shapeSize - 1].Contents = "W";
                }
            }
        } // End of PlaceShapes method
    }
}
```

Figure 4-14: Board model place shapes method



```
/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/09/2020
 * Flood Fill Recursion
 * Activity 3 Part 4
 * References:
 */

namespace FloodFillRecursion.Models
{
    4 references
    internal class CellModel
    {
        // Cell Model Properties
        1 reference
        public int Row { get; set; }
        1 reference
        public int Col { get; set; }
        11 references
        public string Contents { get; set; }

        /// <summary>
        /// Parameterized constructor for CellModel
        /// </summary>
        /// <param name="row"></param>
        /// <param name="col"></param>
        /// <param name="contents"></param>
        1 reference
        public CellModel(int row, int col, string contents)
        {
            Row = row;
            Col = col;
            Contents = contents;
        }
    }
}
```

Figure 4-15: Cell model class citations and constructor



```
        /// <summary>
        /// Create shape to place on the board
        /// </summary>
        1 reference
        public void PlaceShapes()
        {
            // Declare and Initialize
            // Random object to generate numbers
            Random random = new Random();
            int shapeSize = Size / 2, row = 0, col = 0;

            // Create three shapes
            for (int shapes = 0; shapes < NumShapes; shapes++)
            {
                // Generate the row and col for the
                // top left corner of the square
                row = random.Next(0, Size - shapeSize + 1);
                col = random.Next(0, Size - shapeSize + 1);
                for (int offset = 0; offset < shapeSize; offset++)
                {
                    // Top Wall
                    Grid[row, col + offset].Contents = "W";
                    // Bottom Wall
                    Grid[row + shapeSize - 1, col + offset].Contents = "W";
                    // Left Wall
                    Grid[row + offset, col].Contents = "W";
                    // Right Wall
                    Grid[row + offset, col + shapeSize - 1].Contents = "W";
                }
            }
        } // End of PlaceShapes method
    }
}
```

Figure 4-16: UML for Flood Fill Project

PART 4 CHALLENGES

Screen Shots

Figure 4-17: Challenge one running west, east, south, then north.

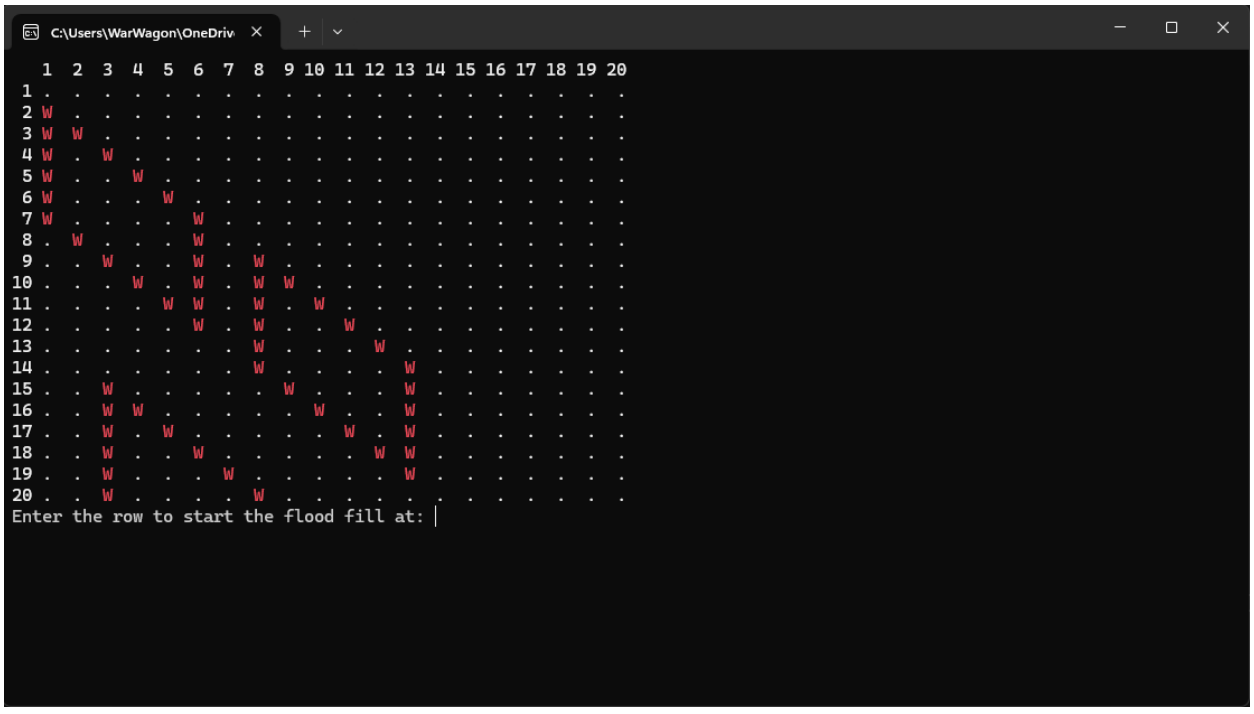


Figure 4-18: Challenge two spawning different shapes

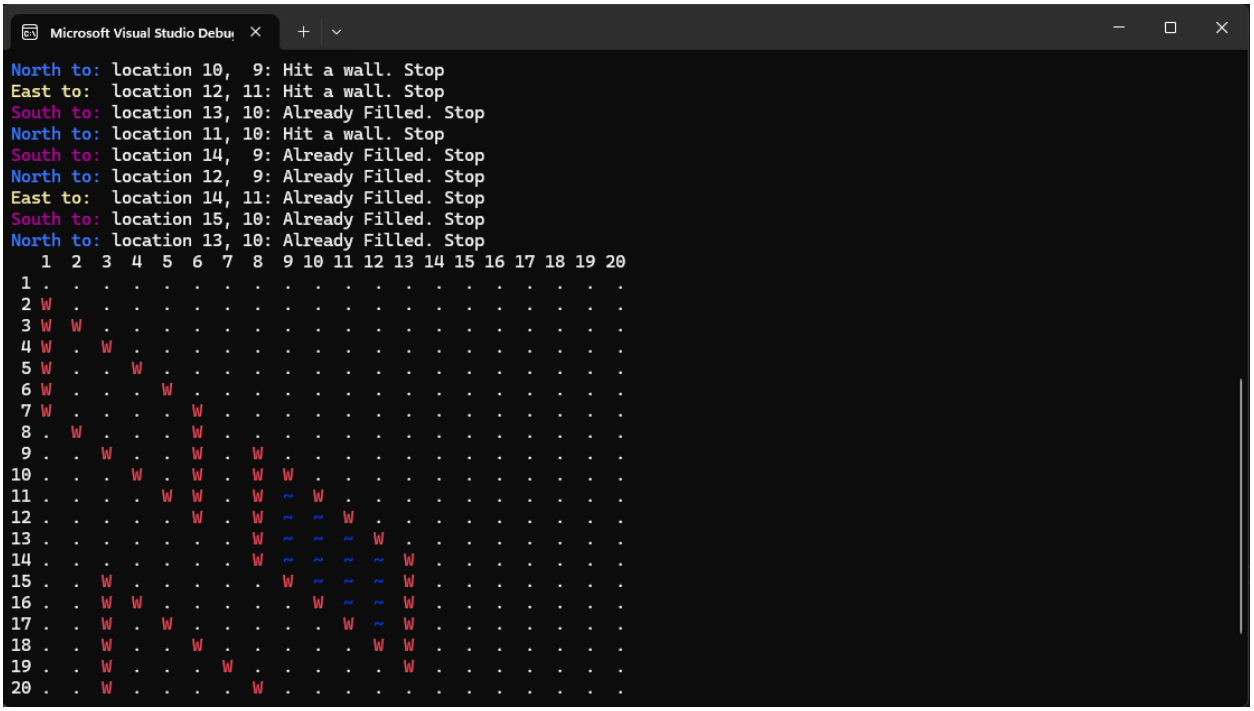


Figure 4-19: Challenge two fill completed. No jumping over diagonals with original north south east west; four direction flooding.

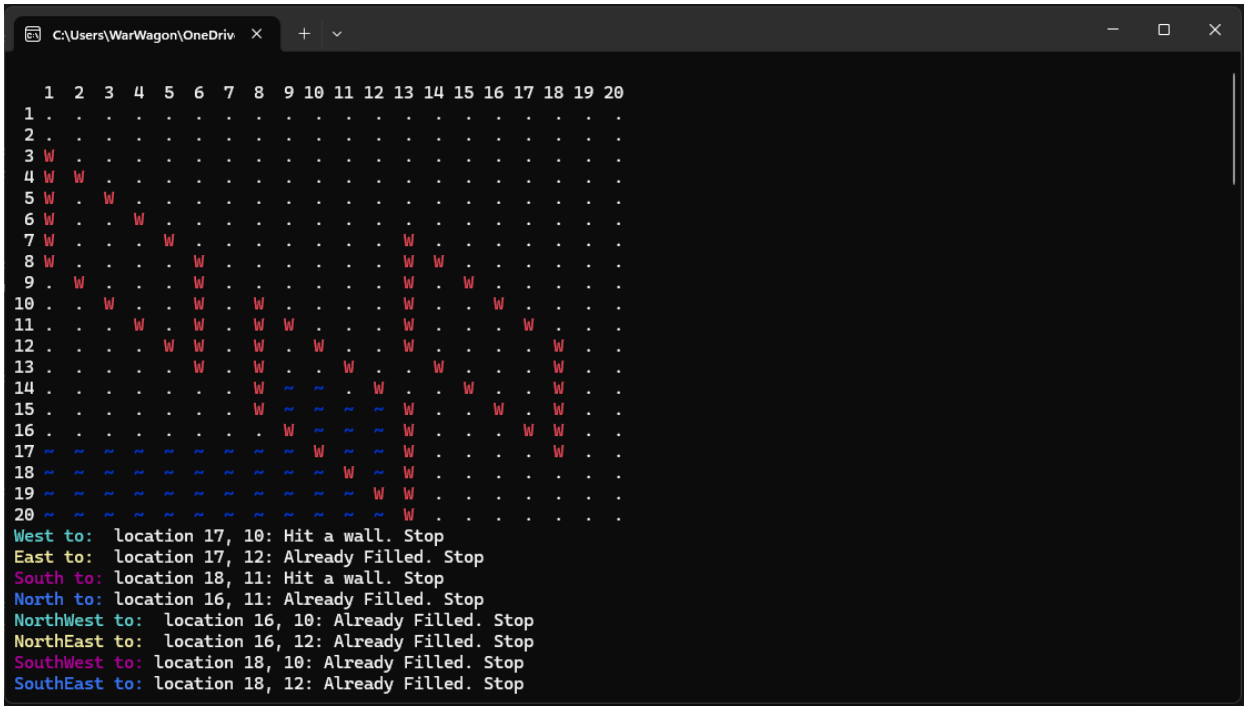
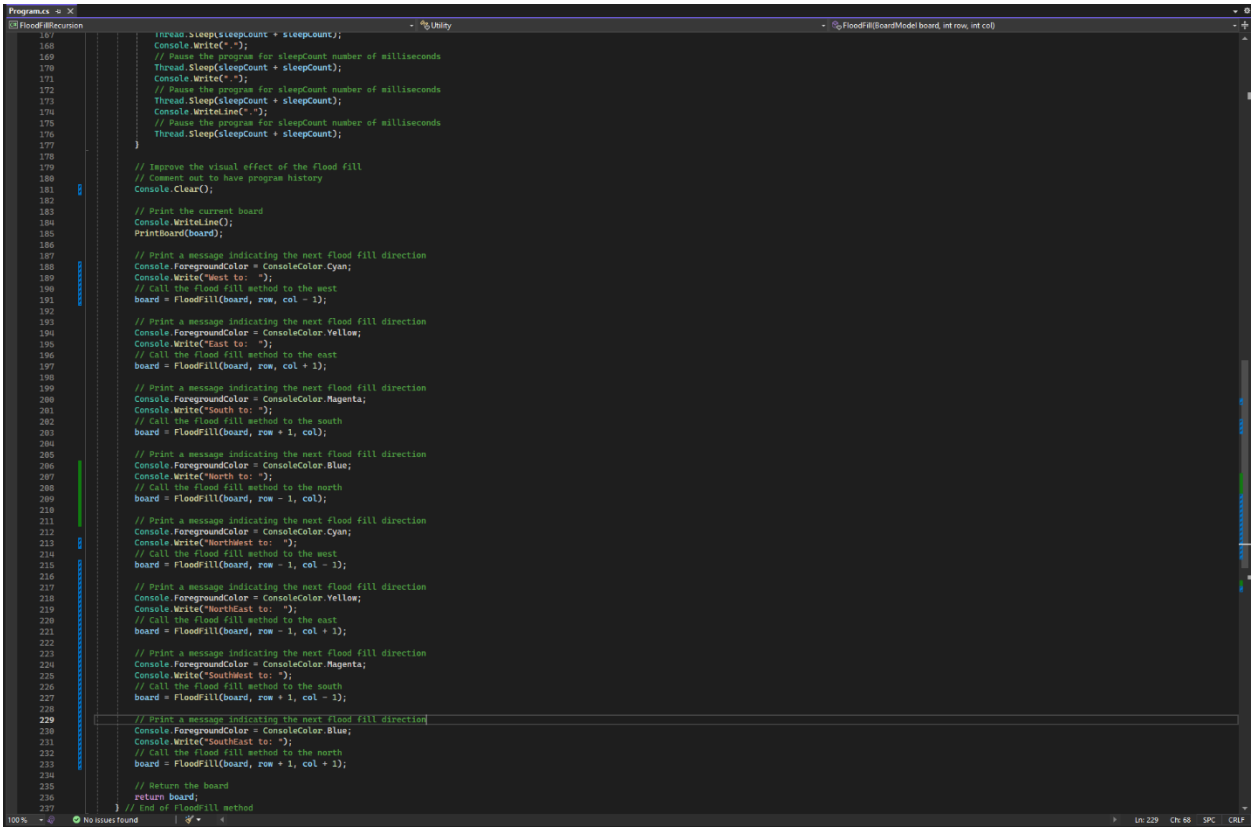


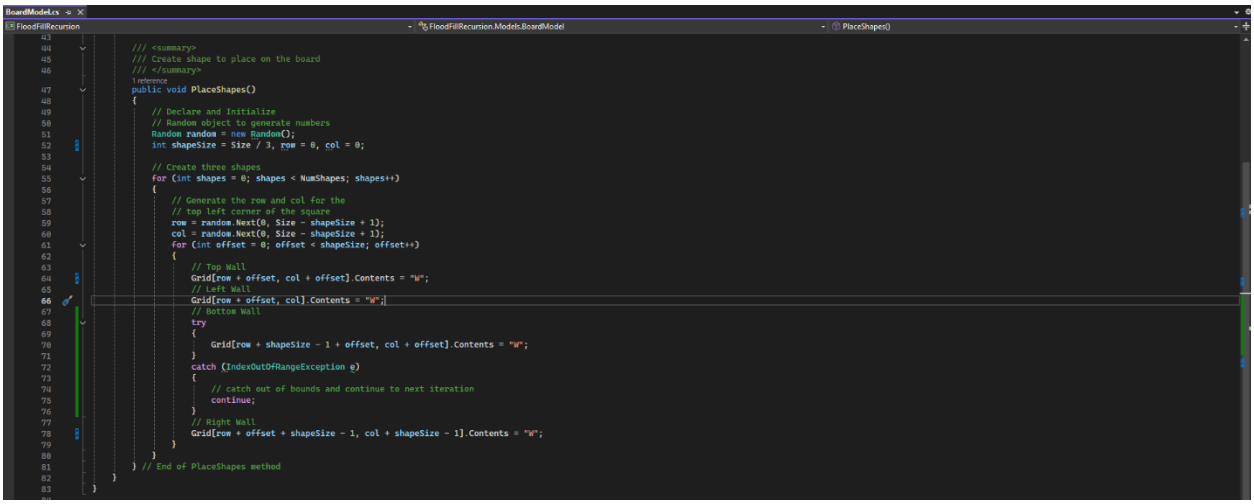
Figure 4-20: Challenge three fill running. Jumping over diagonals happens with calls to NorthWest and the other corners giving eight direction flooding.

Code



```
            Thread.Sleep(sleepCount + sleepCount);
            Console.Write(".");
            // Pause the program for sleepCount number of milliseconds
            Thread.Sleep(sleepCount + sleepCount);
            Console.Write(".");
            // Pause the program for sleepCount number of milliseconds
            Thread.Sleep(sleepCount + sleepCount);
            Console.WriteLine(".");
            // Pause the program for sleepCount number of milliseconds
            Thread.Sleep(sleepCount + sleepCount);
        }

        // Improve the visual effect of the flood fill
        // Comment out to have program history
        Console.Clear();

        // Print the current board
        Console.WriteLine();
        PrintBoard(board);

        // Print a message indicating the next flood fill direction
        Console.ForegroundColor = ConsoleColor.Cyan;
        Console.Write("West to:  ");
        // Call the flood fill method to the west
        board = FloodFill(board, row, col - 1);

        // Print a message indicating the next flood fill direction
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.Write("East to:  ");
        // Call the flood fill method to the east
        board = FloodFill(board, row, col + 1);

        // Print a message indicating the next flood fill direction
        Console.ForegroundColor = ConsoleColor.Magenta;
        Console.Write("South to: ");
        // Call the flood fill method to the south
        board = FloodFill(board, row + 1, col);

        // Print a message indicating the next flood fill direction
        Console.ForegroundColor = ConsoleColor.Blue;
        Console.Write("North to: ");
        // Call the flood fill method to the north
        board = FloodFill(board, row - 1, col);

        // Print a message indicating the next flood fill direction
        Console.ForegroundColor = ConsoleColor.Cyan;
        Console.Write("NorthWest to:  ");
        // Call the flood fill method to the west
        board = FloodFill(board, row - 1, col - 1);

        // Print a message indicating the next flood fill direction
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.Write("NorthEast to:  ");
        // Call the flood fill method to the east
        board = FloodFill(board, row - 1, col + 1);

        // Print a message indicating the next flood fill direction
        Console.ForegroundColor = ConsoleColor.Magenta;
        Console.Write("SouthWest to: ");
        // Call the flood fill method to the south
        board = FloodFill(board, row + 1, col - 1);

        // Print a message indicating the next flood fill direction
        Console.ForegroundColor = ConsoleColor.Blue;
        Console.Write("SouthEast to: ");
        // Call the flood fill method to the north
        board = FloodFill(board, row + 1, col + 1);

        // Return the board
        return board;
    } // End of FloodFill method
```

Figure 4-21: Code changes to flood fill for challenge one and three in the calling different directions section.



```
    /// <summary>
    /// Create shape to place on the board
    /// </summary>
    1 reference
    public void PlaceShapes()
    {
        // Declare and Initialize
        // Random object to generate numbers
        Random random = new Random();
        int shapeSize = Size / 3, row = 0, col = 0;

        // Create three shapes
        for (int shapes = 0; shapes < NumShapes; shapes++)
        {
            // Generate the row and col for the
            // top left corner of the square
            row = random.Next(0, Size - shapeSize + 1);
            col = random.Next(0, Size - shapeSize + 1);
            for (int offset = 0; offset < shapeSize; offset++)
            {
                // Top Wall
                Grid[row + offset, col + offset].Contents = "W";
                // Left Wall
                Grid[row + offset, col].Contents = "W";
                // Bottom Wall
                try
                {
                    Grid[row + shapeSize - 1 + offset, col + offset].Contents = "W";
                }
                catch (IndexOutOfRangeException e)
                {
                    // catch out of bounds and continue to next iteration
                    continue;
                }
                // Right Wall
                Grid[row + offset + shapeSize - 1, col + shapeSize - 1].Contents = "W";
            }
        }
    } // End of PlaceShapes method
}
```

Figure 4-22: Place shapes modifications for making a different shape type

Activity 4 Prep.

Flow Chart

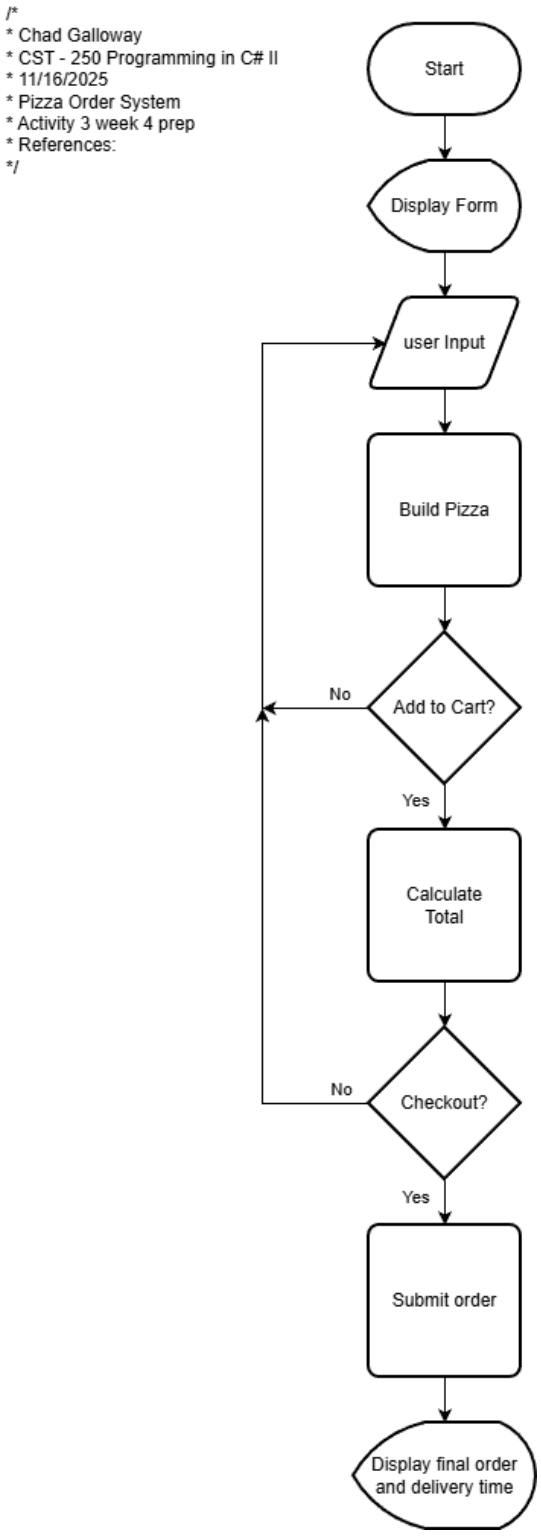


Figure 5-1: Flow chart of Activity 4.

UML Class Diagram

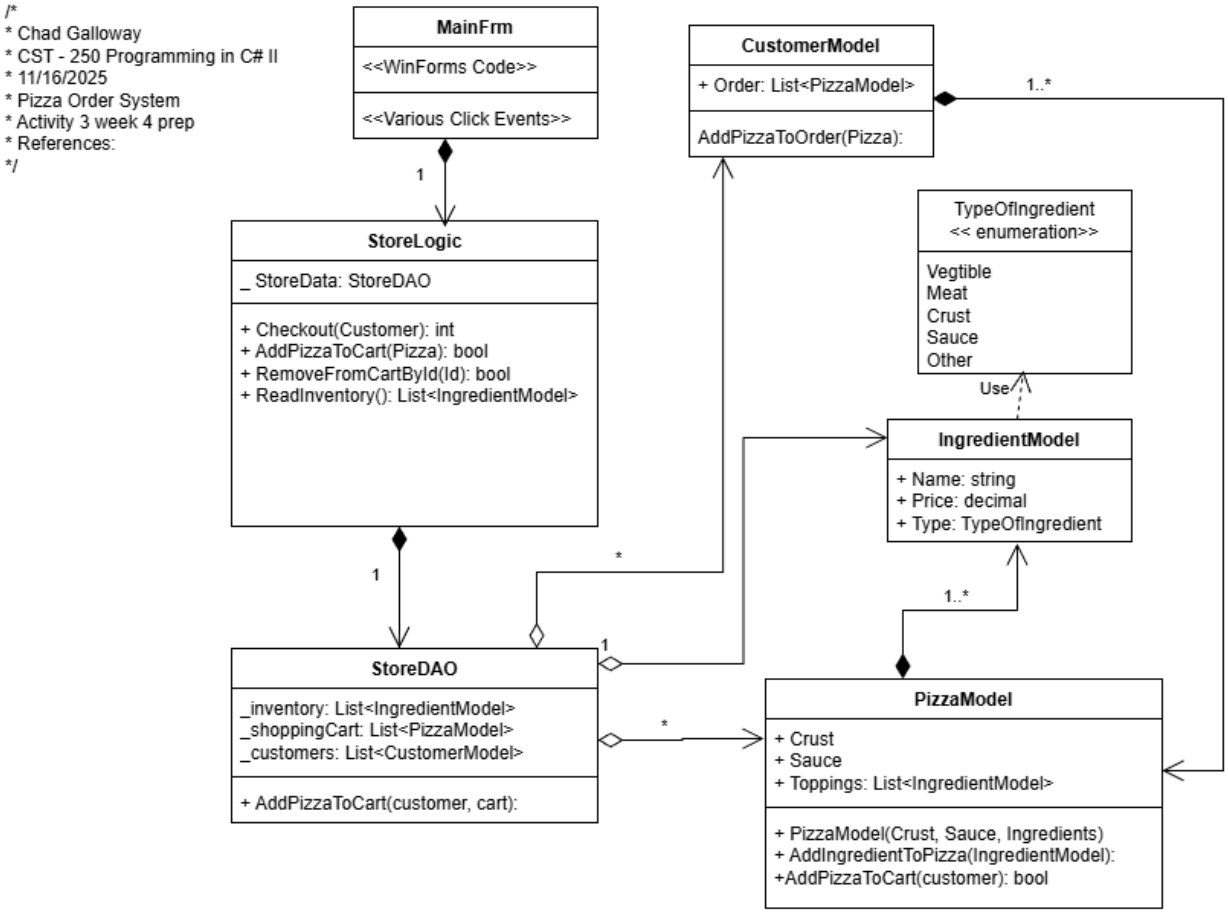


Figure 5-2: UML Class Diagram for Activity 4.